



The Cross-Entropy Method for Combinatorial and Continuous Optimization

REUVEN RUBINSTEIN

e-mail: ierrr01@ie.technion.ac.il

William Davidson Faculty of Industrial Engineering and Management, Technion, Haifa, Israel

Received August 19, 1998; Revised December 23, 1998; Accepted February 10, 1999

Abstract. We present a new and fast method, called the cross-entropy method, for finding the optimal solution of combinatorial and continuous nonconvex optimization problems with convex bounded domains. To find the optimal solution we solve a sequence of simple auxiliary smooth optimization problems based on Kullback-Leibler cross-entropy, importance sampling, Markov chain and Boltzmann distribution. We use importance sampling as an important ingredient for adaptive adjustment of the temperature in the Boltzmann distribution and use Kullback-Leibler cross-entropy to find the optimal solution. In fact, we use the mode of a unimodal importance sampling distribution, like the mode of beta distribution, as an estimate of the optimal solution for continuous optimization and Markov chains approach for combinatorial optimization. In the later case we show almost surely convergence of our algorithm to the optimal solution. Supporting numerical results for both continuous and combinatorial optimization problems are given as well. Our empirical studies suggest that the cross-entropy method has polynomial in the size of the problem running time complexity.

Keywords: combinatorial optimization, global optimization, importance sampling, markov chain monte carlo, simulated annealing, simulation

AMS 1991 Subject Classification: 65C05, 90C27, 94A17.

1. Background

1.1. Introduction

Optimizing the topologies and the configuration of complex large-scale systems such as computer communication systems, reliability systems, manufacturing, robotics and traffic systems is a very difficult task. Most of such optimization problems are multiextremal NP-hard problems. Well known methods for both continuous and discrete optimization problems are simulated annealing (Aarts and Korst (1989), Cohn and Fielding (1999), Geyer and Thompson (1995), Romeijn and Smith (1994)), initiated by Metropolis *et al.* (1953) and later generalized by Hastings (1970) and Kirkpatrick *et al.* (1983), tabu search (Glover and Laguna (1992)) and genetic algorithms (Goldberg (1989)). For a very interesting landmark paper on simulated annealing see Romeijn and Smith (1994). Recent works on combinatorial optimization include the *nested partitioning* (NP) method Shi and Olafsson, the *stochastic comparison method* (Gong *et al.* (1992)), and the method of Andradóttir (1995), (1996). In most of the above methods a Markov chain is constructed and almost sure convergence is proved by analyzing the stationary distribution of the Markov chain.

The basic idea behind the NP method is to systematically partition the feasible region into smaller subregions until some of the subregions contain only one point. The method then moves from one region to another based on information obtained by random sampling. It is shown that the NP algorithm converges to an optimal solution with probability one. The stochastic comparison method is similar to the simulated annealing, but does not require a neighborhood structure. The method of Andradóttir (1995) can be viewed as discrete stochastic approximation method. The method compares two neighboring points in each iteration and moves to the point that is found to be better. This method is shown to converge almost surely to a local optimum. Andradóttir (1996) develops a similar method for finite stochastic optimization which converges almost surely to a global optimum. For additional references on multiextremal and combinatorial optimization see Aarts and Lenstra (1997), Abdullah (1994), Ahuja *et al.* (1993), Ali and Silvey (1966), Cohn and Fielding, Dyer *et al.* (1991), Fox and Heine (1996), Geyer and Thompson (1995), Gilks *et al.* (1996), Horst *et al.* (1996), Jerrum and Sinclair (1989, 1993), Johnson and Kotz (1977), Karp and Luby (1983), Lovasz (1995), Marinari and Parisi (1992), Osman and Laporte (1996), Pinter (1996), Parker and Rardin (1988), Potamianos and Goutsias (1997), Rayward-Smith *et al.* (1996), Reeves (1996), Romeijn and Smith (1994), Rubinstein (1981) and the recent work of Norkin, Pflug and Ruszczyński (1996), where the classical deterministic branch-and-bound method is extended to stochastic problems.

In this work we present a new method, called the *cross-entropy* method for solving both continuous multiextremal and discrete optimization problems. To find the optimal solution we solve a sequence of simple auxiliary smooth optimization problems based on Kullback-Leibler cross-entropy, importance sampling, Markov chain and Boltzmann distribution. More specifically, we use importance sampling as an important ingredient for adaptive adjustment of the temperature in the Boltzmann distribution and use Kullback-Leibler cross-entropy to find the optimal solution. In fact, we use the mode of a unimodal importance sampling distribution, like the mode of beta distribution, as an estimate of the optimal solution for continuous optimization and Markov chains for combinatorial optimization. In the later case we prove almost surely convergence of the proposed algorithms to the optimal solution. Supporting numerical results for both continuous and combinatorial optimization problems indicating polynomial (in the size of the problem) running time complexity are given as well.

Our algorithm employs quite different ideas than the other well known random search algorithms for global optimization including simulated annealing (Aarts and Korst (1989), Geyer and Thompson (1995), Romeijn and Smith (1994)), tabu search (Glover and Laguna (1989)) and genetic algorithms (Goldberg (1989)). These, in fact, represent *local search heuristics* and employ the notion of so-called local neighborhood structures. Although our method, like many others, reconstructs the initial probability measure to make occurrence of the events more likely in the vicinity of a global extremum, it represents, in fact, a global rather than a local random search procedure.

It is not our goal here to compare the efficiency of the proposed method with many other well established alternatives, such as simulated annealing, tabu search and genetic algorithms. Our goal is quite modest: to show that the proposed method has good potentials and to promote it for further applications. Before proceeding further we need the following background.

1.2. Estimation of Probabilities of Rare Events

Let ℓ be the expected performance of a stochastic system given in the form

$$\ell(\mathbf{v}) = \mathbb{E}_f\{\varphi[L(\mathbf{Y})]\} = \int \varphi[L(\mathbf{y})]f(\mathbf{y})d\mathbf{y}, \quad (1.1)$$

where $L(\mathbf{Y})$ is the *sample performance* and the subscript f in $\mathbb{E}_f\{\varphi[L(\mathbf{Y})]\}$ means that the expectation of the random vector $\mathbf{Y}$ is taken with respect to the measure (probability density function) f , and φ is a real valued function.

Examples of $\varphi[L(\mathbf{y})]$ are

$$\varphi[L(\mathbf{y})] = I_{\{L(\mathbf{y}) < x\}} \quad (1.2)$$

and

$$\varphi[L(\mathbf{y})] = \exp^{-\frac{L(\mathbf{y})}{\gamma}}, \quad (1.3)$$

where x and γ are given constants.

As an example of $L(\mathbf{Y})$ consider the shortest path in a network, which can be defined as follows

$$L(\mathbf{Y}) = \min_{j=1,\dots,p} \sum_{i \in \mathcal{B}_j} Y_i. \quad (1.4)$$

Here, $\mathcal{B}_j$ is the j -th complete path from a source to a sink; p is the number of complete paths; and Y_i , $i = 1, \dots, m$, is the duration of the i -th component of the random vector $\mathbf{Y}$. For the longest path we replace min with max in (1.4).

Let $G(\mathbf{y})$ be a probability measure (distribution) such that $dG(\mathbf{y}) = g(\mathbf{y})d\mathbf{y}$, where $g(\mathbf{y})$ is a pdf. Assume that $g(\mathbf{y})$ dominates $f(\mathbf{y})$ in the absolutely continuous sense, that is

$$\text{supp}\{f(\mathbf{y})\} \subset \text{supp}\{g(\mathbf{y})\}. \quad (1.5)$$

Using the pdf g we can represent ℓ as

$$\ell = \int \varphi[L(\mathbf{z})] \frac{f(\mathbf{z})}{g(\mathbf{z})} g(\mathbf{z})d\mathbf{z} = \mathbb{E}_g \left\{ \varphi[L(\mathbf{Z})] \frac{f(\mathbf{Z})}{g(\mathbf{Z})} \right\}, \quad (1.6)$$

where the subscript g means that the expectation is taken with respect to the probability measure (pdf) g , which is called the *importance sampling* (IS) pdf.

An unbiased estimator of ℓ is

$$\bar{\ell}_N = \frac{1}{N} \sum_{i=1}^N \varphi[L(\mathbf{Z}_i)] W(\mathbf{Z}_i), \quad (1.7)$$

where $W(\mathbf{z}) = f(\mathbf{z})/g(\mathbf{z})$ is called the *likelihood ratio* (LR), and $\mathbf{Z}_1, \dots, \mathbf{Z}_N$ is a random sample from $g(\mathbf{z})$. In the particular case where there is no change of measure ($g = f$), we have $W = 1$, and the LR estimator $\bar{\ell}_N$ reduces to the following, the so-called *crude Monte Carlo* (CMC) estimator:

$$\tilde{\ell}_N = \frac{1}{N} \sum_{i=1}^N \varphi[L(\mathbf{Y}_i)] , \quad (1.8)$$

where $\mathbf{Y}_1, \dots, \mathbf{Y}_N$ is a random sample from the pdf $f(\mathbf{y})$.

The choice of the dominating pdf $g(\mathbf{y})$ is crucial for the variance of the LR estimator (1.7), and we consider next the problem of minimizing the variance of $\tilde{\ell}_N$ with respect to the pdf g , that is

$$\min_g \text{Var}_g \left\{ \varphi[L(\mathbf{Z})] \frac{f(\mathbf{Z})}{g(\mathbf{Z})} \right\}. \quad (1.9)$$

It is well known (e.g. Rubinstein and Melamed (1998)) that the solution of the problem (1.9) is

$$g^*(\mathbf{z}) = \frac{|\varphi[L(\mathbf{z})]| f(\mathbf{z})}{\int |\varphi[L(\mathbf{z})]| f(\mathbf{z}) d\mathbf{z}}. \quad (1.10)$$

Note that if $\varphi[L(\mathbf{Z})] \geq 0$, then

$$g^*(\mathbf{z}) = \frac{\varphi[L(\mathbf{z})] f(\mathbf{z})}{\ell} \quad (1.11)$$

and $\text{Var}\{\tilde{\ell}_N\} = 0$. The density $g^*(\mathbf{z})$ as per (1.10) and (1.11) is called the *importance sampling density*.

In general, however, implementation of the importance sampling pdf $g^*(\mathbf{z})$ as per (1.10) and (1.11) is problematic. The main difficulty lies in the fact that knowledge of $g(\mathbf{z})$ implies knowledge (availability) of ℓ . But ℓ is precisely the quantity we want to estimate from the simulation! In most simulation studies the situation is even worse, since the analytical expression for the sample performance L is unknown in advance. To overcome this difficulty one can make a pilot run with the underlying model, obtain a sample $L(\mathbf{Y}_1), \dots, L(\mathbf{Y}_N)$, and then use it to estimate (approximate) the importance sampling pdf $g(\mathbf{z})$. It is important to note that sampling from such an artificially constructed pdf $g(\mathbf{z})$ might be a very complicated and time-consuming task, especially when $g(\mathbf{z})$ is a high-dimensional pdf.

Assume below that $g(\mathbf{y}) = f(\mathbf{y}, \mathbf{v}_0)$, that is $g(\mathbf{y})$ comes from the *same parametric family* as the distributions $f(\mathbf{y}) = f(\mathbf{y}, \mathbf{v})$, $\mathbf{v} \in V$ does. The parameter vector $\mathbf{v}_0$ ($\mathbf{v}_0 \neq \mathbf{v}$) is called the *reference* parameter. In this case, the likelihood ratio W in (1.7) reduces to

$$W(\mathbf{Z}, \mathbf{v}, \mathbf{v}_0) = \frac{f(\mathbf{Z}, \mathbf{v})}{f(\mathbf{Z}, \mathbf{v}_0)},$$

and instead of Equation (1.9) we can consider the following *simpler* one

$$\min_{\mathbf{v}_0} \text{Var}_{\mathbf{v}_0} \{ \varphi[L(\mathbf{Z})] W(\mathbf{Z}, \mathbf{v}, \mathbf{v}_0) \}, \quad (1.12)$$

which is the same as

$$\min_{\mathbf{v}_0} \{ \mathcal{V}(\mathbf{v}_0) = \mathbb{E}_{\mathbf{v}_0} \{ \varphi^2[L(\mathbf{Z})] W(\mathbf{Z}, \mathbf{v}, \mathbf{v}_0)^2 \} \} \quad (1.13)$$

or

$$\min_{\mathbf{v}_0} \{ \mathcal{V}(\mathbf{v}_0) = \mathbb{E}_{\mathbf{v}} \{ \varphi^2[L(\mathbf{Y})]W(\mathbf{Y}, \mathbf{v}, \mathbf{v}_0) \} \}, \quad (1.14)$$

where $\mathbf{Y} \sim f(\mathbf{y}, \mathbf{v})$.

Note that the optimal solution of (1.13) and (1.14), say $\mathbf{v}_0^*$, is typically not available analytically since the variance of $\varphi[L(\mathbf{Z})]W(\mathbf{Z}, \mathbf{v}, \mathbf{v}_0)$ is so.

To overcome this difficulty we can minimize instead a *stochastic counterpart* of (1.13), namely

$$\min_{\mathbf{v}_0} \left\{ \bar{\mathcal{V}}_N(\mathbf{v}_0) = N^{-1} \sum_{i=1}^N \varphi^2[L(\mathbf{Z}_i)]W^2(\mathbf{Z}_i, \mathbf{v}, \mathbf{v}_0) \right\}, \quad (1.15)$$

or a stochastic counterpart of (1.14)

$$\min_{\mathbf{v}_0} \left\{ \widehat{\mathcal{V}}_N(\mathbf{v}) = N^{-1} \sum_{i=1}^N \varphi^2[L(\mathbf{Y}_i)]W(\mathbf{Y}_i, \mathbf{v}, \mathbf{v}_0) \right\}, \quad (1.16)$$

and then take the optimal solution of (1.15) or (1.16), as an estimator of $\mathbf{v}_0^*$, (see Rubinstein and Shapiro (1993) for more details). Note that in (1.15) the random sample $\mathbf{Z}_1, \dots, \mathbf{Z}_N$ is generated from the pdf $f(\mathbf{z}, \mathbf{v}_0)$, while in (1.16) the random sample $\mathbf{Y}_1, \dots, \mathbf{Y}_N$ is generated from the original pdf $f(\mathbf{y}, \mathbf{v})$.

An alternative way for estimating the optimal reference parameter vector $\mathbf{v}_0^*$ is based on Kullback–Liebler relative entropy (Kapur and Kesavan (1992)), or *cross-entropy* (CE), which defines a “distance” between the two probability distributions (densities) $f(\mathbf{y})$ and $g(\mathbf{y})$ and can be written as

$$\mathcal{D} = \int f(\mathbf{y}) \ln \frac{f(\mathbf{y})}{g(\mathbf{y})} d\mathbf{y}. \quad (1.17)$$

Clearly, when $g = f$ we have $d = 0$. Note that both CE and importance sampling are particular cases of Ali–Silvey “distance” (Ali and Silvey (1966)) between two probability densities, which is defined as

$$d(f, g) = \aleph \left[\int f(\mathbf{y}) \psi \left(\frac{f(\mathbf{y})}{g(\mathbf{y})} \right) d\mathbf{y} \right], \quad (1.18)$$

where $\aleph(\cdot)$ is a continuous convex function on $(0, +\infty)$ and $\psi(\cdot)$ is increasing, real-valued function of a real variable. Indeed, in the case for the cross-entropy and importance sampling with $g(\mathbf{y}) = c^{-1} \varphi[L(\mathbf{y})]f(\mathbf{y})$, ($c = \int \varphi[L(\mathbf{y})]f(\mathbf{y})d\mathbf{y}$) we have $\psi(y) = \ln(y)$, $\aleph(y) = y$ and $\psi(y) = y^2$, $\aleph(y) = y$, respectively.

Let $g(\mathbf{y}) = f(\mathbf{y}, \mathbf{v}_0)$ and assume that $\varphi[L(\mathbf{y})] \geq 0$. Taking into account that $\phi(\mathbf{y}, \mathbf{v}) = c^{-1} \varphi[L(\mathbf{y})]f(\mathbf{y}, \mathbf{v})$, ($c = \int \varphi[L(\mathbf{y})]f(\mathbf{y}, \mathbf{v})d\mathbf{y}$) is a pdf, we can define a cross-entropy between $\phi(\mathbf{y}, \mathbf{v})$ and $f(\mathbf{y}, \mathbf{v}_0)$, in analogy to (1.17) as

$$\begin{aligned}\mathcal{D}(\mathbf{v}_0) &= \int \varphi[L(\mathbf{y})]f(\mathbf{y}, \mathbf{v}) \ln \frac{\varphi[L(\mathbf{y})]f(\mathbf{y}, \mathbf{v})}{f(\mathbf{y}, \mathbf{v}_0)} d\mathbf{y} \\ &= \mathbb{E}_{\mathbf{v}} \left\{ \varphi[L(\mathbf{Y})] \ln \frac{\varphi[L(\mathbf{Y})]f(\mathbf{Y}, \mathbf{v})}{f(\mathbf{Y}, \mathbf{v}_0)} \right\},\end{aligned}\quad (1.19)$$

and find the optimal reference parameter vector $\mathbf{v}_0^*$ as a solution of the following problem

$$\min_{\mathbf{v}_0} \left[\mathcal{D}(\mathbf{v}_0) = \mathbb{E}_{\mathbf{v}} \left\{ \varphi[L(\mathbf{Y})] \ln \frac{\varphi[L(\mathbf{Y})]f(\mathbf{Y}, \mathbf{v})}{f(\mathbf{Y}, \mathbf{v}_0)} \right\} \right]. \quad (1.20)$$

It is obvious that the optimal solutions of (1.20) and of

$$\max_{\mathbf{v}_0} [D(\mathbf{v}_0) = \mathbb{E}_{\mathbf{v}} \{ \varphi[L(\mathbf{Y})] \ln f(\mathbf{Y}, \mathbf{v}_0) \}] \quad (1.21)$$

are equivalent.

Given a sample $\mathbf{Y}_1, \dots, \mathbf{Y}_N$ from $f(\mathbf{y}, \mathbf{v})$, we can estimate the optimal solution $\mathbf{v}_0^*$ of (1.21) by the optimal solution of the program

$$\max_{\mathbf{v}_0} \left\{ \widehat{D}_N(\mathbf{v}, \mathbf{v}_0) = \frac{1}{N} \sum_{i=1}^N \varphi[L(\mathbf{Y}_i)] \ln f(\mathbf{Y}_i, \mathbf{v}_0) \right\}. \quad (1.22)$$

Note that the Equations (1.21) and (1.22), based on CE can be considered as alternatives to the Equations (1.14), (1.16), respectively, based on likelihood ratios.

In this section we shall concentrate mainly on estimation of the the optimal reference parameter vector $\mathbf{v}_0^*$ and $\ell(x)$, when $\ell(x)$ is given in the form $\ell(x) = P\{L > x\} = \mathbb{E}I_{\{L > x\}}$ ($\varphi(L) = I_{\{L > x\}}$), that is represents the probability of the rare event $I_{\{L > x\}}$. In this case the basic formulae Equations (1.7), (1.14), (1.16), (1.21) and (1.22) become:

$$\bar{\ell}_N(\mathbf{v}, \mathbf{v}_0) = \frac{1}{N} \sum_{i=1}^N I_{\{L(\mathbf{Z}_i) > x\}} W(\mathbf{Z}_i, \mathbf{v}, \mathbf{v}_0), \quad (1.23)$$

$$\min_{\mathbf{v}_0} \{ \mathcal{V}(\mathbf{v}, \mathbf{v}_0) = \mathbb{E}_{\mathbf{v}} \{ I_{\{L(\mathbf{Y}) > x\}} W(\mathbf{Y}, \mathbf{v}, \mathbf{v}_0) \} \}, \quad (1.24)$$

$$\min_{\mathbf{v}_0} \left\{ \widehat{\mathcal{V}}_N(\mathbf{v}, \mathbf{v}_0) = \frac{1}{N} \sum_{i=1}^N I_{\{L(\mathbf{Y}_i) > x\}} W(\mathbf{Y}_i, \mathbf{v}, \mathbf{v}_0) \right\}, \quad (1.25)$$

$$\max_{\mathbf{v}_0} \{ D(\mathbf{v}, \mathbf{v}_0) = \mathbb{E}_{\mathbf{v}} \{ I_{\{L(\mathbf{Y}) > x\}} \ln f(\mathbf{Y}, \mathbf{v}_0) \} \}, \quad (1.26)$$

$$\max_{\mathbf{v}_0} \left\{ \widehat{D}_N(\mathbf{v}, \mathbf{v}_0) = \frac{1}{N} \sum_{i=1}^N I_{\{L(\mathbf{Y}_i) > x\}} \ln f(\mathbf{Y}_i, \mathbf{v}_0) \right\}, \quad (1.27)$$

respectively, whereas before

$$W(\mathbf{Z}, \mathbf{v}, \mathbf{v}_0) = \frac{f(\mathbf{Z}, \mathbf{v})}{f(\mathbf{Z}, \mathbf{v}_0)},$$

$\mathbf{Z}_1, \dots, \mathbf{Z}_N$ and $\mathbf{Y}_1, \dots, \mathbf{Y}_N$ are random samples from the pdf's $f(\mathbf{z}, \mathbf{v}_0)$ and $f(\mathbf{z}, \mathbf{v})$, respectively.

Lieber (1999) investigates the statistical properties of the likelihood ratio (LR) estimate (1.23) when $\mathbf{v}_0$ represents the solution of Equations (1.25) and (1.27).

If not stated otherwise we shall use below the CE approach.

It is readily seen that Equations (1.26) and (1.27) are useless for estimating $\mathbf{v}_0^*$ in a rare-event context, since owing to the rarity of the events $\{L(\mathbf{Y}_i) > x\}$, the random variables $I_{\{L(\mathbf{Y}_i) > x\}}$, $i = 1, \dots, N$ and the associated derivatives of $\widehat{D}_N(\mathbf{v}, \mathbf{v}')$ at $\mathbf{v}' = \mathbf{v}$ vanish with high probability, provided the sample N is small relative to the reciprocal of the rare event probability $\ell(x)$. To overcome this difficulty we introduce below an auxiliary sequence of vectors $(\gamma_t^*, \mathbf{v}_t^*)$, $t \geq 0$. We start by choosing an initial γ_1 ($\gamma_1 < x$), such that under the original pdf $f(\mathbf{y}, \mathbf{v})$, the probability $\ell(\gamma_1) = \mathbb{E}_{\mathbf{v}}\{I_{\{L > \gamma_1\}}\}$ is not too small, say $\ell(\gamma_1) = \rho \approx 10^{-2}$. We set next $\mathbf{v}_0^* \equiv \bar{\mathbf{v}}_0^* \equiv \mathbf{v}$ and then iterate in both $\mathbf{v}$ and γ as follows.

(a) **Adaptive estimation of $\mathbf{v}_t^*$.** For fixed γ_t we derive $\mathbf{v}_t^*$ from the solution of the program

$$\max_{\mathbf{v}_t} D(\gamma_t, \mathbf{v}_{t-1}^*, \mathbf{v}_t) = \max_{\mathbf{v}_t} \mathbb{E}_{\mathbf{v}_{t-1}^*} \{I_{\{L(\mathbf{Y}) < \gamma_t\}} \ln f(\mathbf{Y}, \mathbf{v}_t)\}, \quad (1.28)$$

where $\mathbf{Y} \sim f(\mathbf{y}, \mathbf{v}_{t-1}^*)$ and $\mathbf{v}_0^* \equiv \mathbf{v}$.

The stochastic counterpart of (1.28) can be written as

$$\max_{\mathbf{v}_t} \widehat{D}_N(\gamma_t, \bar{\mathbf{v}}_{t-1}^*, \mathbf{v}_t) = \max_{\mathbf{v}_t} \left\{ \frac{1}{N} \sum_{j=1}^N I_{\{L(\mathbf{Z}_j) < \gamma_t\}} \ln f(\mathbf{Z}_j, \mathbf{v}_t) \right\}, \quad (1.29)$$

where $\mathbf{Z}_j \sim f(\mathbf{y}, \bar{\mathbf{v}}_{t-1}^*)$ and $\bar{\mathbf{v}}_0^* \equiv \mathbf{v}$.

(b) **Adaptive estimation of γ_t .** For a given $\mathbf{v}_{t-1}^*$, the optimal γ_t^* can be obtained from the following simple one-dimensional root-finding program (see also (1.28))

$$\min \gamma_t \text{ s.t. } \mathbb{E}_{\mathbf{v}_{t-1}^*} \{I_{\{L(\mathbf{Y}) < \gamma_t\}}\} \geq \rho, \quad (1.30)$$

where, say $10^{-2} \leq \rho \leq 10^{-1}$. The stochastic counterpart of (1.30) is

$$\min \gamma_t \text{ s.t. } \left\{ \frac{1}{N} \sum_{j=1}^N I_{\{L(\mathbf{Z}_j) < \gamma_t\}} \right\} \geq \rho. \quad (1.31)$$

The resulting algorithm for estimating $\ell(\gamma)$ can be written as

ALGORITHM 1.1

1. Generate a sample $\mathbf{Z}_1, \dots, \mathbf{Z}_N$ from the pdf $f(\mathbf{y}, \bar{\mathbf{v}}_0^*)$, $\bar{\mathbf{v}}_0^* \equiv \mathbf{v}$ and solve Equation (1.31). Denote the initial solution by $\bar{\gamma}_1^*$.
2. Use the **same** $\mathbf{Z}_1, \dots, \mathbf{Z}_N$ and solve the stochastic Equation (1.29) for $\gamma = \bar{\gamma}_1^*$ and $\bar{\mathbf{v}}_0^* \equiv \mathbf{v}$. Denote the solution by $\bar{\mathbf{v}}_1^* = \bar{\mathbf{v}}_1^*(\bar{\gamma}_1^*)$.

3. Generate a **new** sample $\mathbf{Z}_1, \dots, \mathbf{Z}_N$ from the pdf $f(\mathbf{y}, \bar{\mathbf{v}}_1^*)$ and repeat Steps 1 and 2, again with $\bar{\mathbf{v}}_0^*$ replaced by $\bar{\mathbf{v}}_1^*$. Denote the corresponding solutions by $\bar{\gamma}_2^*$ and $\bar{\mathbf{v}}_2^*$, respectively. Denote the solutions at stage t by $\bar{\gamma}_t^*$ and $\bar{\mathbf{v}}_t^*$, respectively. It is readily seen that, in fact,

$$\bar{\gamma}_t^* = L_{(\lceil(1-\rho)N\rceil)},$$

where $L_{(j)}$ is the j -th order statistics of the sequence $L_j \equiv L(\mathbf{Z}_j)$, $j = 1, \dots, N$. For $\rho = 10^{-2}$ this reduces to

$$\bar{\gamma}_t^* = L_{(\lceil \frac{99}{100} N \rceil)}.$$

4. If $\bar{\gamma}_t^* \geq \gamma$, set $\bar{\gamma}_t^* \equiv \gamma$ and solve the stochastic Equation (1.29) for $\bar{\gamma}_t^* = \gamma$. Denote the solution as $\bar{\mathbf{v}}_{t+1}^*$ and stop; otherwise set $t \equiv t + 1$ and repeat step 3. After stopping:
- Estimate the rare event probability $\ell(x)$ using the LR estimate (1.23), with $\mathbf{v}_0$ replaced by $\bar{\mathbf{v}}_{t+1}^*$.

The monotonicity of the sequence $\bar{\gamma}_0^*, \bar{\gamma}_1^*, \dots$ is an inherent part of Algorithm 1.1. Under rather mild regularity conditions it is shown in Lieber (1998) that the random sequence $\bar{\gamma}_0^*, \bar{\gamma}_1^*, \dots$ generated by the Algorithm 1.1 is monotonically nondecreasing and that the target value γ can be reached with high probability in a finite number of iterations.

To clarify Algorithm 1.1, suppose we want to estimate the probability $P\{Y > x\}$ for large x , say $x = (16, 64, 256)$ (corresponding to $\ell(x) \approx (10^{-7}, 10^{-28}, 10^{-111})$) with $Y \sim \exp(v)$, $v = 1$. If we take the initial value $\gamma_0 = 4$ ($\rho \approx 2 \times 10^{-2}$), then Algorithm 1.1 terminates on the average after (2, 3, 4) iterations, respectively. If, however, we take $\gamma_0 = 7$ ($\rho \approx 10^{-3}$) and the same values of x , then Algorithm 1.1 terminates on the average after (2, 3, 3) iterations.

Figure 1.1 depicts a possible three-stage realization of Algorithm 1.1 for estimating $\ell = P\{Y > x\}$, $Y \sim \text{Exp}(v)$.

To estimate $\mathbf{v}_0^*$ using the LR method we need to replace (1.29) in Algorithm 1.1 by

$$\min_{\mathbf{v}_t} \widehat{\mathcal{V}}_N(\gamma_t, \bar{\mathbf{v}}_{t-1}^*, \mathbf{v}_t) = \min_{\mathbf{v}_t} \left\{ \frac{1}{N} \sum_{j=1}^N I_{\{L(\mathbf{Z}_j) < \gamma_t\}} W(\mathbf{Z}_j, \bar{\mathbf{v}}_{t-1}^*, \mathbf{v}_t) \right\}, \quad (1.32)$$

while all other data remain the same.

It is worth mentioning that there is no need to solve Equations (1.29) and (1.32) accurately for all stages of Algorithm 1.1, except for the final one since our objective at each stage k is to find $\bar{\mathbf{v}}_k^*$, such that at each iteration k , the random variables $I_{\{L(\mathbf{Z}_i) > \gamma_k\}}$, $i = 1, \dots, N$ generated from the pdf $f(\mathbf{y}, \bar{\mathbf{v}}_k^*)$ do not vanish.

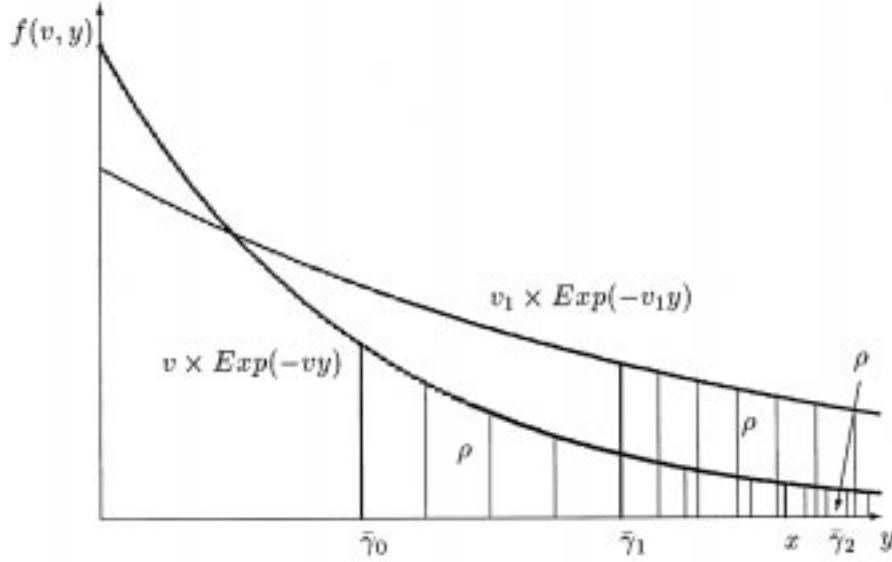


Figure 1.1. A three-stage realization of Algorithm 1.1.

REMARK 1.1 Note that the optimal solution of the single-iteration Equations (1.25) and (1.27) differ generally from their multiple-iteration counterparts (1.29) and (1.32), respectively. The proof of convergence of the estimate $\bar{\mathbf{v}}_i^*$ (of the optimal solution of (1.29)) to the true parameter $\mathbf{v}_0^*$ (of the optimal solution of (1.26)) regardless of the choice of γ_1 is still an open problem.

We present next a theorem due to Lieber (1999) stating that the sequence $\{\gamma_k\}$, $k = 1, 2, \dots$ reaches x in a finite number of iterations.

To do so, let h be the mapping which corresponds to a stage of Algorithm 1.1, i.e. $h : \mathbf{v}_k \mapsto \mathbf{v}_{k+1}$, $k = 1, 2, \dots$ and $\Delta\gamma(\mathbf{v}) \equiv \gamma(h(\mathbf{v})) - \gamma(\mathbf{v})$.

THEOREM 1.1 Assume that the following conditions hold

1. The sequence $\{\gamma_k = \gamma(\mathbf{v}_k)\}$, $k = 1, 2, \dots$ is monotonically increasing.
2. The mapping $\mathbf{v} \mapsto \gamma(\mathbf{v})$ is continuous.
3. The mapping $\mathbf{v} \mapsto \gamma(\mathbf{v})$ is proper, i.e. if $\gamma(\mathbf{v})$ belongs to some closed interval then $\mathbf{v}$ belongs to a compact.
4. The mapping $\mathbf{v} \mapsto \Delta\gamma(\mathbf{v})$ is lower semi-continuous.

Then

1. $\lim_{N \rightarrow \infty} P\{\bar{\gamma}_1 < \bar{\gamma}_2 < \dots < \bar{\gamma}_k < \dots < x\} = 1$.
2. $\exists k < \infty : \lim_{N \rightarrow \infty} P\{\bar{\gamma}_k < x\} = 0$.

Proof: Given in Lieber (1998). ■

1.3. The Root-Finding Problem

In many practical situations one needs to estimate the root x of the non-linear equation

$$\ell = P\{L > x\} = \mathbb{E}I_{\{L > x\}} \quad (1.33)$$

(for given ℓ) rather than to estimate ℓ itself. The solution of (1.33) is quite straightforward: γ_t^* , the optimizer of (1.31), can be taken, in fact, in Algorithm 1.1 as the estimate of the root of $\ell = P\{L > x\}$. The entire Algorithm 1.1 remains the same, provided that Step 4 is replaced by 4*. If $\bar{\ell}_t \equiv \bar{\ell}_N(\mathbf{v}, \bar{\mathbf{v}}_t^*) \leq \ell$, where $\bar{\ell}_N$ is given in (1.23), but with $\mathbf{v}_0$ replaced by $\bar{\mathbf{v}}_t^*$, then solve with respect to γ , the following non-linear equation

$$\frac{1}{N} \sum_{i=1}^N I_{\{L(\mathbf{Z}_i) > \gamma\}} W(\mathbf{Z}_i, \mathbf{v}, \bar{\mathbf{v}}_t^*) = \ell, \quad (1.34)$$

where $\mathbf{Z}_i \sim f(\mathbf{z}, \bar{\mathbf{v}}_t^*)$. Denote the solution as $\bar{\gamma}_{t+1}^*$ and stop; otherwise set $t \equiv t + 1$ and repeat step 3 again.

As an alternative to the estimate $\bar{\gamma}_{t+1}^*$ of (1.34), one can estimate x by using the following linear combination of $\bar{\ell}_N(\mathbf{v}, \mathbf{v}_{t-1}^*)$ and $\bar{\ell}_N(\mathbf{v}, \mathbf{v}_t^*)$:

$$\hat{\gamma}_{t+1}^* = \bar{\gamma}_{t-1}^* + (\bar{\gamma}_t^* - \bar{\gamma}_{t-1}^*) \frac{\bar{\ell}_N(\mathbf{v}, \mathbf{v}_{t-1}^*) - \ell}{\bar{\ell}_N(\mathbf{v}, \mathbf{v}_{t-1}^*) - \bar{\ell}_N(\mathbf{v}, \mathbf{v}_t^*)}. \quad (1.35)$$

The asymptotically properties of the estimate $\bar{\gamma}_t^*$ of x are similar to that of the LR estimate $\bar{\ell}_N(\mathbf{v}, \mathbf{v}_t^*)$ of ℓ and can be obtained directly from Theorem 2.4.1 of Rubinstein and Shapiro (1993).

The rest of our work is organized as follows. In Section 2 we treat continuous multiextremal problems. More specifically, we show that the global minimum $\mathbf{x}^*$ of $L(\mathbf{x}^*)$ can be approximated by solving a sequence of simple auxiliary *continuous and smooth* optimization problems associated with the partition function, Kullback-Leibler cross-entropy and importance sampling and present our main Algorithm 2.1 below. We use the importance sampling, and Kullback-Leibler cross-entropy (see e.g., Kapur and Kesavan (1992) as major ingredients in our algorithm. Section 3 presents application of the cross-entropy for the *shortest (longest) path* (SLP) in network and the *traveling salesman problem* (TSP). In Section 4 we explain intuitively why our algorithm works. Supporting numerical results are given in Section 5, concluding remarks are given in Section 6 and some auxiliary results are presented in Appendix.

2. Continuous Multiextremal Optimization Via Cross-Entropy

2.1. Background

Consider the following minimization problem

$$\min_{\mathbf{x}} L(\mathbf{x}), \quad \mathbf{x} \in \mathcal{X} \subset R^n, \quad (2.1)$$

where $\mathcal{X}$ is a convex bounded domain, $L(\mathbf{x})$ a bounded nonconvex, multiextremal function defined on $\mathcal{X}$. We also allow for either $L(\mathbf{x})$ or $\mathcal{X}$ or both to have discrete (combinatorial) structure (see Sections 3 and 4 below).

We present a fast randomized adaptive algorithm for solving the program (2.1) in the sense that we show numerically that it converges *with very high probability to a very small neighborhood*

$$\{\mathcal{X}^*(\zeta)\} = \{\mathbf{x} : L(\mathbf{x}) < y(\zeta)\}, \quad y(\zeta) = L(\mathbf{x}^*) + \zeta \quad (2.2)$$

of the optimal solution $\mathbf{x}^*$ of Equation (2.1), where ζ is a very small parameter and similarly for the discrete case. We say that Equation (2.1) is solved if the proposed algorithm hits $\mathcal{X}^*(\zeta)$ with very high probability. Our algorithm assumes that $\mathcal{X}$ is a relatively simple region in the sense that one can sample easily on it. For both *discrete* and *continuous* optimizations, the algorithm employs a sequence of *continuous and smooth* optimization problems associated with estimation of certain small quantities, called the *Boltzmann constant* or *partition function* (PF), see (2.6) below. It presents, in fact, a modified version of Algorithm 2.2 of Lieber (1997), and of Algorithm 9.4.1 of Rubinstein (1998), p. 327 for estimating probabilities of rare events in stochastic networks. More specifically, we

1. Break down the “hard” problem, of estimating a very small quantity, the partition function, into a sequence of associated “simple” continuous and smooth optimization problems (see Algorithm 2.1 below).
2. Solve the sequence of “simple” problems based on Kullback-Leibler cross-entropy and importance sampling.
3. Take the mode of the IS pdf, which converges to the unit mass distribution concentrated at the point $\mathbf{x}^*$, as the estimate of the optimal solution $\mathbf{x}^*$.

To explain the main idea of the proposed method let us associate with Equation (2.1) a unimodal probability density function (pdf), called IS pdf, $h(\mathbf{x}, \mathbf{v})$, where $\mathbf{v} = (\mathcal{M}, \boldsymbol{\nu})$ is a parameter vector, $\mathcal{M}$ is the mode and $\boldsymbol{\nu}$ are the remaining parameters of $\mathbf{v}$. Note that if $h(\mathbf{x}, \mathcal{M}, \boldsymbol{\nu})$ is symmetric with respect to $\mathcal{M}$, like normal or double-exponential pdf's, then the mode $\mathcal{M}$ of $h(\mathbf{x}, \mathcal{M}, \boldsymbol{\nu})$ coincides with its expected value. If, for example, $h(x, \mathbf{V}) = h(x, \mathcal{M}, \nu) = \mathcal{N}(\mu, \sigma^2)$, then $\mathcal{M} = \mu$ and $\nu = \sigma^2$. If, for another example, $h(x, \mathbf{v}) = \text{Beta}(x, \alpha, \beta)$, that is

$$h(x, \alpha, \beta) = \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} \left(\frac{1}{b - a} \right)^{\alpha + \beta - 1} (b - x)^{\beta - 1} (x - a)^{\alpha - 1}, \quad a \leq x \leq b, \quad \alpha > 0, \quad \beta > 0, \quad (2.3)$$

then $\mathbf{v} = (\alpha, \beta)$, and the mode $\mathcal{M}$ can be calculated as

$$\mathcal{M} = \frac{\alpha - 1}{\alpha + \beta - 2}, \quad (2.4)$$

provided $\alpha > 1$, $\beta > 1$ and similarly in other cases.

Let next $\mathbf{Z}_1, \dots, \mathbf{Z}_N$ be a sample from $\text{Beta}(\alpha, \beta)$. We use this sample in a simple smooth stochastic optimization Equation (2.11) below, which is based on cross-entropy method Kapur and Kesavan (1992) to estimate eventually the optimal solution $\mathbf{x}^*$ of Equation (2.1). For a fixed sample $\mathbf{Z}_1, \dots, \mathbf{Z}_N$, denote the optimal solution of (1.21) with $\varphi(x) = x$ as $\bar{\mathbf{v}}_1^* = (\bar{\alpha}_1^*, \bar{\beta}_1^*)$. We show empirically that Algorithm 2.1 below ensures that $|\bar{\mathcal{M}}_1^* - \mathbf{x}^*| < |\mathcal{M} - \mathbf{x}^*|$ with high probability, where $\bar{\mathcal{M}}_1^*$ is calculated according to (2.4) except that (α, β) is replaced by $(\bar{\alpha}_1^*, \bar{\beta}_1^*)$. Note that we allow both components of the vector $(\bar{\alpha}_1^*, \bar{\beta}_1^*)$ be negative as well. At the next step, Algorithm 2.1 reduces adaptively the temperature, say γ in the partition function $C(\gamma)$ by solving recursively the program (2.11), where $\text{Beta}(\alpha, \beta)$ is replaced by $\text{Beta}(\bar{\alpha}_1^*, \bar{\beta}_1^*)$ and the original sample $\mathbf{Z}_1, \dots, \mathbf{Z}_N$ from $\text{Beta}(\alpha, \beta)$ is replaced by its counterpart from $\text{Beta}(\bar{\alpha}_1^*, \bar{\beta}_1^*)$. Denote the optimal solution as $\bar{\gamma}_1^*$. For fixed $\bar{\gamma}_1^*$, Algorithm 2.1 solves then Equation (2.11) with respect to $\mathbf{v}$, etc. Denote the pair of the optimal solution of $(\gamma, \mathbf{v})$ after t steps as $(\bar{\gamma}_t^*, \bar{\mathbf{v}}_t^*)$, $\bar{\mathbf{v}}_t^* = (\bar{\alpha}_t^*, \bar{\beta}_t^*)$. We take the mode $\bar{\mathcal{M}}_t^* = \bar{\mathcal{M}}_t^*(\bar{\alpha}_t^*, \bar{\beta}_t^*)$ calculated according to (2.4) as approximation of the optimal solution $\mathbf{x}$ ($\mathbf{x}^* \in \mathcal{X}^*(\zeta)$) and show numerically that after a finite number of steps t with Algorithm 2.1, the Boltzmann function $C(\bar{\gamma}_t^*)$ is very small, ($C(\bar{\gamma}_t^*) \leq 10^{-10}$) and as result $\bar{\mathcal{M}}_t^* \in \mathcal{X}^*(\zeta)$ with very high probability. We also show numerically that the estimate $\bar{\mathcal{M}}_t^*$ of $\mathbf{x}^*$ possesses low variance (short confidence intervals).

2.2. The Boltzmann Function

Let $L(\mathbf{x})$ be a continuous positive function over a closed bounded domain $\mathcal{X}$ and $\mathbf{x}^* = (x_1^*, x_2^*, \dots, x_n^*)$ be its unique global minimum. Define

$$H(\mathbf{x}, \gamma) = \exp\left(-\frac{L(\mathbf{x})}{\gamma}\right). \quad (2.5)$$

Commonly $H(\mathbf{x}, \gamma)$ is called the *Boltzmann function*, $L(\mathbf{x})$ —the *potential function*, γ —the *control parameter* or *temperature* and

$$C(\gamma) = \int_{\mathcal{X}} \exp\left(-\frac{L(\mathbf{x})}{\gamma}\right) d\mathbf{x} \quad (2.6)$$

—the *partition function*.

In statistical literature (Geyer and Thompson (1995)), $H(\mathbf{x}, \gamma)$ is often called the *unnormalized density* to distinguish it from the *normalized density* (Boltzmann distribution)

$$\tilde{h}(\mathbf{x}, \gamma) = C^{-1}(\gamma) \exp\left(-\frac{L(\mathbf{x})}{\gamma}\right). \quad (2.7)$$

Calculating (estimating) the partition function $C(\gamma)$ in a Boltzmann distribution for very small temperatures is of major importance in image analysis (Potamianos and Goutsias (1997)), statistical mechanics, physics, operations research and computer science. Estimation of $C(\gamma)$ is typically associated with the *Markov chain Monte Carlo* (MCMC) method (Geyer and Thompson (1995), Gilks *et al.* (1996), Marinari and Parisi

(1992), Potamianos and Goutsias (1997)) and its modification—the simulating annealing algorithms (Jerrum and Sinclair (1989), (1993), Lovas (1995)).

We present below an algorithm for *simultaneous* estimation of the global minimum $\mathbf{x}^*$ of Equation (2.1) and the partition function $C(\gamma)$. As in the MCMC approach, we deal with the unnormalized density $H(\mathbf{x}, \gamma)$ (family of unnormalized densities $\hat{\mathcal{H}} = \{H(\mathbf{x}, \gamma) : \gamma \in \Gamma\}$). In contrast to the MCMC method, however, where sampling is taken from the *normalized* density $\tilde{h}(\mathbf{x}, \gamma)$ (family of normalized densities $\tilde{\mathcal{H}} = \{\tilde{h}(\mathbf{x}, \gamma) : \gamma \in \Gamma\}$), by using an *arbitrary proposal* density (Gilks *et al.* (1996)), we sample from a *simple quite arbitrary unimodal* density $h(\mathbf{z}, \mathbf{v})$ (family of normalized unimodal densities $\mathcal{F} = \{h(\mathbf{x}, \mathbf{v}) : \mathbf{v} \in \Omega\}$). We do so since in practice sampling from $\tilde{h}(\mathbf{x}, \gamma)$ via the MCMC requires fast finding of associated Markov chain (using, say, the well-known Metropolis-Hastings algorithm (Rubinstein (1981), Gilks *et al.* (1996))), which is typically not the case, provided γ is small and $H(\mathbf{x})$ is multiextremal (Geyer and Thompson (1995), Marinari and Parisi (1992)). Some remedies for speeding up convergence in global optimization using clever modifications of the MCMC are given in Geyer and Thompson (1995) and Marinari and Parisi (1992).

The price we pay for generating from h rather than from $\tilde{h}$ is in having to solve, in addition, two sets of *convex but quite simple stochastic programs*. More specifically, simultaneously with $C(\gamma)$, we estimate adaptively both γ (see (2.18)) and the so-called reference parameter vector $\mathbf{v}^*$ of the IS pdf $h(\mathbf{z}, \mathbf{v})$, via solution of a sequence of stochastic programs of the type (2.11) below. We take the mode $\mathcal{M}(\mathbf{v}^*)$ of the IS $h(\mathbf{z}, \mathbf{v}^*)$ as estimate of $\mathbf{x}^*$. Our numerical studies in Section 4 below show that our algorithm works quite well for an arbitrary unimodal density $h(\mathbf{x})$ dominating the normalized density $\tilde{h}(\mathbf{x}, \gamma)$ in the absolute continuous sense and with marginals $h_i(x_i)$, $i = 1, \dots, n$ converging to delta functions as $\text{Var}(X_i) \rightarrow 0$, $i = 1, \dots, n$. Clearly, the rate of convergence of our algorithm depends on the particular choice of h . In this respect the density h resembles the proposal density in the MCMC method (Gilks *et al.* (1996)).

We proceed as follows. Using *likelihood ratios* (LR), we write C in analogy to (1.23) as

$$C(\gamma, \mathbf{v}) = \mathbb{E}_{\mathbf{v}}\{H(\mathbf{Z}, \gamma)W(\mathbf{z}, \mathbf{v})\}, \quad (2.8)$$

where

$$W(\mathbf{Z}_i, \mathbf{v}) = \frac{1}{h(\mathbf{z}, \mathbf{v})}$$

is the LR, $h(\mathbf{z}, \mathbf{v})$ is the dominating pdf, $\mathbf{Z} \sim h(\mathbf{z}, \mathbf{v})$ and $\mathbf{v}$ is the *reference parameter* vector. The LR estimator of C is

$$\bar{C}_N(\gamma, \mathbf{v}) = N^{-1} \sum_{i=1}^N H(\mathbf{Z}_i, \gamma) W(\mathbf{Z}_i, \mathbf{v}). \quad (2.9)$$

We shall call $\bar{C}_N(\gamma, \mathbf{v})$ the *average partition function*. Clearly, for small γ , the partition function $C(\gamma)$ is small as well, say $C(\gamma) \leq 10^{-10}$, and the challenge is to show that for small γ , the mode $\mathcal{M}(\mathbf{v}^*)$ of the importance sampling pdf $h(\mathbf{z}, \mathbf{v}^*)$ approximates $\mathbf{x}^*$ well, that is $\mathcal{M}(\mathbf{v}^*) \approx \mathbf{x}^*$.

2.3. The Main Algorithm

Our Algorithm 2.1 starts by choosing an initial value γ_0 and initial unimodal pdf $h(\mathbf{x}, \delta)$, say $h(\mathbf{x}, \delta) = \mathcal{U}(\mathbf{x} \in \mathcal{X})$, ($\mathcal{U}$ denoting the uniform distribution), such that under $h(\mathbf{x}, \delta)$ the partition function $C(\gamma_0)$ in (2.8) is rather small, say $C(\gamma_0) \approx 10^{-2}$. Then the algorithm shrinks adaptively both the parameter γ and the corresponding partition function $C(\gamma)$ to zero. Denote the estimates of the corresponding values γ and $C(\gamma)$ obtained at the t -th step of Algorithm 2.1 as $\bar{\gamma}_t^*$ and $\bar{C}_t$, respectively. Denote also $\bar{\mathbf{v}}_t^* = \bar{\mathbf{v}}_t^*(\gamma)$ the optimal parameter vector $\mathbf{v}$ obtained from the solution of Equation (2.11) at the t -th stage for fixed γ . We show numerically that iterating *adaptively* in both $\bar{\mathbf{v}}_t^*$ and $\bar{\gamma}_t^*$, after a finite number t of iterations, the mode $\bar{\mathcal{M}}_t = \bar{\mathcal{M}}_t(\bar{\mathbf{v}}_t^*) \in \{\mathcal{X}^*(\zeta)\}$ with very high probability, where $\mathcal{X}^*(\zeta)$ is the set of the optimal solutions of Equation (2.1).

We discuss next separately the associated programs for adaptive estimation of both $\mathbf{v}^*$ and γ^* .

2.3.1. Adaptive Estimation of the Optimal Reference Vector

We present here an estimate of the optimal reference vector $\mathbf{v}^*$ based on Kullback-Leibler cross-entropy (see Appendix A).

Consider (2.8). Applying Kullback-Leibler cross-entropy we can define at stage t (for fixed γ) in analogy to (1.21) and (7.15) the following program

$$\max_{\mathbf{v}_t} D(\gamma, \mathbf{v}_t, \mathbf{v}_{t-1}^*) = \max_{\mathbf{v}_t} \mathbb{E}_{\mathbf{v}_{t-1}^*} \{H(\mathbf{Z}, \gamma) \ln h(\mathbf{Z}, \mathbf{v}_t)\}, \quad (2.10)$$

where $\mathbf{Z} \sim h(\mathbf{z}, \mathbf{v}_{t-1}^*)$ and the initial pdf $h(\mathbf{z}, \mathbf{v}_0^*) = h(\mathbf{z}, \delta) = \mathcal{U}(\mathbf{x} \in \mathcal{X})$. It is important to note that according to (2.10), at each stage t , the function $H(\mathbf{x}, \gamma)$ is averaged (smoothed out) with the kernel (pdf) $h(\mathbf{z}, \mathbf{v}_{t-1}^*)$. The stochastic counterpart of (2.10) can be written as

$$\max_{\mathbf{v}_t} \left\{ \widehat{D}_N(\gamma, \mathbf{v}_t, \mathbf{v}_{t-1}^*) = \frac{1}{N} \sum_{i=1}^N H(\mathbf{Z}_i, \gamma) \ln h(\mathbf{Z}_i, \mathbf{v}_t) \right\}. \quad (2.11)$$

LEMMA 2.1 *If L is an additive function and can be presented as*

$$L(\mathbf{x}) = \sum_{k=1}^n L_k(x_k), \quad (2.12)$$

then the optimal solution $\mathbf{v}_t^$ of Equation (2.10) can be derived from the optimal solution of the program*

$$\max_{\mathbf{v}_t} D(\gamma, \mathbf{v}_t, \mathbf{v}_{t-1}^*) = \max_{\mathbf{v}_t} \mathbb{E}_{\mathbf{v}_{t-1}^*} \left\{ \sum_{k=1}^n H_k(Z_k, \gamma) \ln h_k(Z_k, \mathbf{v}_t) \right\}, \quad (2.13)$$

where

$$H_k(z_k, \gamma) = \exp\left(-\frac{L_k(z_k)}{\gamma}\right)$$

and

$$h(\mathbf{Z}, \mathbf{v}_t) = \prod_{k=1}^n h_k(Z_k, \mathbf{v}_{kt}).$$

Proof: Write (2.10) as

$$\max_{\mathbf{v}_t} D(\gamma, \mathbf{v}_t, \mathbf{v}_{t-1}^*) = \max_{\mathbf{v}_t} \mathbb{E}_{\mathbf{v}_{t-1}^*} \left\{ \sum_{k=1}^n \ln h_k(Z_k, \mathbf{v}_{kt}) \prod_{k=1}^n H_k(Z_k, \gamma) \right\}, \quad (2.14)$$

which is the same as

$$\max_{\mathbf{v}_t} D(\gamma, \mathbf{v}_t, \mathbf{v}_{t-1}^*) = \max_{\mathbf{v}_t} \mathbb{E}_{\mathbf{v}_{t-1}^*} \left\{ \sum_{k=1}^n C_k \ln h_k(Z_k, \mathbf{v}_{kt}) H_k(Z_k, \gamma) \right\}. \quad (2.15)$$

Here

$$C_k = \mathbb{E}_{\mathbf{v}_{t-1}^*} \prod_{j=1, j \neq k}^n H_j(Z_j, \gamma)$$

and it does not depend either on Z_k or on $\mathbf{v}_t = (\mathbf{v}_{1t}, \dots, \mathbf{v}_{kt}, \dots, \mathbf{v}_{nt})$. Since $D(\mathbf{v}_t)$ is convex and differentiable in $\mathbf{v}_t$, we can obtain the optimal solution $\mathbf{v}_t^*$ of Equation (2.15) by solving the following set of non-linear equations

$$\nabla D(\mathbf{v}_t) = \mathbf{0}. \quad (2.16)$$

Applying (2.16) to both (2.15) and (2.13) it readily follows that their optimal solutions coincide.

Clearly, solving (2.13) is less time-consuming than solving (2.10).

It is crucial to realize that all above stochastic programs are *smooth and concave* in $\mathbf{v}$, provided $h(\mathbf{Z}, \mathbf{v})$ is concave in $\mathbf{v}$.

2.3.2. Adaptive Estimation of the Temperature

Once $\bar{\mathbf{v}}_t^*$ has been derived at the t -th stage, it is employed in a stochastic program (see Equation (2.18) below) to estimate adaptively the optimal temperature γ_t^* .

Denote $\bar{\mathbf{v}}_t^*$ the optimal vector at stage t , obtained by solving Equation (2.11) for a given γ_{t-1}^* . For given $\bar{\mathbf{v}}_t^*$ an optimal γ_t^* at stage t can be obtained from the solution of the following program

$$\min \gamma \text{ s.t. } (\mathcal{C}(\gamma, \bar{\mathbf{v}}_t^*) = \mathbb{E}_{\bar{\mathbf{v}}_t^*} \{H(\mathbf{Z}, \gamma)\}) \geq \rho, \quad (2.17)$$

where $\rho_1 \leq \rho \leq \rho_2$, and ρ is quite small, say $\rho_1 = 10^{-6}$, $\rho_2 = 10^{-2}$.

The stochastic counterpart of Equation (2.17) is

$$\min \gamma \text{ s.t. } \left(\bar{\mathcal{C}}_N(\gamma, \bar{\mathbf{v}}_t^*) = N^{-1} \sum_{i=1}^n H(\mathbf{Z}_i, \gamma) \right) \geq \rho, \quad (2.18)$$

where $\mathbf{Z}_i \sim h(\mathbf{z}, \bar{\mathbf{v}}_t^*)$. We call $\bar{\mathcal{C}}_N(\gamma, \mathbf{v})$ the *simulated partition function* (SPF).

For fixed ρ , Equation (2.18) calls for finding the root $\bar{\gamma}_t^*$ such that at each stage t (under the optimal sampling from $h(\mathbf{z}, \bar{\mathbf{v}}_t^*)$) the resulting SPF $\bar{\mathcal{C}}_N(\bar{\gamma}_t^*, \bar{\mathbf{v}}_t^*)$ is not too small, say

$$\bar{\mathcal{C}}_N(\bar{\gamma}_t^*, \bar{\mathbf{v}}_t^*) \approx \rho, \quad 10^{-6} \leq \rho \leq 10^{-2}. \quad (2.19)$$

Note that (2.17) contains only the first part of C in (2.8), namely the part H , but not the part W , and similarly (2.18). Our algorithm ensures that (2.19) holds for all $t = 0, 1, \dots$. Note also that for fixed ρ the function $\bar{\mathcal{C}}_N(\bar{\gamma}, \bar{\mathbf{v}}_t^*)$ is monotone nondecreasing in γ .

There exists a trade-off between Equation (2.11) and (2.18) in the sense that the convergence rate of (2.11) increases as γ decreases, but, at the same time, cooling the temperature γ too fast may result in a local minimum. The goal of (2.18) with $10^{-6} \leq \rho \leq 10^{-2}$ is to resolve the cooling schedule adaptively, namely to

1. Keep γ small, that is to ensure fast convergence of $\bar{\mathbf{v}}_t^*$ in (2.11) to $\mathbf{x}$.
2. Insure that for small γ , the function

$$H(\mathbf{Z}_i, \gamma) \ln h(\mathbf{Z}_i, \mathbf{v}_t)$$

in Equation (2.11) will be still computational tractable and that Equation (2.11) is able to avoid a local minima.

For an additive function L , (2.17) reduces to

$$\min \gamma \text{ s.t. } \left(\mathcal{C}(\gamma, \bar{\mathbf{v}}_t^*) = \mathbb{E}_{\bar{\mathbf{v}}_t^*} \left\{ \prod_{k=1}^n H_k(\mathbf{Z}_k, \gamma) \right\} \right) \geq \rho, \quad (2.20)$$

and similarly to (2.18), its stochastic version.

2.3.3. The Algorithm

As already mentioned, our Algorithm 2.1 below can viewed as the one which breaks-down the “hard” problem of estimating the very small quantity $C(\gamma)$, into a set of “simple” problems by actually generating the following four sequences:

1. An approximately *constant* sequence of SPF’s $\bar{\mathcal{C}}_N(\bar{\gamma}_t^*, \bar{\mathbf{v}}_t^*) \approx \rho, \quad t = 1, 2, \dots$
2. A sequence of temperatures

$$\bar{\gamma}_1^*, \bar{\gamma}_2^*, \dots, \bar{\gamma}_t^*, \dots \quad (2.21)$$

converging to zero.

3. A sequence of partition functions in Equation (2.9)

$$\bar{C}_N(\bar{\gamma}_1^*, \bar{\mathbf{v}}_1^*), \bar{C}_N(\bar{\gamma}_2^*, \bar{\mathbf{v}}_2^*), \dots, \bar{C}_N(\bar{\gamma}_t^*, \bar{\mathbf{v}}_t^*), \dots \quad (2.22)$$

converging to zero.

4. A sequence of solution vectors $\mathcal{M}(\bar{\mathbf{v}}_t^*)$, $t = 1, 2, \dots$ converging to $\mathbf{x}^*$.

Clearly, the first three sequences are auxiliary, while the fourth is the main sequence. Our numerical results suggest that both sequences $\bar{\gamma}_t^*$ and $\bar{C}_N(\bar{\gamma}_t^*)$ are monotone decreasing, that is

$$\bar{\gamma}_1^* > \bar{\gamma}_2^* > \dots > \bar{\gamma}_t^* > \dots$$

and

$$\bar{C}_N(\bar{\gamma}_1^*, \bar{\mathbf{v}}_1^*) > \bar{C}_N(\bar{\gamma}_2^*, \bar{\mathbf{v}}_2^*) > \dots > \bar{C}_N(\bar{\gamma}_t^*, \bar{\mathbf{v}}_t^*) > \dots$$

respectively, provided L is a convex function. For multiextremal functions our algorithm allows both $\bar{\gamma}_t^*$ and $\bar{C}_N(\bar{\gamma}_t^*)$ to increase as well. This typically occurs when climbing uphill on $L(\mathbf{x})$, that is while moving towards the global minimum from one local minimum to another.

As mentioned earlier our algorithm iterates adaptively, using interchangeably the stochastic Equations (2.11) and (2.18), until the desired accuracy is obtained. It can be summarized as follows.

ALGORITHM 2.1

1. Choose an initial pdf $h(\mathbf{x}, \boldsymbol{\delta})$, say $h = \mathcal{U}(\mathbf{x} \in \mathcal{X})$, which corresponds, say, to $h(\mathbf{x}, \boldsymbol{\delta}) = \text{Beta}(\alpha = 1, \beta = 1)$. Generate a sample $\mathbf{Z}_1, \dots, \mathbf{Z}_N$ from the pdf $h(\mathbf{x}, \boldsymbol{\delta})$ and solve Equation (2.18) to obtain an initial γ . Denote the solution as $\bar{\gamma}_0^*$.
2. Use the **same** sample $\mathbf{Z}_1, \dots, \mathbf{Z}_N$ from the pdf $h(\mathbf{x}, \boldsymbol{\delta})$ and solve (with respect to $\mathbf{v}$ for $\gamma = \bar{\gamma}_0^*$) the stochastic Equation (2.11) based on Kullback-Leibler cross-entropy. Denote the solution as $\bar{\mathbf{v}}_1^*$.
3. Generate a **new** sample $\mathbf{Z}_1, \dots, \mathbf{Z}_N$ from the pdf $h(\mathbf{x}, \bar{\mathbf{v}}_1^*)$ and solve (with respect to γ for fixed $\bar{\mathbf{v}}_1^*$) the stochastic Equation (2.18). Denote the solution as $\bar{\gamma}_1^*$.
4. Repeat Steps 2 and 3, again with $\boldsymbol{\delta}$ ($\boldsymbol{\delta} \equiv \bar{\mathbf{v}}_0^*$) and $\bar{\gamma}_0^*$ replaced by $\bar{\mathbf{v}}_1^*$ and $\bar{\gamma}_1^*$, respectively. Denote the corresponding optimal solutions at stage k as $\bar{\mathbf{v}}_k^*$ and $\bar{\gamma}_k^*$, respectively.
5. If for some k and some r , say $r \geq 5$

$$\max_{j \leq r} \{ \mathcal{M}(\bar{\mathbf{v}}_k^*) - \mathcal{M}(\bar{\mathbf{v}}_{k+j}^*) \} \leq \varepsilon, \quad (2.23)$$

where ε is a very small number, say $\varepsilon = 10^{-6}$, repeat Step 3 and stop, otherwise repeat Steps 2–4 again. After stopping:

- Estimate C using (2.9) with $\mathbf{v}$ and γ equal to $\bar{\mathbf{v}}_k^*$ and $\bar{\gamma}_k^*$, respectively.
- Take the mode $\mathcal{M}(\bar{\mathbf{v}}_k^*)$ as the estimate of the global minimum $\mathbf{x}^*$ of $L(\mathbf{x})$.

An alternative stopping rule is: stop if for some t and some k , say $k \geq 5$

$$\max_{j \leq k} \{|\bar{\gamma}_t^* - \bar{\gamma}_{t+j}^*|\} \leq \varepsilon_1, \text{ and } \max_{j \leq k} \{\|\bar{\mathbf{v}}_t^* - \bar{\mathbf{v}}_{t+j}^*\|\} \leq \varepsilon_2, \quad (2.24)$$

where ε_1 and ε_2 are very small numbers.

According to (2.24), Algorithm 2.1 stops when k consecutive stages (iterations) result in negligible changes in both $\bar{\gamma}_k^*$ and $\bar{\mathbf{v}}_k^*$. Note again that the auxiliary random sequence of Boltzmann temperatures $\bar{\gamma}_1^*, \bar{\gamma}_2^*, \dots$ is an inherent part of Algorithm 2.1. This sequence is designed with a view to fast convergence to zero and such that the target value $\mathbf{x}^*$ can be reached with high probability after a finite number of iterations.

We call Algorithm 2.1, the *cross-entropy* (SE) algorithm.

To speed up the algorithm and to make it more flexible, one can further employ the following

REMARK 2.1

1. In order to find the global maximum instead of the global minimum we have to change the sign $-$ in (2.5) into the sign $+$, that is to use instead of (2.5)

$$H^+(\mathbf{x}, \gamma) = \exp\left(+\frac{L(\mathbf{x})}{\gamma}\right). \quad (2.25)$$

2. Use instead of the Beta distribution, a more general one based on m -parameter exponential family distributions given in the following canonical form:

$$h(y, \mathbf{v}) = c(\mathbf{v}) \exp\left(\sum_{k=1}^m \nu_k t_k(y)\right) r(y), \quad (2.26)$$

where $c(\mathbf{v}) > 0$ is a real-valued function of the parameter vector $\mathbf{v}$, and $t_k(y)$ and $r(y)$ are real-valued functions of y .

It is desirable that $h(y, \mathbf{v})$ in (2.26) be log-convex in $\mathbf{v}$ and approach Dirac's delta function as $\text{Var}\{Y\} \rightarrow 0$. As example, consider

$$h(y, \mathbf{v}) = c(\mathbf{v}) \exp[(\alpha - 1) \ln y + (\beta - 1) \ln(1 - y) + \mu y + \sigma y^2], \quad \mathbf{v} = (\alpha, \beta, \mu, \sigma). \quad (2.27)$$

Note that for $\mu = \sigma = 0$, (2.27) coincides with the Beta pdf.

3. Combine the MCMC Metropolis-Hastings algorithm (Gilks et al. (1996)) with Algorithm 2.1. In particular, at step 2 of Algorithm 2.1 replace “Use the **same** sample $\mathbf{Z}_1, \dots, \mathbf{Z}_N$ from the pdf $h(\mathbf{x}, \delta)$ ” by “Take a sample $\mathbf{Z}_1, \dots, \mathbf{Z}_N$ from the Boltzmann density $\tilde{h}(\mathbf{x}, \bar{\gamma}_0^*)$ ” the rest of the sentence in step 2 remaining the same. This will typically also result in a good starting point $\bar{\mathbf{v}}_1^*$, since sampling from the IS density

$$\tilde{h}(\mathbf{x}, \bar{\gamma}_0^*) = C^{-1}(\bar{\gamma}_0^*)H(\mathbf{x}, \bar{\gamma}_0^*)$$

is more desirable than sampling, say from $h(\mathbf{x}, \delta) = \text{Beta}(\alpha = 1, \beta = 1)$.

4. Take the maximal value of the order statistics $L_{(N)}$, say $\bar{\mathbf{x}}_{(N)}^*$, as an estimate of the optimal solution $\mathbf{x}^*$ of $L(\mathbf{x})$ instead of the mode $\mathcal{M}(\bar{\mathbf{v}}_k^*)$, (see step 5).
5. If for any t and the same k , say $k \geq 3$, we obtain simultaneously that

$$\bar{\gamma}_{t+k}^* < \bar{\gamma}_{t+k-1}^* < \dots < \bar{\gamma}_t^*$$

and

$$\bar{C}_N(\bar{\gamma}_{t+k}^*) < \bar{C}_N(\bar{\gamma}_{t+k-1}^*) < \dots < \bar{C}_N(\bar{\gamma}_t^*),$$

then declare that Algorithm 2.1 has reached the region of attraction of the global minimum and switch to a faster method for quasi-convex optimization, like the “simplex method” of Mead and Nedler (1965).

6. Use in (2.18) instead of a fixed ρ the following adaptive procedure for ρ

$$\rho_{s+1} = \begin{cases} C_s \rho_s & \text{if } \max_{j \leq d} \{\mathcal{M}(\bar{\mathbf{v}}_s^*) - \mathcal{M}(\bar{\mathbf{v}}_{s+j}^*)\} \leq \varepsilon, \\ \rho_s & \text{if } \mathcal{M}(\bar{\mathbf{v}}_s^*) > \mathcal{M}(\bar{\mathbf{v}}_{k+1}^*), \quad s = 0, 1, \dots, \end{cases} \quad (2.28)$$

where $\rho_0 = \rho$, (see (2.18)) $10^{-3} \leq \rho \leq 10^{-5}$, $10^{-4} \leq C_s \leq 10^{-1}$ and d is fixed, say $d = 3$. In our numerical results with continuous optimizations in Ch 5 below we take $\rho_0 = 10^{-3}$, $d = 3$ and

$$C_s = 10^{-3(s+1)}, s = 0, 1, \dots \quad (2.29)$$

It follows from (5.5)–(2.29) that Algorithm 2.1 starts with $\rho_0 = 10^{-3}$, and then, each time the performance does not decrease below ε in turn 3 times ($\max_{j \leq d} \{\mathcal{M}(\bar{\mathbf{v}}_s^*) - \mathcal{M}(\bar{\mathbf{v}}_{s+j}^*)\} \leq \varepsilon$, in turn 3 times), it chooses a smaller ρ . The resulting adaptive sequence of ρ_s is therefore

$$\rho_s = 10^{-3}, 10^{-6}, 10^{-9}, \dots \quad (2.30)$$

If, for example, Algorithm 2.1 stops when the event

$$A = \left\{ \max_{j \leq r} \{ \mathcal{M}(\bar{\mathbf{v}}_s^*) - \mathcal{M}(\bar{\mathbf{v}}_{s+j}^*) \} \leq \varepsilon \right\}$$

occurs five times in turn, (see (5.6)), then the final ρ_s (just before stopping) will be $\rho_5 = 10^{-18}$.

3. Combinatorial Optimization: SLP and TSP

In this section we present our main CE algorithms for the shortest (longest) path (SLP) in a directed network and the traveling salesman problem (TSP). In Subsections 3.1 and 3.2 we deal with some relevant combinatorial optimization problems, namely with estimation of the average path and estimation of probability of rare events in SLP and TSP. These subsections are of independent interest. In addition they might be considered as some prerequisite to the combinatorial optimization problems discussed in Subsection 3.3.

The shortest path problem can be defined as follows. Find

$$\min_{j=1, \dots, p} \sum_{i \in \mathcal{B}_j} L_i. \quad (3.1)$$

Here, $\mathcal{B}_j$ is the j -th complete path from a source to a sink; p is the number of complete paths; and L_i , $i = 1, \dots, m$, are the durations of the i -th component. For the longest path we replace min with max in (3.1).

Problem (3.1) naturally extends to its stochastic (expected) version (see (1.4)) as: Find

$$\mathbb{E} \left\{ \min_{j=1, \dots, p} \sum_{i \in \mathcal{B}_j} Y_i \right\}, \quad (3.2)$$

where $Y_i \sim F_i(y_i)$, $i = 1, \dots, m$, is a random variable representing the duration of the i -th entity and $F_i(y_i)$ is the cdf of Y_i .

The traveling salesman problem (TSP) can be formulated as follows: Find

$$\min_{\pi} \left\{ \sum_{i=1}^{n-1} L_{\pi_i \pi_{i+1}} + L_{\pi_n \pi_1} \right\} \quad (3.3)$$

where $\pi = (\pi(1), \pi(2), \dots, \pi(n))$ denotes a permutation, $\pi(i)$, $i = 1, 2, \dots, n$ is the i -th city to be visited in the tour represented by π , and L_{ij} is the distance from city i to city j . Its stochastic version can be defined similarly to (3.2).

3.1. The Expected Performance in a Network

We consider separately the expected performance for the shortest (longest) path and the TSP problem.

3.1.1. The Expected Shortest (Longest) Path

Let L_{ij} be the length of the arc (ij) (time to travel from node i to node j) in a directed network and let $P = (P_{ij})$ be an associated random matrix (Markov chain) with an absorbing state at the sink n and transition probabilities

$$P_{ij} = \begin{cases} > 0, & \text{if } L_{ij} > 0, \ i, j = 1, 2, \dots, n, \ i \neq n \\ 1, & \text{if } i = j = n \\ 0 & \text{otherwise,} \end{cases} \quad (3.4)$$

where $\sum_{j=1}^n P_{ij} = 1$. Denote by τ the time of absorption at state n . Let

$$i_1, i_2, \dots, i_\tau$$

be a random walk (trajectory) through the nodes of the network generated by the MC (3.4), where $i_1 \equiv 1$, that is, it is assumed that i_1 coincides with the source (first node) of the network. Then the expected path length from the source to the sink can be defined as

$$\ell(P) = \mathbb{E}_P \left\{ \sum_{t=1}^{\tau} L_{i_t i_{t+1}} \right\}. \quad (3.5)$$

Here

$$i_{\tau+1} \equiv i_\tau, \ L_{i_\tau i_{\tau+1}} = 0. \quad (3.6)$$

Formula (3.5) can be extended as

$$\ell(P) = \mathbb{E}_P \left\{ \varphi \left[\sum_{t=1}^{\tau} L_{i_t i_{t+1}} \right] \right\}, \quad (3.7)$$

where $\varphi(x)$ is a real-valued function. Examples of $\varphi(x)$ are given in (1.2) and (1.3).

Note that expected performances $\ell(P)$ and $\ell(\mathbf{v})$ in (3.7) and (1.1) differ quite substantially in the sense that the randomness in the former is presented via the MC $P = (P_{ij})$ (the elements L_{ij} of distance matrix $L = (L_{ij})$ are deterministic quantities), while the randomness in the latter is presented via the random variables $L_{ij} = Y_{ij}$ (the random matrix $P = (P_{ij})$ does not come into play in the latter case at all).

Using likelihood ratios (LR) for (P_{ij}) we can write $\ell(P)$ (see also Rubinstein and Melamed (1998)) as

$$\ell(P) = \mathbb{E}_{P_0} \left\{ \varphi \left[\sum_{t=1}^{\tau} L_{i_t i_{t+1}} \right] \widetilde{W}_\tau \right\}, \quad (3.8)$$

where Rubinstein and Melamed (1998)

$$\tilde{W}_\tau = \prod_{s=1}^{\tau} W_{i_s i_{s+1}}, \quad W_{i_s i_{s+1}} = \frac{P_{i_s i_{s+1}}}{P_{0, i_s i_{s+1}}} \quad (3.9)$$

and we assume that

$$P_{i_\tau i_{\tau+1}} = P_{0, i_\tau i_{\tau+1}} = 1.$$

The LR estimate of $\ell(P)$ is

$$\tilde{\ell}_N(P) = \frac{1}{N} \sum_{j=1}^N \varphi \left[\sum_{i=1}^{\tau_j} L_{i_i i_{i+1}, j} \right] \tilde{W}_{\tau, j} \quad (3.10)$$

where in analogy to (3.9)

$$\tilde{W}_{\tau, j} = \prod_{s=1}^{\tau_j} W_{i_s i_{s+1}, j}, \quad W_{i_s i_{s+1}, j} = \frac{P_{i_s i_{s+1}, j}}{P_{0, i_s i_{s+1}, j}}. \quad (3.11)$$

If not stated otherwise, we assume in this subsection below that $\varphi(x) = x$.

The optimal matrix P_0^* in the LR estimate (3.10) can be obtained in analogy to (1.28) either via importance sampling by minimizing the variance of $\tilde{\ell}_N(P)$ with respect to P_0 , that is

$$\begin{aligned} \min_{P_0} \text{Var} \left\{ \sum_{t=1}^{\tau} L_{i_t i_{t+1}} \tilde{W}_\tau \right\}, \\ \text{s.to } \sum_{s=1}^n P_{0, i_s i_{s+1}} = 1, \quad r = 1, \dots, n, \end{aligned} \quad (3.12)$$

or, in analogy to (1.30) via the cross-entropy method by solving the following program

$$\begin{aligned} \max_{P_0} D(P, P_0) &= \max_{P_0} \mathbb{E}_P \left\{ \sum_{t=1}^{\tau} L_{i_t i_{t+1}} \ln \tilde{P}_{0, i_t i_{t+1}} \right\} \\ &= \max_{P_0} \mathbb{E}_P \left\{ \sum_{t=1}^{\tau} L_{i_t i_{t+1}} \sum_{s=1}^{\tau} \ln P_{0, \tau} = \prod_{s=1}^{\tau} P_{0, i_s i_{s+1}} \right\}, \\ \text{s.to } \sum_{s=1}^n P_{0, rs} &= 1, \quad r = 1, \dots, n. \end{aligned} \quad (3.13)$$

For better insight consider the following simple example

EXAMPLE 3.1 Bridge Network in Figure 3.1

In this case the MC P can be written as

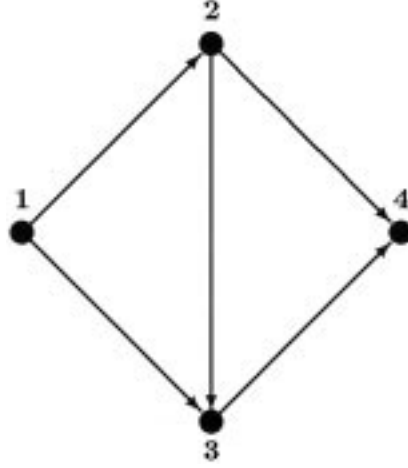


Figure 3.1. The bridge network.

$$P = \begin{pmatrix} 0 & P_{12} & P_{13} & 0 \\ 0 & 0 & P_{23} & P_{24} \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (3.14)$$

There are only three trajectories associated with the MC P :

$$1 \rightarrow 2 \rightarrow 4, \quad (\tau = 2); \quad 1 \rightarrow 3 \rightarrow 4, \quad (\tau = 2); \quad 1 \rightarrow 2 \rightarrow 3 \rightarrow 4, \quad (\tau = 3), \quad (3.15)$$

which are generated with probabilities

$$P_{12}P_{24}P_{44}; \quad P_{13}P_{34}P_{44}; \quad P_{12}P_{23}P_{34}P_{44}, \quad (3.16)$$

respectively with $P_{34} = 1$, $P_{44} = 1$. In this case formula (3.5) reduces to

$$\ell(P) = (L_{12} + L_{24})P_{12}P_{24} + (L_{13} + L_{34})P_{13}P_{34} + (L_{12} + L_{23} + L_{34})P_{12}P_{23}P_{34}. \quad (3.17)$$

Take $N = 3$ and assume also that we obtained from the simulation the following three trajectories

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 4; \quad 1 \rightarrow 2 \rightarrow 4; \quad 1 \rightarrow 3 \rightarrow 4.$$

Then (3.10) and (3.13) reduce to

$$\begin{aligned} \tilde{\ell}_N(P) = & \frac{1}{3} [(L_{12} + L_{23} + L_{34})W_{12}W_{23}W_{34} \\ & + (L_{12} + L_{24})W_{12}W_{24} \\ & + (L_{13} + L_{34})W_{13}W_{34}] \end{aligned} \quad (3.18)$$

and

$$\begin{aligned}
& \max_{P_0} D(P, P_0) \\
&= \max_{P_0} \{ (L_{12} + L_{24})P_{12}P_{24}(\ln P_{0,12} + \ln P_{0,24}) \\
&\quad + (L_{12} + L_{23} + L_{34})P_{12}P_{23}(\ln P_{0,12} + \ln P_{0,23}) \\
&\quad + (L_{13} + L_{34})P_{13} \ln P_{0,13} \}, \\
&\text{s.to } P_{0,12} + P_{0,13} = 1, P_{0,23} + P_{0,24} = 1,
\end{aligned} \tag{3.19}$$

respectively. Here

$$W_{ij} = \frac{P_{ij}}{P_{0,ij}}$$

and $P_0 = (P_{0,ij})$ is a MC dominating the MC P in (3.14).

Let, for example

$$L_{12} = L_{13} = L_{23} = L_{34} = 1, L_{24} = 10^{10};$$

$$P_{12} = p_1, P_{13} = p_2, (p_1 + p_2 = 1), P_{23} = 1 - \varepsilon, P_{24} = \varepsilon,$$

where ε is a very small number, say $\varepsilon = 10^{-6}$. It is readily seen that $\ell(P) \approx 10^4$, and solving program (3.13), or its stochastic counterpart, one obtains

$$P_{0,12}^* \approx P_{0,24}^* \approx 1 - \varepsilon; P_{0,13}^* \approx P_{0,23}^* \approx \varepsilon$$

irrespective of p_1 and p_2 , provided $p_1 \gg \varepsilon$. That is, $P_{0,23}^*$ and $P_{0,24}^*$ will nearly interchange with P_{24} and P_{23} , respectively. This is so, since the trajectory $1 \rightarrow 2 \rightarrow 4$ will dominate (will be chosen more often than) the remaining two trajectories.

3.1.2. The Expected Traveling Salesman Problem

In analogy to (3.7) consider the the following expected traveling salesman problem

$$\ell(P) = \mathbb{E}_P \left\{ \varphi \left[\sum_{t=1}^{n-1} L_{i_{t-1}i_t} + L_{i_{n-1}i_0} \right] \right\}. \tag{3.20}$$

Here

$$P_{ij} = \begin{cases} > 0 & \text{if } i \neq j \\ 0 & \text{otherwise} \end{cases} \tag{3.21}$$

is a transition matrix associated with the distance matrix (L_{ij}) .

To estimate the expected performance $\ell(P)$ we introduce below a random sequence

$$\mathcal{P}_n = (P^{(0)}, P^{(1)}, \dots, P^{(n-1)}),$$

which is generated from the original MC (3.21). The aim of the sequence $\mathcal{P}_n$ is to ensure that the traveling salesman visits each city in a given walk (tour) *only once*. To satisfy this requirement, define a trajectory (random walk),

$$i_0^{(0)}, i_1^{(1)}, \dots, i_{n-1}^{(n-1)}, \quad (3.22)$$

and an associated probability

$$P_{i_0 i_1}^{(0)} P_{i_1 i_2}^{(1)} \cdots P_{i_{n-1} i_0}^{(n-1)}. \quad (3.23)$$

The pair

$$\left(i_0^{(0)}, i_1^{(1)}, \dots, i_{n-1}^{(n-1)}, P_{i_0 i_1}^{(0)} P_{i_1 i_2}^{(1)} \cdots P_{i_{n-1} i_0}^{(n-1)} \right)$$

is generated recursively from the sequence $\mathcal{P}_k = (P^{(0)}, P^{(1)}, \dots, P^{(k-1)})$, $k = 1, \dots, n$ of Markov chains as follows: Generate an initial state $i_0^{(0)} = i_0$ (randomly or deterministically, see below) and the first associated transition $P_{i_0 i_1}^{(0)} \equiv P_{i_0 i_1}$ from the original $n \times n$ MC P in (3.21). The next transition $P_{i_1 i_2}^{(1)}$ is generated from the $(n-1) \times (n-1)$ MC $P^{(1)} = (P_{ij}^{(1)})$, which is obtained from the original MC (3.21) by eliminating first, the *row* i_0 and the *column* i_1 , and then normalizing the remaining elements such that the diagonal elements of $P^{(1)}$ equal zero. Proceeding recursively, we generate $P_{i_{k-1} i_k}^{(k-1)}$, ($k \leq n-1$) from the $(n-k) \times (n-k)$ MC $P^{(k)} = (P_{ij}^{(k)})$, which is obtained from the MC $P^{(k-1)} = (P_{ij}^{(k-1)})$ by eliminating the *row* i_{k-1} and the *column* i_k and then again normalizing the remaining elements such that the diagonal elements of $P^{(k)}$ equal zero. Clearly $P^{(n-2)}$ is a 2×2 matrix with the off-diagonal elements equal unity, and $P^{(n-1)} = (P_{ij}^{(n-1)}) = 1$. Note that the length of the “cycle” to absorption equals exactly n , and the random walk $i_0^{(0)}, i_1^{(1)}, \dots, i_{n-1}^{(n-1)}$ associated with the sequence $\mathcal{P}_N = (P^{(0)}, P^{(1)}, \dots, P^{(n-1)})$ of MC behaves as a random sequence in a classic urn model without replacement. For this reason we call $\mathcal{P}_N = (P^{(0)}, P^{(1)}, \dots, P^{(n-1)})$, generated from the original MC matrix (3.21)—the *Markov chain without replacement* (MCWR) sequence.

Instead of MCWR one can use the acceptance-rejection method, by first generating from the original MC $P \equiv P^{(0)}$, and then, depending on whether the state j was previously visited or not in a given trajectory, one either rejects or accepts the associated transition P_{ij} from a fixed state i to the state j . The efficiency of such an acceptance-rejection method relative to MCWR typically decreases with the dimensionality n .

Assuming below that $\varphi(x) = x$, the CMC estimate of $\ell(P)$ in (3.20) is

$$\tilde{\ell}_N(P) = \frac{1}{N} \sum_{j=1}^N \left[\sum_{i=1}^{n-1} L_{i_{t-1} i_t, j} + L_{i_{n-1} i_0, j} \right]. \quad (3.24)$$

Using LR we can write $\ell(P)$ as

$$\ell(P, P_0) = \mathbb{E}_{\mathcal{P}_N} \left\{ \sum_{t=1}^{n-1} (L_{i_{t-1} i_t} + L_{i_{n-1} i_0}) \tilde{W}_n \right\}, \quad (3.25)$$

where

$$\tilde{W}_n = \prod_{s=1}^{n-1} W_{i_{s-1}i_s}^{(s-1)} + W_{i_{n-1}i_0}, \quad W_{i_{s-1}i_s}^{(s-1)} = \frac{P_{i_{s-1}i_s}^{(s-1)}}{P_{0,i_{s-1}i_s}}, \quad W_{i_{n-1}i_0} = \frac{1}{P_{0,i_{n-1}i_0}} \quad (3.26)$$

and similarly the stochastic counterpart of (3.25).

The optimal matrix P_0^* which minimizes the variance of the LR estimator of (3.25) can be obtained via cross-entropy by solving, in analogy to (3.13), program

$$\max_{P_0} D_1(P, P_0) = \max_{P_0} \mathbb{E}_{\mathcal{P}_n} \left\{ \sum_{t=1}^{n-1} (L_{i_{t-1}i_t} + L_{i_{n-1}i_0}) \ln \tilde{P}_{0,n} \right\}, \quad (3.27)$$

where

$$\tilde{P}_{0,n} = \prod_{s=1}^{n-1} P_{0,i_{s-1}i_s} + P_{0,i_{n-1}i_0} \quad (3.28)$$

and the elements of the matrix P_0 are normalized similarly to (3.13).

Note that both $\tilde{\ell}_N(P)$ and $\ell(P)$ in (3.24) and (3.25), as well as the optimal solution P_0^* in (3.27), depend on the initial state $i_0^{(0)} = i_0$. In practice it is desirable to simulate the original MC P starting each trajectory either at random or deterministically, but each time starting from a different state. If N is the total number of generated trajectories, then $\frac{N}{n}$ is the approximate number of starting points for each state.

EXAMPLE 3.2 Four Node Non-symmetric Traveling Salesman Problem

The underlying MC P can be written as

$$P = \begin{pmatrix} 0 & P_{12} & P_{13} & P_{14} \\ P_{21} & 0 & P_{23} & P_{24} \\ P_{31} & P_{32} & 0 & P_{34} \\ P_{41} & P_{42} & P_{43} & 0 \end{pmatrix}. \quad (3.29)$$

Starting the simulation from state 1 we obtain the following six trajectories:

$$\begin{aligned} &1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1; \quad 1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1; \quad 1 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 1; \\ &1 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1; \quad 1 \rightarrow 3 \rightarrow 4 \rightarrow 2 \rightarrow 1; \quad 1 \rightarrow 4 \rightarrow 2 \rightarrow 3 \rightarrow 1 \end{aligned} \quad (3.30)$$

and similarly when starting the simulation from other states of the MC. We call the first three and second three trajectories in (3.30) the forward-loop and backward-loop trajectories respectively, and similarly for larger MC. The two sets of trajectories are selected with probabilities

$$\begin{aligned}
& P_{12}^{(0)} P_{23}^{(1)} P_{34}^{(2)} P_{41}^{(3)}, (P_{34}^{(2)} = 1, P_{41}^{(3)} = 1); \\
& P_{12}^{(0)} P_{24}^{(1)} P_{43}^{(2)} P_{31}^{(3)}, (P_{43}^{(2)} = 1, P_{31}^{(3)} = 1); \\
& P_{13}^{(0)} P_{32}^{(1)} P_{24}^{(2)} P_{41}^{(3)}, (P_{24}^{(2)} = 1, P_{41}^{(3)} = 1),
\end{aligned} \tag{3.31}$$

$$\begin{aligned}
& P_{14}^{(0)} P_{43}^{(1)} P_{32}^{(2)} P_{21}^{(3)}, (P_{32}^{(2)} = 1, P_{21}^{(3)} = 1); \\
& P_{13}^{(0)} P_{34}^{(1)} P_{42}^{(2)} P_{21}^{(3)}, (P_{42}^{(2)} = 1, P_{21}^{(3)} = 1); \\
& P_{14}^{(0)} P_{42}^{(1)} P_{23}^{(2)} P_{31}^{(3)}, (P_{23}^{(2)} = 1, P_{31}^{(3)} = 1),
\end{aligned} \tag{3.32}$$

respectively. In this case, starting at state 1, the (conditional) expected traveling distance is

$$\begin{aligned}
& \ell(P)|_{i_0=1} \\
&= (L_{12} + L_{23} + L_{34} + L_{41})[P_{12}^{(0)} P_{23}^{(1)} P_{34}^{(2)} P_{41}^{(3)} + P_{14}^{(0)} P_{43}^{(1)} P_{32}^{(2)} P_{21}^{(3)}] \\
&+ (L_{12} + L_{24} + L_{43} + L_{31})[P_{12}^{(0)} P_{24}^{(1)} P_{43}^{(2)} P_{31}^{(3)} + P_{13}^{(0)} P_{34}^{(1)} P_{42}^{(2)} P_{21}^{(3)}] \\
&+ (L_{13} + L_{32} + L_{24} + L_{41})[P_{13}^{(0)} P_{32}^{(1)} P_{24}^{(2)} P_{41}^{(3)} + P_{14}^{(0)} P_{42}^{(1)} P_{23}^{(2)} P_{31}^{(3)}].
\end{aligned} \tag{3.33}$$

Clearly that $\ell(P)|(i_0 = k) \neq \ell(P)|(i_0 = j)$, $k \neq j$, and one has to average out further the random variables $\ell(P)|_{i_0}$ through all possible states $i_0 = 1, \dots, n$.

We show next how to derive the MCWR sequence. To do so, consider the following trajectory

$$1 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 1$$

and the associated probability vector

$$P_{13}^{(0)} P_{32}^{(1)} P_{24}^{(2)}.$$

The MC $P^{(1)}$ and $P^{(2)}$ generated by the MCWR sequence $P = (P^{(0)}, P^{(1)}, P^{(2)})$ corresponding to $P_{13}^{(0)} P_{32}^{(1)} P_{24}^{(2)}$ are equal to

$$P^{(1)} = \begin{pmatrix} 0 & P_{32}^{(1)} & P_{34}^{(1)} \\ P_{21}^{(1)} & 0 & P_{24}^{(1)} \\ P_{41}^{(1)} & P_{42}^{(1)} & 0 \end{pmatrix} \tag{3.34}$$

$$P^{(2)} = \begin{pmatrix} 0 & P_{24}^{(2)} \\ P_{41}^{(2)} & 0 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \tag{3.35}$$

and

$$P^{(3)} = P_{41}^{(3)} = 1, \tag{3.36}$$

respectively. They are obtained from the matrices

$$\begin{pmatrix} P_{21}^{(1)} & 0 & P_{24}^{(1)} \\ P_{31}^{(1)} & P_{32}^{(1)} & P_{34}^{(1)} \\ P_{41}^{(1)} & P_{42}^{(1)} & 0 \end{pmatrix} \quad (3.37)$$

and

$$\begin{pmatrix} P_{21}^{(2)} & P_{24}^{(2)} \\ P_{41}^{(2)} & 0 \end{pmatrix}, \quad (3.38)$$

as follows: the first—from the original one $P \equiv P^{(0)}$ by eliminating the first row and the third column, the second—from $P^{(1)}$ by eliminating the first row and second column.

For a simple example let $L_{ij} = c$, $i \neq j$, $i, j = 1, 2, 3, 4$ and $P_{ij} = 1/3$, $i \neq j$, $i, j = 1, 2, 3, 4$. Then it is readily seen that our urn model without replacement chooses each trajectory with the following probability

$$P_{i_0 i_1}^{(0)} P_{i_1 i_2}^{(1)} P_{i_2 i_3}^{(2)} P_{i_3 i_0}^{(3)} = 1/6. \quad (3.39)$$

In this case $\ell(P) = 4c$, and solving Equation (3.13) one obtains (by symmetry)

$$P_{0,ij}^* = 1/3, \quad i \neq j, \quad i, j = 1, 2, 3, 4.$$

For another example let

$$L_{12} = L_{24} = 10^{10},$$

with the remaining elements of the matrix $(L_{ij}) = 1$. Let also

$$P_{12} = P_{24} = \varepsilon,$$

where ε is a very small number, say $\varepsilon = 10^{-6}$, with the remaining elements of the matrix (P_{ij}) arbitrary and such that $1 > P_{ij} \gg \varepsilon$. Then it is readily seen that $\ell(P) \approx 10^4$, and solving program (3.13) one obtains (in analogy to the average path problem) that

$$P_{0,12}^* \approx P_{0,24}^* \approx P_{0,43}^* \approx P_{0,31}^* \approx 1 - \varepsilon.$$

Note that in this case the trajectory $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$ will dominate the remaining five trajectories.

REMARK 3.1 *For a symmetric traveling salesman problem one can deal either only with the forward loop or only with the backward loop trajectories in the sense that one can convert each into the other by considering the associated trajectories in the reverse order (see (3.30)). To distinguish between the forward and the backward loop trajectories one can use the following simple rule: depending on whether $i_1 < i_{n-1}$ or $i_1 > i_{n-1}$, each trajectory $i_0, i_1, \dots, i_{n-1}, i_0$ belongs either to the forward or to the backward loop, respectively.*

3.2. Estimation of the Probability of Rare Events in a Network

We consider separately estimation of the probability of rare events for the SLP and the TSP problems.

3.2.1. Estimation of the Probability of Rare Events for SLP

Let $\varphi(\gamma)$ in (3.7) be the following indicator function

$$\varphi(\gamma) = I_{\left\{ \sum_{t=1}^r L_{i_t i_{t+1}} \leq \gamma \right\}}. \quad (3.40)$$

In this case (3.7) reduces to

$$\ell(\gamma) = \mathbb{E}_P \left\{ I_{\left\{ \sum_{t=1}^r L_{i_t i_{t+1}} \leq \gamma \right\}} \right\}. \quad (3.41)$$

Assume that for given matrices L and P , the parameter (temperature) γ is chosen such that $\ell(\gamma)$ is a very small number, say 10^{-6} or less. Consider, for example, the shortest path. One may think that $\gamma = \mathcal{L}_{(1)} + \varepsilon$, where $\mathcal{L}_{(1)}$ corresponds to the shortest path given in (3.1), that is

$$\mathcal{K}_{(1)} \equiv \min_{j=1, \dots, p} \sum_{i \in \mathcal{B}_j} L_i, \quad (3.42)$$

ε is a very small number, and similarly for the longest path.

Using likelihood ratios we have, in analogy to (3.8)

$$\ell(\gamma, P, P_0) = \mathbb{E}_{P_0} \left\{ I_{\left\{ \sum_{t=1}^r L_{i_t i_{t+1}} \leq \gamma \right\}} \tilde{W}_\tau \right\}, \quad (3.43)$$

where $\tilde{W}_\tau$ is given in (3.9).

The LR estimator of $\ell(\gamma, P, P_0)$ is

$$\tilde{\ell}_N(\gamma, P, P_0) = \frac{1}{N} \sum_{j=1}^N I_{\left\{ \sum_{t=1}^{\tau_j} L_{i_t i_{t+1}} \leq \gamma \right\}} \tilde{W}_{\tau, j}. \quad (3.44)$$

The optimal matrix P_0^* in the LR of (3.44) can be obtained by solving, in analogy to (3.13), the following (cross-entropy) program

$$\max_{P_0} D(\gamma, P, P_0) = \max_{P_0} \mathbb{E}_P \left\{ I_{\left\{ \sum_{t=1}^{\tau} L_{i_t i_{t+1}} \leq \gamma \right\}} \sum_{s=1}^{\tau} \ln P_{0, i_s i_{s+1}} \right\}, \quad (3.45)$$

provided that the elements of the matrix P_0 are normalized (see (3.13)). Its stochastic counterpart can be written as

$$\max_{P_0} \widehat{D}_N(\gamma, P, P_0) = \max_{P_0} \left\{ \frac{1}{N} \sum_{j=1}^N I_{\left\{ \sum_{t=1}^{\tau_j} L_{i_t i_{t+1} j} \leq \gamma \right\}} \sum_{k=1}^{\tau_j} \ln P_{0, i_k i_{k+1} j} \right\}, \quad (3.46)$$

provided again that the elements of the matrix P_0 are normalized.

EXAMPLE 3.3 Bridge Network in Figure 3.1 continued

Consider formulas (3.43)–(3.45). Assume that

$$\begin{aligned} P_{12} = P_{13} = 0.5, P_{24} = \varepsilon, P_{23} = 1 - \varepsilon, \\ L_{12} = L_{13} = L_{23} = L_{34} = 1, \text{ and } L_{24} = 1 - \delta, \end{aligned}$$

where ε and δ are small numbers. It is not difficult to see that if we take $\gamma = 2 - \delta$, then solving Equation (3.45) we obtain $P_{0,12}^ = P_{0,24}^* \approx 1$.*

Note that in analogy to (1.27), the stochastic counterpart given in (3.46) is meaningless, since all indicators

$$I_{\left\{ \sum_{t=1}^{\tau_j} L_{i_t i_{t+1} j} \leq \gamma \right\}}, \quad j = 1, \dots, N$$

results in zero with very high probability, provided the sample N is small relative to the reciprocal of the rare event probability $\ell(\gamma)$. To overcome this difficulty we introduce below an auxiliary sequence of vectors (γ_t^*, P_t^*) , $t \geq 0$. We start the algorithm by choosing an initial γ_1 ($\gamma_1 < \gamma$), such that under the original MC P , the probability

$$\ell(\gamma_1) = E_P \{ I_{\left\{ \sum_{t=1}^{\tau} L_{i_t i_{t+1}} > \gamma_1 \right\}} \}$$

is not too small, say $\ell(\gamma_1) = \alpha_0 \approx 10^{-2}$. We set $P_0^* = \bar{P}_0^* \equiv P$ and then, similar to (1.28)–(1.31), iterate in both P and γ as follows.

(a) **Adaptive estimation of P_t^* .** For fixed γ_t we derive P_t^* from the solution of the program

$$\max_{P_t} D(\gamma_t, P_{t-1}^*, P_t) = \max_{P_t} \mathbb{E}_{P_{t-1}^*} \left\{ I \left\{ \sum_{i=1}^{\tau} L_{i,t+1} \leq \gamma_t \right\} \sum_{k=1}^{\tau} \ln P_{t,i_k i_{k+1}} \right\} \quad (3.47)$$

subject to normalization of the transition probabilities (see (3.13)).

The stochastic counterpart of (3.47) can be written as

$$\max_{P_t} \widehat{D}_N(\gamma_t, \bar{P}_{t-1}^*, P_t) = \max_{P_t} \frac{1}{N} \left\{ \sum_{j=1}^N I \left\{ \sum_{i=1}^{\tau_j} L_{i,t+1,j} \leq \gamma_t \right\} \sum_{k=1}^{\tau_j} \ln P_{t,i_k i_{k+1},j} \right\} \quad (3.48)$$

subject to normalization of the transition probabilities.

(b) **Adaptive estimation of γ_t^* .** For a given MC P_{t-1}^* , the optimal γ_t^* can be obtained from the following simple one-dimensional root-finding program

$$\min \gamma_t \text{ s.t. } \mathbb{E}_{P_{t-1}^*} \left\{ I \left\{ \sum_{i=1}^{\tau} L_{i,t+1} \leq \gamma_t \right\} \right\} \geq \rho, \quad (3.49)$$

where, say $10^{-2} \leq \rho \leq 10^{-1}$. The stochastic counterpart of (3.49) is

$$\min \gamma_t \text{ s.t. } \frac{1}{N} \left\{ \sum_{j=1}^N I \left\{ \sum_{i=1}^{\tau_j} L_{i,t+1,j} \leq \gamma_t \right\} \right\} \geq \rho. \quad (3.50)$$

The resulting algorithm for estimating $\ell(\gamma)$ can be written, in analogy to Algorithm (1.1) as

ALGORITHM 3.1

1. Generate N trajectories $\{(i_1, i_2, \dots, i_{\tau}), j\}$; $j = 1, 2, \dots, N$ from the fixed matrix $\bar{P}_1 = P_0^* \equiv P$ and solve Equation (3.50). Denote the initial solution by $\bar{\gamma}_1^*$.
2. Use the **same** N trajectories $\{(i_1, i_2, \dots, i_{\tau}), j\}$, $j = 1, 2, \dots, N$ and solve analytically the stochastic Equation (3.48) for $\gamma = \bar{\gamma}_1^*$ and $\bar{P}_0^* \equiv P$. Denote the solution by $\bar{P}_1^* = \bar{P}_1^*(\bar{\gamma}_1^*)$.
3. Generate N **new** trajectories $\{(i_1, i_2, \dots, i_{\tau}), j\}$, $j = 1, 2, \dots, N$ from $\bar{P}_1^*$ and repeat Steps 1 and 2, again with $\bar{P}_0^*$ replaced by $\bar{P}_1^*$. Denote the corresponding solutions by $\bar{\gamma}_2^*$ and $\bar{P}_2^*$, respectively. Denote the solutions at stage t by $\bar{\gamma}_t^*$ and $\bar{P}_t^*$, respectively. It is readily seen that, in fact,

$$\bar{\gamma}_t^* = \mathcal{L}_{((1-\rho)N)},$$

where $\mathcal{L}_{(j)}$ is the j -th order statistics of the sequence $\mathcal{L}_j \equiv \sum_{i=1}^{\tau_j} L_{i,t+1,j}$, $j = 1, \dots, N$. For $\rho = 10^{-2}$ this reduces to

$$\bar{\gamma}_t^* = \mathcal{L}_{(\frac{99}{100}N)}.$$

4. If $\bar{\gamma}_t^* \geq \gamma$, set $\bar{\gamma}_t^* \equiv \gamma$ and solve the stochastic Equation (3.48) for $\bar{\gamma}_t^* = \gamma$. Denote the solution as $\bar{P}_{t+1}^*$ and stop; otherwise set $t \equiv t + 1$ and repeat step 3. After stopping:
 - Estimate $\ell(P)$ using the LR estimate (3.44), with P_0 replaced by $\bar{P}_{t+1}^*$.

The monotonicity of the sequence $\bar{\gamma}_0^*, \bar{\gamma}_1^*, \dots$ is an inherent part of Algorithm 3.1. In analogy to Proposition 9.4.1, pp. 329–330 of Rubinstein and Melamed (1998) it can be shown that the random sequence $\bar{\gamma}_0^*, \bar{\gamma}_1^*, \dots$ generated by the Algorithm 3.1 is monotonic nondecreasing and in analogy to Lieber (1998), it can be shown that as the sample size $N \rightarrow \infty$, the target value γ will be reached with high probability in a finite number of iterations.

3.2.2. Estimation of Probabilities of Rare Events for TSP

For the TSP Algorithm 3.1 remains the same, provided (3.48) and (3.50) are replaced by

$$\max_{P_t} \left\{ \hat{D}_N(\gamma_t, \hat{P}_{t-1}^*, P_t) = \frac{1}{N} \sum_{j=1}^N I_{\left\{ \sum_{t=1}^{n-1} L_{i_{t-1}i_tj} + L_{i_{n-1}i_0j} \leq \gamma_t \right\}} \left(\sum_{k=1}^{n-1} \ln P_{i_{k-1}i_kj} + \ln P_{i_{n-1}i_0j} \right) \right\} \quad (3.51)$$

and

$$\min \gamma_t \text{ s.t. } \frac{1}{N} \left\{ \sum_{j=1}^N I_{\left\{ \sum_{t=1}^{n-1} L_{i_{t-1}i_tj} + L_{i_{n-1}i_0j} \leq \gamma_t \right\}} \right\} \geq \rho, \quad (3.52)$$

respectively.

3.3. Combinatorial Optimization

In this subsection we present our main Algorithm 3.2 for solving the shortest (longest) path and the TSP problem. Before proceeding, note that the shortest path problem has been chosen for illustration purposes only, since there are plenty of very efficient algorithms for finding the shortest path.

We shall first present our important observation, namely, we shall show how to *link estimation of the probability of rare events with combinatorial optimization*, (see also Subsection 1.3). In particular, we shall show that the optimal solution $\bar{\gamma}_t^*$ of the programs (3.50) and (3.52) can be taken as an estimate of the optimal solution of SLP and TSP, respectively. This implies that estimating the probability of rare events with Algorithm 3.1, one can simultaneously estimate the optimal solution of a combinatorial program.

Consider for concreteness TSP (the treatment for SLP is similar). Denote (see (3.24))

$$\mathcal{L}_j = \left[\sum_{t=1}^{n-1} L_{i_{t-1}i_tj} + L_{i_{n-1}i_0j} \right], \quad j = 1, 2, \dots, |\mathcal{X}|, \quad (3.53)$$

the length of the j -th tour in TSP, where $\{\mathcal{X}\}$ denotes the set of all possible trajectories in the network and $|\mathcal{X}|$ is the cardinality of the set $\{\mathcal{X}\}$. Note that for TSP we have $|\mathcal{X}| = (n-1)!$.

The shortest tour is clearly

$$\mathcal{K}_{(1)} = \min_{j=1,2,\dots,|\mathcal{X}|} \mathcal{K}_j, \text{ where } \mathcal{K}_{(1)} \text{ is given (3.42).} \quad (3.54)$$

Assume that $\mathcal{K}_{(1)}$ is unique and, in analogy to (3.41), consider

$$\ell(\gamma) = \mathbb{E}_P \left\{ I \left\{ \sum_{t=1}^{n-1} L_{i_{t-1}i_t} + L_{i_{n-1}i_0} \leq \gamma \right\} \right\}. \quad (3.55)$$

It is obvious that if $\gamma < \mathcal{K}_{(1)}$ then $\ell(\gamma) = 0$ regardless of the choice of P in (3.55); and if $\gamma = \mathcal{K}_{(1)}$ and P is uniformly distributed, that is

$$P_{ij} = \begin{cases} \frac{1}{n-1} & \text{if } i \neq j \\ 0 & \text{otherwise,} \end{cases} \quad (3.56)$$

then $\ell(\gamma) = \frac{1}{(n-1)!}$. The opposite is also true in the sense that if in (3.55) we take $\ell(\gamma) = \frac{1}{(n-1)!}$ and P_{ij} as in (3.56), then the root γ of (3.55) equals to $\mathcal{L}_{(1)}$.

Consider for a moment Algorithm 3.1 but with stochastic counterparts (3.51) and (3.52) replaced by their original expected values, which can be written in analogy to (3.47) and (3.49) as

$$\max_{P_t} D(\gamma_t, P_{t-1}^*, P_t) = \max_{P_t} \mathbb{E}_{P_{t-1}^*} \left\{ I \left\{ \sum_{t=1}^{n-1} L_{i_{t-1}i_t} + L_{i_{n-1}i_0} \leq \gamma_t \right\} \left(\sum_{k=1}^{n-1} \ln P_{t,i_{k-1}i_k} + \ln P_{t,i_{n-1}i_0} \right) \right\} \quad (3.57)$$

and

$$\min \gamma_t \text{ s.t. } \mathbb{E}_{P_{t-1}^*} \left\{ I \left\{ \sum_{t=1}^{n-1} L_{i_{t-1}i_t} + L_{i_{n-1}i_0} \leq \gamma_t \right\} \right\} \geq \rho, \quad (3.58)$$

respectively. We shall call such designed Algorithm 3.1, the *deterministic rare-event* (DRE) Algorithm. Note that the DRE Algorithm has no practical use at all, since knowledge of the expected values of (3.51) and of (3.52) implies knowledge of all trajectories $j \in \mathcal{X}$ and their associated tours (function values) $\mathcal{L}_j$. Knowledge of all trajectories is equivalent in turn to the exhaustive enumeration, which is clearly meaningless. Although the DRE Algorithm is practically useless, nevertheless it shades some light on the original (stochastic) Algorithm 3.1, which we also call the *Stochastic Rare-Event* (CRE) Algorithm and which will be considered as a candidate algorithm for combinatorial optimization. We consider separately the DRE and the SRE Algorithms.

3.3.1. DRE Algorithm and its Extensions

Assume for a moment that $\gamma = \mathcal{K}_{(1)}$ and consider the following two cases with respect to the initial value γ_1 in (3.58): (a) $\gamma_1^* = \gamma$ and (b) $\gamma_1^* > \gamma$.

(a) In this case the multi-stage Equation (3.47) reduces to the following single-stage one

$$\max_{P_1} D(\gamma_t, P, P_1) = \max_{P_1} \mathbb{E}_P \left\{ I_{\{\sum_{t=1}^{n-1} L_{i_{t-1}i_t} + L_{i_{n-1}i_0} \leq \gamma_t\}} \left(\sum_{k=1}^{n-1} \ln P_{1,i_{k-1}i_k} + \ln P_{1,i_{n-1}i_0} \right) \right\}, \quad (3.59)$$

subject to normalization of the transition probabilities, (see (3.13)). Since the CE function $D(\gamma, P, P_1)$ is convex with respect to each element P_{ij} of P , it is readily seen that assuming $\mathcal{L}_{(1)}$ is unique and P_{ij} is positive when L_{ij} is positive, the optimal solution P_1^* of (3.59) becomes the degenerated one (see Lemma 7.1 and Lemma 3.1) in the sense that

1. *Only a single* element at each row of the matrix P_1^* equals to unity, while the remaining elements of $P_1^* = (P_{1,ij}^*)$ equal zero.
2. The elements of P_1^* equal to unity define the shortest tour in TSP.

As an example, consider the four node non-symmetric TSP with the P in (3.29). If, for example, the trajectory

$$1 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$$

corresponds to the shortest route then, regardless of the values of the elements P_{ij} , the DRE Algorithm will converge (in a single iteration) to the following matrix

$$P_1^* = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}. \quad (3.60)$$

We denote the matrix P_1^* with the transition probabilities of type (3.60) [corresponding to the shortest tour] by P^* and call it the *optimal degenerated MC* (ODMC).

(b) Arguing as in Example 7.1 (see also Lemma 7.1 and Lemma 3.1) it is readily seen that P_t^* in the DRE Algorithm converges deterministically to ODMC P^* in a finite number of iterations $t = t(\rho)$, provided $\gamma_1 > \gamma$. Vice versa also holds in the sense that if we take

$$\ell(\gamma) = \frac{1}{|\mathcal{X}|} = \frac{1}{(n-1)!},$$

then the root $\gamma = \mathcal{K}_{(1)}$, (presenting actually the solution of the program (3.58)), will be found again in a finite number of iterations. Moreover, P_t^* will converge again deterministically to ODMC P^* (see also Subsection 1.3).

In most combinatorial problems, like SLP, graph coloring, etc., the cardinality $|\mathcal{X}|$ is not available. But as follows from Algorithms DRE and SRE, neither of them actually relies on the availability of $|\mathcal{X}|$, since both Algorithms, DRE and SRE just keep iterating in both γ_t and P_t until stopping.

We finally show how one could apply the DRE Algorithm to the original combinatorial problems, like (3.1) and (3.3), that is in situations, where the matrix P is absent (missing). In this case, we *convert* the original *deterministic* combinatorial problems (3.1) and (3.3)

into *stochastic* ones (suitable for the DRE Algorithm) by introducing an auxiliary, quite arbitrary matrix P_0 which replaces the “missing” one, P . It is readily seen that the DRE Algorithm is robust with respect to the choice of P_0 , provided it chosen as per (3.21). Typically one would choose P_0 as in (3.56), that is, with elements P_{ij} uniformly distributed over each state i , or as the one obtained by normalizing the elements L_{ij} of the matrix L with respect to each row.

So far we linked (in analogy to Subsection 1.3) the probabilities of rare events $\ell(\gamma)$ to the root (optimal solution) $\mathcal{K}_{(1)} \equiv \gamma$, by showing that the DRE version of Algorithm 3.1 finds $\mathcal{K}_{(1)}$ in a finite number of iterations $t = t(\rho)$, provided the original deterministic combinatorial problems (3.1) and (3.3) are converted into associated stochastic ones by introducing an auxiliary P_0 .

At this end, let us pose the following question. Are there any alternatives to the rare-event indicator function $I(\cdot)$ in (3.55) in order to find $\mathcal{K}_{(1)}$ (in a single or multiple-iteration) using a DRE type algorithm? The answer is positive. Consider, for example, the DRE Algorithm 3.1 with the indicator function $I(\cdot)$ replaced by the Boltzmann function

$$H(\gamma) = \exp\left(-\frac{L}{\gamma}\right), \quad \gamma > 0$$

and call it, the *deterministic Boltzmann* (DB) Algorithm. It is readily seen that after stopping with the DB Algorithm

1. One obtains (in analogy to the DRE Algorithm) that the DB version converges deterministically to ODMC P^* in a finite number of iterations, provided $\gamma_1 > 0$ and P_0 satisfies (3.21).
2. In contrast to the DRE Algorithm, $\gamma^* \neq \mathcal{K}_{(1)}$ for the DB version. This, however, must not cause difficulty, since P^* defines uniquely the optimal solution $\mathcal{K}_{(1)}$.

Note finally that both, the DRE and DB Algorithms also converges deterministically to P^* without resorting to the problem (3.58), that is by updating P_t^* alone according to (3.57) with $\gamma = \text{const.}$

3.3.2. SRE Algorithm and its Extensions

As for the DRE Algorithm consider again (a) $\gamma_1^* = \gamma$ and (b) $\gamma_1^* > \gamma$.

(a) This case, as the entire DRE Algorithm, has no practical use, since estimation of $\ell(\gamma)$ for $\gamma = \mathcal{L}_{(1)}$ (in a single iteration) requires a sample N of at least $|\mathcal{X}| = (n-1)!$ trajectories, which is clearly meaningless.

(b) This is the original version of Algorithm 3.1, which might be viewed as the only practical version considered so far. Clearly, as $N \rightarrow \infty$ we have that the estimate $\bar{P}_t^*$ of Algorithm 3.1 converges almost surely to ODMS P^* in a finite number t of iterations. The challenge with Algorithm 3.1 is (for a given network configuration and a given distance

matrix $L = (L_{ij})$, is to choose N and ρ such that it will converge to a very small vicinity of $\mathcal{K}_{(1)}$ with high probability.

We say that an Algorithm performs accurately if

$$\frac{\bar{\mathcal{L}}_{t,(1)} - \mathcal{K}_{(1)}}{\mathcal{K}_{(1)}} \leq \varepsilon_1, \quad (3.61)$$

with high probability. Here $\bar{\mathcal{K}}_{t,(1)}$ is the estimate of $\mathcal{K}_{(1)}$ generated by Algorithm 3.1 and ε_1 is a small number, say $\varepsilon_1 = 10^{-2}$. Note that for Algorithm 3.1 we have that $\bar{\mathcal{L}}_{t,(1)} \equiv \bar{\gamma}_t^*$.

Our extensive numerical results suggest that

1. The Boltzmann function $H(\gamma)$ is the most natural and most appropriate one. If not stated otherwise we shall use below the Boltzmann function $H(\cdot)$ instead of the indicator function $I(\cdot)$ in Algorithm 3.1. We call such modification of the SRE Algorithm 3.1 as the *stochastic Boltzmann* (SB) Algorithm.
2. Taking a sample size of $N = O(n^2)$ trajectories at each step of Algorithm 3.1 (with the Boltzmann function instead of $I(\cdot)$) it stops after $t = O(n)$ iterations, delivering accurate results in the sense of (3.61).

Below we discuss in more details the SB Algorithm, which is our main Algorithm.

3.3.3. The Stochastic Boltzmann Algorithm

Unless stated otherwise, we shall deal below with the shortest path problem. Taking into account that for combinatorial problems the sample function L , associated with the trajectory $i_1, i_2, \dots, i_\tau$, is an additive function, that is

$$\mathcal{L} = \sum_{t=1}^{\tau} L_{i_t i_{t+1}}, \quad (3.64)$$

we can write the Boltzmann function $H(\gamma)$ as

$$H(\gamma) = \prod_{t=1}^{\tau} \exp\left(-\frac{L_{i_t i_{t+1}}}{\gamma}\right) = \prod_{t=1}^{\tau} H_{i_t i_{t+1}}, \quad (3.65)$$

where in analogy to (3.6)

$$i_{\tau+1} \equiv i_\tau, \quad L_{i_\tau i_{\tau+1}} = 0. \quad (3.66)$$

For the longest path we reverse the minus sign in (3.65), that is, we use

$$H^+(\gamma) = \prod_{t=1}^{\tau} \exp\left(+\frac{L_{i_t i_{t+1}}}{\gamma}\right) = \prod_{t=1}^{\tau} H_{i_t i_{t+1}}^+ \quad (3.67)$$

instead of (3.65).

In analogy to Algorithm 3.1 we consider separately adaptive estimation of P_t^* and γ_t^* .

(a) **Adaptive estimation of P_t^* .** In analogy to (3.47) we derive P_t^* from the solution of the program

$$\max_{P_t} D(\gamma, P_{t-1}^*, P_t) = \max_{P_t} \mathbb{E}_{P_{t-1}^*} \left\{ \prod_{s=1}^{\tau} H_{i_s i_{s+1}}(\gamma) \sum_{k=1}^{\tau} \ln P_{t, i_k i_{k+1}} \right\} \quad (3.68)$$

subject to normalization of the transition probabilities (see (3.13)).

The stochastic counterpart of (3.68) can be written as

$$\max_{P_t} \left\{ \widehat{D}_N(\gamma, \widehat{P}_{t-1}^*, P_t) = \frac{1}{N} \sum_{j=1}^N \left[\prod_{s=1}^{\tau_j} H_{i_s i_{s+1}, j}(\gamma) \sum_{k=1}^{\tau_j} \ln P_{t, i_k i_{k+1}, j} \right] \right\}. \quad (3.69)$$

We shall show in (3.77) and (3.78) below how to solve Equations (3.68) and (3.69) analytically.

(b) **Adaptive estimation of γ_t^* .** For fixed P_{t-1}^* the optimal γ_t^* can be found (in analogy to (3.49)) from the following simple one-dimensional root-finding program

$$\min \gamma_t \text{ s.t. } \mathbb{E}_{P_{t-1}^*} \left\{ \prod_{s=1}^{\tau} H_{i_s i_{s+1}}(\gamma_t) \right\} \geq \rho, \quad (3.70)$$

where $10^{-6} \leq \rho \leq 10^{-2}$. Note that $H(\gamma)$ is monotone non decreasing in γ . The stochastic counterpart of (3.71) is

$$\min \gamma_t \text{ s.t. } \frac{1}{N} \sum_{i=1}^N \left[\prod_{s=1}^{\tau_i} H_{i_s i_{s+1}, j}(\gamma_t) \right] \geq \rho. \quad (3.71)$$

There exists a trade-off between Equations (3.69) and (3.71) in the sense that the convergence rate of (3.69) increases as γ decreases, but, at the same time, cooling the temperature γ too fast may result in a local minimum. The goal of (3.71) with $10^{-6} \leq \rho \leq 10^{-2}$ is to resolve the cooling schedule adaptively, namely to

1. Keep γ_t small for all $t \geq 0$, that is to ensure fast convergence of $\widehat{P}_t^*$ in (3.69) to ODM.
2. Ensure that for small γ , the function in (3.69) will be still computational tractable (being away from computational zero), that is

$$\prod_{s=1}^{\tau_j} H_{i_s i_{s+1}, j}(\gamma) \sum_{k=1}^{\tau_j} \ln P_{t, i_k i_{k+1}, j} > \eta, \quad (3.72)$$

where $\eta > 0$ is a small number, and that program (3.69) is not trapped in local minima.

For the longest path we replace $10^{-6} \leq \rho \leq 10^{-2}$ with $10^2 \leq \rho \leq 10^6$.

It readily follows from (3.68) and (3.70) that the solution γ_t^* of (3.70) is monotone non increasing in the parameter t .

EXAMPLE 3.4 Bridge Network in Figure 3.1 continued

In this case (3.68) reduces to

$$\begin{aligned}
\max_{P_t} D(P_{t-1}^*, P_t) &= \max_{P_t} \{ H_{12} H_{24} P_{t-1,12}^* P_{t-1,24}^* (\ln P_{t,12} + \ln P_{t,24}) \\
&\quad + H_{12} H_{23} H_{34} P_{t-1,12}^* P_{t-1,23}^* (\ln P_{t,12} + \ln P_{t,23}) \\
&\quad + H_{13} H_{34} P_{t-1,13}^* \ln P_{t,13} \} \\
\text{s.to } P_{t,12} + P_{t,13} &= 1, \quad P_{t,23} + P_{t,34} = 1.
\end{aligned} \tag{3.73}$$

Using Lagrange multipliers we obtain

$$\begin{aligned}
P_{t,12}^* &= \frac{H_{12} H_{24} P_{t-1,12}^* P_{t-1,24}^* + H_{12} H_{23} H_{34} P_{t-1,12}^* P_{t-1,23}^*}{H_{12} H_{24} P_{t-1,12}^* P_{t-1,24}^* + H_{12} H_{23} H_{34} P_{t-1,12}^* P_{t-1,23}^* + H_{13} H_{34} P_{t-1,13}^*}, \\
P_{t,13}^* &= \frac{H_{13} H_{34} P_{t-1,13}^*}{H_{12} H_{24} P_{t-1,12}^* P_{t-1,24}^* + H_{12} H_{23} H_{34} P_{t-1,12}^* P_{t-1,23}^* + H_{13} H_{34} P_{t-1,13}^*}, \\
P_{t,23}^* &= \frac{H_{12} H_{23} H_{34} P_{t-1,12}^* P_{t-1,23}^*}{H_{12} H_{24} P_{t-1,12}^* P_{t-1,24}^* + H_{12} H_{23} H_{34} P_{t-1,12}^* P_{t-1,23}^* + H_{13} H_{34} P_{t-1,13}^*}, \\
P_{t,24}^* &= \frac{H_{12} H_{24} P_{t-1,12}^* P_{t-1,24}^*}{H_{12} H_{24} P_{t-1,12}^* P_{t-1,24}^* + H_{12} H_{23} H_{34} P_{t-1,12}^* P_{t-1,23}^* + H_{13} H_{34} P_{t-1,13}^*}.
\end{aligned} \tag{3.74}$$

Consider the stochastic counterpart of (3.73). Using again Lagrange multipliers we obtain in analogy to (3.74)

$$\begin{aligned}
\hat{P}_{t,12}^* &= \frac{H_{12} H_{24} \bar{P}_{t-1,12} \bar{P}_{t-1,24} + H_{12} H_{23} H_{34} \bar{P}_{t-1,12} \bar{P}_{t-1,23} \bar{P}_{t-1,34}}{H_{12} H_{24} \bar{P}_{t-1,12} \bar{P}_{t-1,24} + H_{12} H_{23} H_{34} \bar{P}_{t-1,12} \bar{P}_{t-1,23} \bar{P}_{t-1,34} + H_{13} H_{34} \bar{P}_{t-1,13} \bar{P}_{t-1,34}}, \\
\hat{P}_{t,13}^* &= \frac{H_{13} H_{34} \bar{P}_{t-1,13} \bar{P}_{t-1,34}}{H_{12} H_{24} \bar{P}_{t-1,12} \bar{P}_{t-1,24} + H_{12} H_{23} H_{34} \bar{P}_{t-1,12} \bar{P}_{t-1,23} \bar{P}_{t-1,34} + H_{13} H_{34} \bar{P}_{t-1,13} \bar{P}_{t-1,34}}, \\
\hat{P}_{t,23}^* &= \frac{H_{12} H_{23} H_{34} \bar{P}_{t-1,12} \bar{P}_{t-1,23} \bar{P}_{t-1,34}}{H_{12} H_{24} \bar{P}_{t-1,12} \bar{P}_{t-1,24} + H_{12} H_{23} H_{34} \bar{P}_{t-1,12} \bar{P}_{t-1,23} \bar{P}_{t-1,34} + H_{13} H_{34} \bar{P}_{t-1,13} \bar{P}_{t-1,34}}, \\
\hat{P}_{t,24}^* &= \frac{H_{12} H_{24} \bar{P}_{t-1,12} \bar{P}_{t-1,24}}{H_{12} H_{24} \bar{P}_{t-1,12} \bar{P}_{t-1,24} + H_{12} H_{23} H_{34} \bar{P}_{t-1,12} \bar{P}_{t-1,23} \bar{P}_{t-1,34} + H_{13} H_{34} \bar{P}_{t-1,13} \bar{P}_{t-1,34}},
\end{aligned} \tag{3.75}$$

where

$$\bar{P}_{t,rs} = \frac{N_{t,rs}}{N_{t,r}},$$

$N_{t,rs}$ is the number of times the trajectories pass through the link r (passing through the edge from node r to node s) at the t -th iteration and $N_{t,r}$ is the number of times the trajectories pass through the node r at the t -th iteration.

Consider again the three trajectories

$$1 \rightarrow 2 \rightarrow 4; \quad 1 \rightarrow 3 \rightarrow 4; \quad 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$$

of the MC (see (3.14)), and their associated probabilities

$$P_{0,12}P_{0,24}; P_{0,13}; P_{0,12}P_{0,23}$$

(see (3.16)).

Assume that the initial values of the elements $P_{0,ij}$ of matrix P_0 are given as

$$P_{0,12} = 2/3, P_{0,13} = 1/3, P_{0,23} = P_{0,24} = 1/2.$$

This implies that each of the three paths in (3.15) will be chosen with equal probability = $1/3$. Assume for a moment that the first trajectory $1 \rightarrow 2 \rightarrow 4$ is the shortest one. It readily follows from (3.74) that for a fixed γ we have

$$P_{1,12}^* > P_{1,13}^*; P_{1,24}^* > P_{1,23}^*$$

and similarly if any of the two remaining trajectories is the shortest one.

Proceeding recursively with (3.74) we obtain for $t \geq 1$ that

$$P_{t,12}^* > P_{t,13}^*; P_{t,24}^* > P_{t,23}^*.$$

As $t \rightarrow \infty$ we have that

$$P_{t,12}^* \rightarrow 1; P_{t,24}^* \rightarrow 1$$

(deterministically) and similarly, as $t \rightarrow \infty$ and $N \rightarrow \infty$

$$\hat{P}_{t,12}^* \rightarrow 1; \hat{P}_{t,24}^* \rightarrow 1$$

(almost surely). It is readily seen that the almost surely convergence $\hat{P}_{t,12}^* \rightarrow 1; \hat{P}_{t,24}^* \rightarrow 1$ is robust with respect to perturbations in the initial values of $P_{0,12}, P_{0,13}, P_{0,23}, P_{0,24}$. Finally, similarly to Lemma 7.1 of the Appendix it is possible to choose $\gamma > 0$ (small enough), such that both inequalities

$$\hat{P}_{t,12}^* \geq 1 - \varepsilon; \hat{P}_{t,24}^* \geq 1 - \varepsilon$$

hold after a single iteration with (3.75), provided $P_{0,12}, P_{0,13}, P_{0,23}, P_{0,24}$ are positive, and $\varepsilon > 0$. To provide more insight, let

$$L_{12} = L_{13} = 1, L_{23} = L_{24} = 2, L_{34} = 3,$$

$\gamma = 1$, and let the initial MC P_0 be given as

$$P_0 = \begin{pmatrix} 0 & 1/2 & 1/2 & 0 \\ 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (3.76)$$

Then the below table can be obtained by straightforward calculations using (3.74).

It follows from Table 3.1 that the shortest path $L_{12} + L_{24} = 3$ corresponding to the trajectory $1 \rightarrow 2 \rightarrow 4$ will be found in 4 iterations. It can be also readily checked that

1. If we solve (3.70) for $\rho = 10^{-2}$ say, then P_t^* in (3.74) converges to ODMC in $t = 3$ iterations. The resulting values of γ_t are $\gamma_1 = 0.8, \gamma_2 = 0.6$ and $\gamma_3 = 0.5$, respectively.

Table 3.1. Convergence of P_t^* in (3.74) to ODMC for the bridge network.

t	$P_{t,12}^*$	$P_{t,13}^*$	$P_{t,23}^*$	$P_{t,24}^*$	$\mathbb{E}_{P_{t-1}^*} \left\{ \prod_{s=1}^t H_{i_s i_{s+1}} (\gamma = 1) \right\}$
1	0.6	0.4	0.95	0.05	0.22
2	0.8	0.2	0.99	0.01	0.36
3	0.9	0.1	0.998	0.002	0.43
4	1.0	0.0	1.0	0.0	0.45

2. The number of iterations t increases as the traveling distances L_{ij} defined on all three trajectories

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 4; \quad 1 \rightarrow 2 \rightarrow 4; \quad 1 \rightarrow 3 \rightarrow 4.$$

get closer to each other. If for example we take $L_{34} = 2.2$ instead of $L_{34} = 3$, all other data remaining the same, then it is not difficult to check that convergence to ODMC takes place in 6 iterations instead of 4.

3. P_t^* in (3.74) converges to ODMC in a single iteration if we choose $\gamma = -\frac{1}{\ln \varepsilon}$. If, for instance, we take $\varepsilon = e^{-k}$, then $\gamma = \frac{1}{k}$ and it is readily seen that

$$\mathbb{E}_{P_0} \left\{ \prod_{s=1}^t H_{i_s i_{s+1}} (\gamma = k^{-1}) \right\} \approx e^{-3k}$$

which is a very small number provided k is large, say $k = 10$.

Note, however, that convergence of the estimates $\hat{P}_{t,ij}^*$ (see (3.75)) of $P_{t,ij}^*$ to ODMC in a single iteration might be problematic, since for very small γ the inequalities (3.71), (3.72) will be typically violated and as a result, the estimate $\hat{P}_{t,ij}^*$ will be typically unreliable (i.e., have large relative variance). To avoid such a situation, multiple iterations based on (3.71) and (3.72) must be used. In addition, multiple iterations have some other nice features discussed below.

Now we can generalize Equations (3.74) and (3.75) of Example 3.4 in the sense that we can derive analytic solutions for Equation (3.68) and (3.69), respectively. To proceed, let $\{rs\}$ denote the set of all trajectories passing through the link rs and let $\{r\}$ denote the set of all trajectories passing through node r . Then it is readily seen that, at stage t , the optimal (analytic) solution of (3.68) is

$$P_{t,rs}^* = \frac{\sum_{j \in \{rs\}} \prod_{l=1}^{\tau_j} H_{i_l i_{l+1}} P_{t-1, i_l i_{l+1}}^*}{\sum_{k \in \{r\}} \prod_{l=1}^{\tau_k} H_{i_l i_{l+1}} P_{t-1, i_l i_{l+1}}^*}. \quad (3.77)$$

The optimal solution $\hat{P}_{t,rs}^*$ of the stochastic counterpart (3.69) can be written in analogy to (3.77) as

$$\hat{P}_{t,rs}^* = \frac{\sum_{1 \leq j \leq N: j \in \{rs\}} \prod_{l=1}^{\tau_j} H_{i_l i_{l+1}}}{\sum_{1 \leq j \leq N: k \in \{r\}} \prod_{l=1}^{\tau_k} H_{i_l i_{l+1}}}. \quad (3.78)$$

Note that $1 \leq j \leq N : j \in \{rs\}$ and $1 \leq j \leq N : j \in \{r\}$ in (3.78) denote the set of all trajectories passing through the link rs and the node r , respectively from the set of N generated trajectories, while $j \in \{rs\}$ and $j \in \{r\}$ in (3.77) denote the set of all trajectories passing through the link rs and the node r , respectively from the set of *all possible* trajectories in the network.

Note also that

1. Equation (3.78) is quite general and easy to implement.
2. As $N_t \rightarrow \infty$ each element $\hat{P}_{t,rs}^*$ converges almost surely to the associated one $P_{t,rs}^*$.
3. In the traveling salesman problem the denominator in (3.78) is the same for all elements $\hat{P}_{t,rs}^*$, since every trajectory passes through all cities.

Note also that Equation (3.77), as well as the entire deterministic Boltzmann (DB) Algorithm, has no practical use, since it requires explicit knowledge of all trajectories and their associated values of the objective function (tour length)

$$\mathcal{L}_j = \sum_{t=1}^{\tau_j} L_{i_t i_{t+1}}, \quad j = 1, 2, \dots, |\mathcal{X}|,$$

where, as before, $\{\mathcal{X}\}$ denotes the set of all possible trajectories in the network. Knowledge of all trajectories is equivalent to exhaustive enumeration, which is clearly meaningless. As we shall see below, the stochastic counterpart (3.78) of (3.77) is very useful, however, provided the sample size (the number of generated trajectories) is of order of n^2 .

Below we state a lemma about convergence of the recursive procedures (3.68) and (3.69) (see also Lemma 7.1). Its proof is omitted.

LEMMA 3.1 *Consider the CE recursive procedures (3.68) and (3.69). Assume that the number of nodes n in the network is finite, that the shortest path $\mathcal{K}_{(1)}$ in (3.42)*

differs from the remaining ones at least by a fixed quantity say, η , and finally, that all initial transition probabilities of the MC P_0 are chosen as per (3.4). Then

1. *For fixed γ , the optimal MC P_t^* with elements $P_{t,rs}^*$, converges asymptotically in t to the ODMC with the transition probability $P_{t,rs,(1)}^*$ corresponding to $\mathcal{L}_{(1)}$. The inequality*

$$P_{t,rs,(1)}^* \geq 1 - \varepsilon, \quad \varepsilon > 0 \tag{3.79}$$

holds simultaneously for all states $i = 1, \dots, n$ after a finite number of iterations with (3.68).

2. *There exists $\gamma = \gamma(\varepsilon, \eta, P_0) > 0$, such that the inequality*

$$P_{t,rs,(1)}^* \geq 1 - \varepsilon$$

holds simultaneously for all states $i = 1, \dots, n$ after a single iteration with (3.68).

3. Both items 1 and 2 are valid for the stochastic counterpart (3.69), provided $N(\varepsilon) \rightarrow \infty$ as $\varepsilon \rightarrow 0$.

Now we can turn to the modified version of (3.78). We provide first some insight into this matter by proceeding with the bridge network in Example 3.4.

EXAMPLE 3.5 Example 3.4 continued

Assume that while simulating the bridge network at some stage t at least one of the three feasible trajectories given in (3.15) has not been selected. Let, for concreteness, this trajectory be $1 \rightarrow 2 \rightarrow 4$. In that case (3.75) reduces to

$$\begin{aligned}\hat{P}_{t,12}^* &= \frac{H_{12}H_{23}H_{34}\bar{P}_{t-1,12}\bar{P}_{t-1,23}\bar{P}_{t-1,34}}{H_{12}H_{23}H_{34}\bar{P}_{t-1,12}\bar{P}_{t-1,23}\bar{P}_{t-1,34} + H_{13}H_{34}\bar{P}_{t-1,13}\bar{P}_{t-1,34}}, \\ \hat{P}_{t,13}^* &= \frac{H_{13}H_{34}\bar{P}_{t-1,13}\bar{P}_{t-1,34}}{H_{12}H_{23}H_{34}\bar{P}_{t-1,12}\bar{P}_{t-1,23}\bar{P}_{t-1,34} + H_{13}H_{34}\bar{P}_{t-1,13}\bar{P}_{t-1,34}}, \\ \hat{P}_{t,23}^* &= 1, \\ \hat{P}_{t,24}^* &= 0,\end{aligned}\tag{3.80}$$

since $\bar{N}_{t-1,24} = 0$. Assume also, without loss of generality, that trajectory $1 \rightarrow 2 \rightarrow 4$ is the shortest one. Clearly, in this case, (3.80) will converge to the wrong answer, namely to a local minimum. To avoid such an undesirable situation we employ the following simple rule: **Take instead of $\hat{P}_{t,24}^* = 0$ its old value, namely**

$$\hat{P}_{t,24}^* \equiv \hat{P}_{t-1,24}^*,\tag{3.81}$$

and then normalize the remaining elements in the second row of the MC $P_{t,ij}^*$. After normalization, the new value of $\hat{P}_{t,23}^*$ becomes

$$\hat{P}_{t,23}^* = 1 - \hat{P}_{t-1,24}^*\tag{3.82}$$

instead of $\hat{P}_{t,23}^* = 1$ as per (3.80). It is readily seen that combining (3.80) with (3.81) and (3.82) one still remains in the framework of (3.75).

With (3.81) and (3.82) to hand, let us turn to the general case. More specifically, let us apply solution (3.78) of (3.69) for a moderate sample N , say $N = n$ trajectories. It can be seen that by doing so some elements $\hat{P}_{t-1,rs}^*$ in (3.78) will be zeros with high probability. This in turn imply convergence to a local minimum. To avoid such a situation of “zeros” we argue exactly as in (3.81), (3.82), namely, instead of $\hat{P}_{t,rs}^* = 0$ we take

$$\hat{P}_{t,rs}^* \equiv \hat{P}_{t-1,rs}^*.\tag{3.83}$$

Next, in analogy to (3.82), we re-normalize the positive transition probabilities of (3.78) currently derived from the solution of (3.69) with those $\hat{P}_{t-1,rs}^*$ given in (3.83). It is readily seen that such a combination of (3.78) with (3.83) leads [in analogy to (3.80)–(3.82)] again to (3.78), provided that all zero probabilities $\hat{P}_{t,rs}^* = 0$ in (3.78) are replaced according to (3.83) and then renormalized with the positive ones, currently derived from (3.69). In

short, in the modified version of (3.78) at each stage t we adjust *only* those transition probabilities for which the generated trajectories pass through, while the other transition probabilities remain untouched (see (3.83)).

Consider now the case where $N > n$, that is $N = Cn$, $C > 1$. Assume, as before, that the initial elements $P_{0,ij}$ of the MC P_0 are uniformly distributed and consider the traveling salesman problem. Denote $N_{*,n}$ the minimal number of trajectories required in order to pass through all transition probabilities $P_{0,ij}$ at least once with high probability P_N . Note that with such a sample N_N at hand we must obtain that all transition probabilities in (3.78) (at stage $t = 1$) will be positive with high probability. Our extensive simulation studies show that for $50 \leq n \leq 1000$, C_n satisfies $N_{*,n} = C_n n$, where $C_n = K_n \ln n$ and $K_n \approx e$.

Table 3.2 presents the results of such simulation studies with $P_N = 0.999$.

Notice that for the traveling salesman problem $N_{*,n} = f(n)$ can be derived by using the theory of *urn models*, and in particular using the *inclusion-exclusion principle for occupancy problems* (Johnson and Kotz (1977), p. 108) which allows one to find the probability that not one of the urns will be empty (not one of the transition probabilities in (3.78) will be zero), while distributing N balls into n cells (generating N trajectories through the network containing n nodes).

With the sample $N_{*,n} = C_n n$ at our disposal we can, in fact, avoid using (3.83) and try to converge to ODMC by using either a single stage or multiple stages. Our Algorithm 3.2 below uses multiple stages, since as in Example 3.4 a single stage typically assumes a very small γ , which in turn might lead to violation of (3.71), (3.72) and thus in unreliable (having large relative error) estimates $\hat{P}_{t,rs}^*$ of $P_{t,rs}^*$. In addition, a multiple stage procedure based on (3.78), (3.83) leads to a dramatic speedup compared to a single stage procedure. An intuitive explanation of such a speedup is based on a high positive correlation between the neighboring trajectories, which will be discussed below.

It is important to note, and easy to observe, that part 1 of Lemma 3.1 holds for the modified version (3.78), (3.83) as well.

ALGORITHM 3.2

1. Choose an initial reference MC $P_0 \equiv \bar{P}_0^*$, say with elements uniformly distributed at each row (see e.g., (3.4)). Generate N trajectories $\{(i_1, i_2, \dots, i_\tau), j\}$; $j = 1, 2, \dots, N$ from P_0 and solve Equation (3.71). Denote the initial solution by $\bar{\gamma}_1^*$.
2. Use the same N trajectories $\{(i_1, i_2, \dots, i_\tau), j\}$, $j = 1, 2, \dots, N$ associated with P_0 and solve analytically the stochastic program for $\gamma = \bar{\gamma}_1^*$ and P_0 (3.69) by applying, in fact, Equation (3.78). Denote the solution by $\bar{P}_1^*$.

Table 3.2. C_n as a function of n for the traveling salesman problem with $P_N = 0.999$.

n	C_n	$N_{*,n}$	$\ln n$
50	8	400	3.9
100	9	900	4.6
250	10	2,500	5.5
500	12	6,000	6.2
750	13	9,750	6.6
1000	14	14,000	6.9

3. Generate N new trajectories $\{(i_1, i_2, \dots, i_\tau), j\}$, $j = 1, 2, \dots, N$ from $\bar{P}_1^*$ and repeat steps 1 and steps 2 again with $\bar{P}_0^*$ replaced by $\bar{P}_1^*$. Denote the solution by $\bar{\gamma}_2^*$ and $\bar{P}_2^*$, respectively. Denote the corresponding solutions at stage t by $\bar{\gamma}_t^*$ and $\bar{P}_t^*$, respectively.
4. If for any $t \geq 1$ and some k , say $k = 5$

$$\widehat{\mathcal{L}}_{t,(1)} = \widehat{\mathcal{L}}_{t+1,(1)} = \dots = \widehat{\mathcal{L}}_{t+k,(1)} \quad (3.84)$$

repeat Step 3 and stop; otherwise repeat steps 2-4 again. Here

$$\widehat{\mathcal{L}}_{t,(1)} = \min_{j \in \{\mathcal{X}_t\}} \sum_{i=1}^{\tau_j} L_{i,i_{t+1}} \quad (3.85)$$

is the estimate of the shortest path corresponding to the minimal order statistics, and $\{\mathcal{X}_t\}$ is the set of all trajectories generated so far. After stopping:

- Estimate C according to (3.90) below with P and γ equal to $\bar{P}_t^*$ and $\bar{\gamma}_t^*$ respectively.
- Take $\widehat{\mathcal{L}}_{t,(1)}$ as the estimate of the shortest path.

Note that according to (3.84) Algorithm 3.2 stops if its k consecutive iterations result in the same value $\widehat{\mathcal{L}}_{t+s,(1)}$, $s = 1, \dots, k$. An alternative stopping rule is: stop if for any $t \geq 1$ and same k , say $k = 5$

$$\max_{j \leq k} \{|\bar{\gamma}_t^* - \bar{\gamma}_{t+j}^*|\} \leq \varepsilon_1, \text{ and } \max_{j \leq k} \{|\widehat{\mathcal{L}}_{t,(1)} - \widehat{\mathcal{L}}_{t+j,(1)}|\} \leq \varepsilon_2, \quad (3.86)$$

ε_1 and ε_2 being very small numbers.

A rigorous proof of speed of convergence of Algorithm 3.2 will be given elsewhere.

Note that Algorithm 3.2 can be readily modified for the TSP as well.

REMARK 3.2 One can use in Algorithm 3.2 instead of the Boltzmann function

$$H(\gamma) = \prod_{i=1}^{\tau} \exp\left(-\frac{L_{i,i_{t+1}}}{\gamma}\right)$$

(see (3.65)) different ones and still obtain convergence. For example, one can use the following power function

$$H_1(\gamma) = \left(\sum_{i=1}^{\tau} L_{i,i_{t+1}}\right)^{\gamma},$$

or the rare events indicator function

$$H_2(\gamma) = I_{\left\{\sum_{i=1}^{\tau} L_{i,i_{t+1}} \leq \gamma_t\right\}},$$

(see (3.47) and Algorithm 3.1). Our numerical results show that Algorithm 3.2 with the Boltzmann function performs better than with $H_1(\gamma)$ and $H_2(\gamma)$.

REMARK 3.3 *To speed up the convergence rate we can generalize (3.83) by considering its following discounted version*

$$\hat{P}_{t,rs}^* \equiv (1 - \alpha_t) \hat{P}_{t-1,rs}^*, \quad (3.87)$$

where $0 \leq \alpha_t \leq 1$, $\alpha_0 = 0$ and α_t increases in t , (for more details see Section 3). In analogy to (3.83), we need to re-normalize the discounted elements in (3.87) with the positive ones (3.78) currently derived from the solution of (3.69). Note that for $\alpha = 1$ we have $\hat{P}_{t,rs}^* = 0$ and therefore (3.78) and (3.87) reduce just to (3.78).

REMARK 3.4 *Program (3.69) as stated requires, at each state of the transition matrix $\hat{P}_t^*$, generation from a discrete n -point pdf. A straightforward routine is the inverse-transform (Rubinstein and Melamed (1998)) technique. It is typically time-consuming, in particular if n is large. To speed up Algorithm 3.2 one can use the alias method (Walker (1977)) instead. Generation of random variables from an arbitrary discrete distribution by the alias method is typically much faster than by the inverse-transform method, especially for large n .*

If we need to estimate the partition function

$$C = \sum_{j \in \{\mathcal{X}\}} \prod_{t=1}^{\tau} H_{i_t i_{t+1}}(\gamma) \quad (3.88)$$

for very small γ simultaneously with the shortest (longest) path, where, as before, $\{\mathcal{X}\}$ denotes the set of all feasible trajectories, we can use likelihood ratios, that is present C as

$$C = \mathbb{E}_P \left\{ \sum_{j \in \{\mathcal{X}\}} \prod_{t=1}^{\tau_j} H_{i_t i_{t+1}, j}(\gamma) W_{i_t i_{t+1}, j}(P) \right\}, \quad (3.89)$$

where

$$W_{i_s i_{s+1}}(P) = \frac{1}{P_{i_s i_{s+1}}}$$

is the LR. The LR estimate of C is

$$\bar{C}_N(\gamma, P) = \sum_{j=1}^N \prod_{t=1}^{\tau_j} H_{i_t i_{t+1}, j}(\gamma) W_{i_t i_{t+1}, j}(P). \quad (3.90)$$

As Algorithm 3.2 evolves we obtain the following sequence

$$\bar{C}_N(\bar{\gamma}_1^*, \hat{P}_1^*), \bar{C}_N(\bar{\gamma}_2^*, \hat{P}_2^*), \dots, \bar{C}_N(\bar{\gamma}_t^*, \hat{P}_t^*), \dots \quad (3.91)$$

converging to the partition function C in (3.88).

4. Why CE Algorithms are Fast

We explain now why Algorithm 3.2 works fast. In fact, we show that the reason is a very high positive correlation between the neighboring paths. To be more precise, consider the traveling salesman problem with n cities and assume without loss of generality that the initial transition matrix P_0 is uniformly distributed over the states, that is

$$P_{0,rk} = \begin{cases} \frac{1}{n-1} & \text{if } r \neq k, r = 1, \dots, n \\ 0 & \text{otherwise.} \end{cases} \quad (4.1)$$

As we mentioned earlier the suggested sample size (number of trajectories generated) at each iteration of Algorithm 3.2 is of order of n^2 . We, however, consider here for simplicity only two trajectories, denoted as

$$i = \{i_0, i_1, \dots, i_{n-2}, i_{n-1}, i_0\}$$

and

$$j = \{j_0, j_1, \dots, j_{n-2}, j_{n-1}, j_0\}$$

respectively. Assume finally that the first trajectory is the shortest of the two, that is

$$\mathcal{L}_i = \sum_{s=1}^{n-1} L_{i_{s-1}i_s} + L_{i_{n-1}i_0} < \mathcal{L}_j = \sum_{s=1}^{n-1} L_{j_{s-1}j_s} + L_{j_{n-1}j_0}$$

and consider several iterations of Algorithm 3.2. Applying our main iterative procedure (3.69) (for a fixed γ) and solving it according to (3.78), (3.83), we obtain that some of the transition probabilities $\hat{P}_{1,i_{s-1}i_s}^*$ associated with trajectory i will be larger than the transition probabilities $\hat{P}_{1,j_{s-1}j_s}^*$ associated with trajectory j , that is we obtain after the first iteration that

$$\hat{P}_{1,i_{s-1}i_s}^* \geq \hat{P}_{1,j_{s-1}j_s}^*.$$

Note that (3.78), (3.83) ensures automatic normalization of $\hat{P}_{1,i_{s-1}i_s}^*$ and $\hat{P}_{1,j_{s-1}j_s}^*$ as

$$\sum_{s=1}^{n-1} \hat{P}_{1,i_{s-1}i_s}^* + \hat{P}_{1,i_{n-1}i_0}^* + \sum_{s=1}^{n-1} \hat{P}_{1,j_{s-1}j_s}^* + \hat{P}_{1,j_{n-1}j_0}^* = \frac{2}{n}. \quad (4.2)$$

Consider now the subsequent iterations ($t > 1$) of Algorithm 3.2. This is exactly where the high positive correlation between the neighboring trajectories comes into play. To proceed, notice first that there are *only* n^2 transition probabilities and a *huge* number, $(n-1)!$, of trajectories passing through different states of the network, each of length n . Clearly, since the distances L_{rk} are nonnegative, the correlation among the neighboring trajectories must be positive. Returning to our two-trajectory case, we have that increasing, at stage t , the transition probabilities for the i -th trajectory, we automatically *increase*, at the next stage $t+1$, those of a huge subset of neighboring trajectories associated with i . Similarly, reducing, at stage t , the transition probabilities for the j -th trajectory, we automatically also *reduce* those of a huge subset of neighboring trajectories associated with j . A formal definition of neighborhood structures and the so-called k -change is given in Aarts and Korst (1989), Section 1.2. In particular, the k -change, defines for each

trajectory i a neighborhood $\mathcal{X}_i$ consisting of the set of solutions obtainable from that trajectory by removing k edges from the tour corresponding to it and replacing them with k other edges such that again a tour is obtained.

Assume for a moment that we generate randomly a set of traveling salesman problems with the elements L_{rk} of the distance matrix (L_{rk}) being i.i.d. (independent identically distributed) random variables. It is readily seen that in this case the correlation ρ_k , ($k \geq 2$) between the k -change neighboring trajectories is

$$\rho_k \approx \frac{n-k}{n}.$$

For k small relative to the size of the network n , we have that $\rho_k \approx 1$.

In conclusion we obtain that starting from $t = 2$, Algorithm 3.2 generates adaptively a set of highly positively correlated trajectories converging quickly to trajectory associated with the shortest distance. This is the same as to say that Algorithm 3.2 generates a Markov process with a positive drift converging quickly to the ODMC corresponding to the shortest distance.

5. Numerical Results

5.1. Continuous Multiextremal Optimization

Consider the following multiextremal function

$$L(\mathbf{x}) = \sum_{i=1}^n 8 \sin^2(\eta(x_i - a_i)^2) + 6 \sin^2(2\eta(x_i - a_i)^2) + \mu(x_i - a_i)^2 + 1 \quad (5.1)$$

with $L(\mathbf{x}^*) = 1$ and $x_i^* = a_i$.

Table 5.1 presents the performance of Algorithm 2.1 for the multiextremal function

Table 5.1. Performance of the SPF Algorithm 2.1 for the multiextremal function (5.1) with $n = 10$, $\eta = 7$ and $\mu = 0.1$.

t	$\bar{\gamma}^*$	$L(\mathbf{x})$	$\bar{C}_N(\bar{\gamma}^*, \bar{\mathbf{v}}^*)$
1	1.34e+01	5.22e+01	8.83e-03
2	1.33e+01	4.30e+01	8.80e-03
3	1.29e+01	4.47e+01	6.82e-03
4	1.32e+01	6.58e+01	1.05e-02
5	1.35e+01	5.18e+01	9.41e-03
10	1.29e+01	6.55e+01	3.64e-03
15	1.19e+01	2.58e+01	7.49e-04
20	7.68e+00	9.75e+00	2.35e-06
25	2.77e+00	6.55e+00	3.64e-10
30	1.67e+00	1.11e+00	7.49e-11
35	5.32e-01	1.03e+00	2.13e-14
40	3.53e-01	1.00e+00	8.55e-17
45	2.68e-01	1.00e+00	5.63e-18

(5.1) with $n = 10$, $\eta = 7$, $\mu = 0.1$, $a_i = a = 0.9$ and the non adaptive ρ . In particular, it presents the evolution in time (for iterations 1,2,3,4,5,10,15,20,25,30,35,40,45) of the output performance functions $\bar{\gamma}^*$, $L(\mathbf{x})$ and $\bar{C}_N(\bar{\gamma}^*, \bar{\mathbf{v}}^*)$, respectively. The sample size in step 2 and step 3 of Algorithm 2.1 was chosen as $N = 10n = 100$ and the parameter ρ in (2.18) was set as $\rho = 10^{-2}$. The algorithm stopped after 45 iterations. Note that the number of local minima $\mathcal{N}$ for this case (see Pinter (1996)) is $\mathcal{N} = 1048576$.

We found that the SPF $\bar{\mathcal{C}}_N(\gamma_t, \mathbf{v}_t)$ $t = 1, \dots, 45$ varied in the interval $(0.95e - 03, 1.02e - 02)$; - all 10 final estimates of $x_i^* = a$, $i = 1, \dots, 10$ settled down in the interval $(0.89, 0.91)$, while all 10 final values α_i , $i = 1, \dots, 10$ and β_i , $i = 1, \dots, 10$ of $\mathbf{a}$ and $\mathbf{\beta}$ settled down in the intervals $(7.2e + 03, 7.5e + 03)$ and $(8.1e + 02, 8.4e + 02)$, respectively. The total number of function L evaluations was 4550 and the CPU time on a Pentium computer was 67 sec.

It is readily seen that for some t , namely for $t = 3, 4, 5$, both $\bar{\gamma}^*$ and $L(\mathbf{x})$ increase. This is in agreement with the theory, and is due to the fact that $L(\mathbf{x})$ is multiextremal. Similar results were obtained for the adaptive ρ (see (2.30)), different parameter settings of η , μ , n , a , provided $\eta \leq 10$, $\mu \geq 0.01$, $n \leq 1000$, and $0 \leq a \leq 1$ as well as some other functions. In particular, we applied Algorithm 2.1 to the following multiextremal test functions:

Rosenbrock's function

$$L(\mathbf{x}) = \sum_{i=1}^{n-1} 100(x_{i+1} - x_i^2)^2 + (x_i - 1), \quad -5.12 \leq x_i \leq 5.12, \quad (5.2)$$

$$L(\mathbf{x}^*) = L(1, \dots, 1) = 0 \text{ and}$$

Rastrigin's function

$$L(\mathbf{x}) = 10n + \sum_{i=1}^n (x_i^2 - A \cos(2\pi x_i) + 1), \quad -5.12 \leq x_i \leq 5.12, \quad (5.3)$$

$$L(\mathbf{x}^*) = L(1, \dots, 1) = 1.$$

In some experiments we found that Algorithm 2.1 estimates quite accurately the optimal function value $L(\mathbf{x}^*)$ (within 1%), but fails to find the optimal parameter value $\mathbf{x}^*$. We found, however, that such a situation typically occurs when there are either several global minima or the minima are almost indistinguishable. Below we present such data (from one of our simulation experiments) for the following parameters' setting: $n = 10$, $\eta = 7$, $\mu = 0.01$, ($\mathcal{N} = 1048576$) and $a_i = a = 0.99$. The estimate of the true $L(\mathbf{x}^*) = 1$ settled down (after 50 iterations) at point 1.04, while the estimates, x_i , $i = 1, \dots, 10$, of the optimal parameter values x_i^* did not settle down correctly, namely we obtained

$$\begin{aligned} x_1 &= 9.473771e - 01, & x_2 &= 2.016075e - 01, & x_3 &= 2.977174e - 01, \\ x_4 &= 8.511324e - 01, & x_5 &= 9.516205e - 01, & x_6 &= 1.502054e - 01, \\ x_7 &= 6.997233e - 01, & x_8 &= 8.009481e - 01, & x_9 &= 8.998981e - 01, \\ x_{10} &= 9.538900e - 01. \end{aligned}$$

instead of $x_i^* = a = 0.99$.

Since the function L in (5.1) is additive we also used the stochastic counterpart of (2.13) instead of the program (3.69). We found that estimate $\bar{\mathbf{v}}^*$ of $\mathbf{x}^*$ based on (2.13) outperforms its non additive counterpart based on (3.69). In particular, for the data of Table 5.1, we found that Algorithm 2.1 with the additive version stopped after 6 iterations resulting in a similar performance to the non additive version (3.69), which stopped after 45 iterations. In addition, we found that the additive version permits optimization of high-dimensional problems. In particular we found that for the function (5.1) the additive version performs quite reliably for n of the order of thousands, while the non additive version performs quite reliably for n of the order of hundreds only. In addition, using the adaptive parameter ρ_t we were able to accelerate the of convergence by a factor of 5–10.

Let

$$M_n = N_n \cdot K_n,$$

which is taken as a measure of complexity of Algorithm 2.1, be the total number of function evaluations needed by Algorithm 2.1 before it stops. Here n is the dimensionality, N_n is the sample size and K_n is the number of iterations. Choosing $10n \leq N_n \leq 50n$ we found numerically that Algorithm 2.1 stops after $20 \leq K_n(N) \leq 60$ iterations regardless of n . For example, let $n = 500$. If we take $N_n = 10n$ and if Algorithm 2.1 stops after $K_n = 30$ iterations, then the total number of function evaluations is $M_n = 10n \cdot 30 = 150,000$. We also found that Algorithm 2.1 is robust with respect to the perturbations on the initial value of the parameter vector $\mathbf{v}_0$. More specifically, we found that the vectors $\mathbf{v}_{0i}$, $i = 1, \dots, n$ in the pdf's $h_i(x, \mathbf{v}_{0i})$ can be perturbed by 10%. This, for example, means that if $\text{Beta}(\boldsymbol{\alpha}, \boldsymbol{\beta})$ is the dominating pdf, then one can start Algorithm 2.1, say with $h_i(x, \mathbf{v}_i) = \text{Beta}(\alpha_i, \beta_i)$, where $1 \leq \alpha_i \leq 2$, $1 \leq \beta_i \leq 2$ instead of the uniform distribution, which by itself is a $\text{Beta}(\alpha_i = 1, \beta_i = 1)$.

5.2. Combinatorial Optimization

Below we present the performance of Algorithm 3.2 for the shortest and longest path (in a directed network) and for the traveling salesman problem by choosing the initial reference MC P_0 according to the uniform distribution (see e.g., (4.1)) and using both, the nonadaptive and adaptive (see (5.5), (5.6)) choices of ρ . By the performance of the shortest path, we mean the estimators $\bar{\mathcal{L}}_{t,(1)}$ and $\widehat{\mathcal{L}}_{t,(1)}$ of the true shortest path $\mathcal{L}_{(1)}$. Here $\bar{\mathcal{L}}_{t,(1)}$ is defined in analogy to $\widehat{\mathcal{L}}_{t,(1)}$ as

$$\bar{\mathcal{L}}_{t,(1)} = \min_{j \in \{\widehat{\mathcal{Y}}_t\}} \sum_{i=1}^{\tau_j} L_{i, i_{t+1}}. \quad (5.4)$$

Note that $\bar{\mathcal{L}}_{t,(1)}$ presents the estimate of the shortest path found at t -th iteration, and $\{\widehat{\mathcal{Y}}_t\}$ is the set of the trajectories generated at that iteration, while

$$\widehat{\mathcal{L}}_{t,(1)} = \min_{j=1, \dots, t} \bar{\mathcal{L}}_{j,(1)}$$

represents the estimate of the shortest path found in *all* t iterations. Note also that for the longest path we need to replace the operator $\min$ in (5.4) with $\max$ and write $\bar{\mathcal{L}}_{j,(n)}$ instead of $\mathcal{L}_{j,(1)}$, while for the traveling salesman we need to replace τ_j in (5.4) with n .

To generate different case studies we chose the elements L_{rs} of the distance matrix (L_{rs}) as iid random variables distributed according to exponential family, (see Rubinstein and Melamed (1998)). Taking into account the results of Table 3.2 for large samples, we assume, without loss of generality, that $\alpha = 1$. We stopped Algorithm 3.2 according to stopping rule (3.86) with the parameter $k = 6$. In all our examples we also present the running time of Algorithm 3.2 on a SUN station and based on our numerical results discuss its complexity properties.

To speed up the convergence of Algorithm 3.2 we use in (3.71), instead of a fixed ρ , the following adaptive procedure for ρ

$$\rho_{s+1} = I_A C_s \rho_s + I_{\bar{A}} \rho_s, \quad s = 0, 1, \dots, \quad (5.5)$$

where $\rho_0 = \rho$ (see (3.71)), $10^{-1} \leq \rho \leq 10^{-5}$, $10^{-4} \leq C_s \leq 10^{-1}$, and I_A is the indicator of the event (3.84), where k is replaced by r , $r < k$, that is

$$I_A = \begin{cases} 1 & \text{if } \widehat{\mathcal{L}}_{t,(1)} = \widehat{\mathcal{L}}_{t+1,(1)} = \dots = \widehat{\mathcal{L}}_{t+r,(1)}, \quad t > 1 \\ 0 & \text{otherwise.} \end{cases} \quad (5.6)$$

In all our numerical combinatorial examples below we take $\rho_0 = 10^{-3}$, $r = 3$, $k = 6$ and C_s as in (2.29). It follows from (5.5)–(5.6) that Algorithm 3.2 starts with $\rho_0 = 10^{-3}$, and then, each time the performance $\widehat{\mathcal{L}}_{t,(1)}$ does not improve in turn 3 times ($r = 3$), it chooses a smaller ρ . The resulting adaptive sequence of ρ_s is therefore

$$\rho_s = 10^{-3}, 10^{-6}, 10^{-9}, \dots$$

Again, if, for example, Algorithm 3.2 stops when $s = 5$, that is when the event

$$A = \{\widehat{\mathcal{L}}_{t,(1)} = \widehat{\mathcal{L}}_{t+1,(1)} = \dots = \widehat{\mathcal{L}}_{t+r,(1)}\},$$

occurs five times in turn (see the stopping rule (3.84)), the final ρ_s (just before stopping) will be $\rho_5 = 10^{-18}$.

5.2.1. Shortest Path

We consider a directed network with the following distance matrix

$$(L_{ij}) = \begin{pmatrix} 0 & L_{12} & L_{13} & \dots & L_{1(n-k)} & 0 & \dots & 0 & 0 \\ 0 & 0 & L_{23} & L_{34} & \dots & L_{2(n-k+1)} & 0 & \dots & 0 \\ 0 & \dots & 0 & L_{r(r+1)} & \dots & L_{r(n-k+r-1)} & 0 & \dots & 0 \\ 0 & \dots & 0 & 0 & \dots & 0 & L_{s(s+1)} & \dots & L_{s(n-k+s-1)} \\ 0 & \dots & 0 & 0 & \dots & 0 & 0 & \dots & L_{nn} \end{pmatrix}, \quad (5.7)$$

where $r < k < s < n$. We call (5.7) the distance matrix with configuration $n \times m$, where $m = n - k$. If not stated otherwise we take $m = 0.1n$ and use the *table look* (TL) method Devroye (1986) for generation random variables from multinomial distributions defined for each row i and each iteration t of the MC ($P_{t,ij}$) and take a sample of $N = 10nm$ trajectories in all our experiments below. To check the accuracy of Algorithm 3.2 we run Dijkstra's algorithm (Ahuja *et al.* (1993)) to compare the results. Note that our numerical results for the shortest path are given for illustration purposes only, since Dijkstra's algorithm is exact, having polynomial in the size of problem complexity.

Table 5.2 presents the performance of Algorithm 3.2 for the configuration $n \times m = 10,000 \times 100$ with non adaptive $\rho = 10^{-3}$ and $L_{ij} \sim \mathcal{U}(1, 10)$, where $\mathcal{U}$ denotes the uniform distribution. In particular, they present the evolutions in time (iterations) of $\bar{\gamma}_t^*$, $\widehat{\mathcal{P}}_{t,(1)}$ and $\bar{\mathcal{P}}_{t,(1)}$ and $N_{t,traj}$, where $N_{t,traj}$ denote the number of distinct (different) trajectories passing through the network for the iteration t . We say that two trajectories in a directed network are distinct if at least one transition is not common for both. It is readily seen that for $P_{0,ij}$ being uniformly distributed and N not too large ($N = O(mn)$), we expect (see 3.2) that $N_{0,traj} \approx N$, and we expect $N_{t,traj} \rightarrow 1$ as $t \rightarrow \infty$, that is as $P_{t,ij}$ approaches to ODMC.

Similar results were obtained for $\rho = 10^{-4}$ and $\rho = 10^{-5}$.

Table 5.3 presents data similar to Table 5.2 for the adaptive ρ with the initial value $\rho_0 = 10^{-3}$.

The CPU times for the above cases are 259.4 and 241.2 sec., respectively, that is the adaptive version of Algorithm 3.2 is a little faster than the non adaptive one. The relative accuracy in all four cases (compared to Dijkstra's exact algorithm) $\varepsilon = 0$, that is the SE method is exact.

5.2.2. Longest Path

As for the shortest path, we run Algorithm 3.2 for the same distance matrix (5.7), using both, the non adaptive and adaptive ρ (see (5.5)–(5.6)), with the same parameter set given in (2.30), using the same stopping rule (3.85) with $k = 6$, and taking a sample of $N = 10nm$ trajectories.

As mentioned earlier, the only difference between Algorithm 3.2 for the shortest and the longest path is that in the latter the sign $-$ in $\rho = 10^{-k}$ in (3.71) is changed to the sign $+$.

Since no exact method is known for the longest path, we purposely select a *single* path, which is known in advance to be the longest one. For example, taking $L_{i,i+1} = 10$ in (5.7) for all, $i = 1, 2, \dots, n - 1$ and the rest of $L_{ij} \sim U(1, 10)$, $j \geq i + 1$, we define a trajectory associated with $L_{i,i+1}$ such that

$$\sum_{i=1}^n L_{i,i+1} = \mathcal{L}_{(|\mathcal{X}|)} = 10(n - 1),$$

Table 5.2. Performance of Algorithm 3.2 for the shortest path problem with the configuration $n \times m = 1000 \times 100$ and non adaptive $\rho = 10^{-3}$.

t	$\bar{\mathcal{L}}_{t,(1)}$	$\widehat{\mathcal{L}}_{t,(1)}$	γ_t	ρ_t	$N_{t,raj}$
1	54.40621	54.40621	16.48073	1.00e-03	100000
2	42.44575	42.44575	16.46162	1.00e-03	100000
3	32.71201	32.71201	13.43338	1.00e-03	100000
4	25.04891	25.04891	11.23080	1.00e-03	100000
5	23.13656	23.13656	9.271676	1.00e-03	100000
6	20.60568	20.60568	7.630352	1.00e-03	100000
7	17.95172	17.95172	6.277011	1.00e-03	100000
8	16.17352	16.17352	5.215937	1.00e-03	100000
9	14.73566	14.73566	4.411229	1.00e-03	99999
10	13.91147	13.91147	3.807220	1.00e-03	99932
11	13.91147	13.91147	3.347646	1.00e-03	98798
12	13.91147	13.91147	2.988115	1.00e-03	92198
13	13.91147	13.91147	2.700195	1.00e-03	75062
14	13.91147	13.91147	2.476869	1.00e-03	51531
15	13.91147	13.91147	2.319054	1.00e-03	31059
16	13.82759	13.82759	2.218373	1.00e-03	17501
17	13.91147	13.82759	2.156497	1.00e-03	10743
18	13.91147	13.82759	2.124286	1.00e-03	6554
19	13.82759	13.82759	2.098657	1.00e-03	4320
20	13.91147	13.82759	2.081506	1.00e-03	3005
21	13.82759	13.82759	2.072956	1.00e-03	2179
22	13.82759	13.82759	2.063727	1.00e-03	1711
23	13.82759	13.82759	2.058176	1.00e-03	1415
24	13.82759	13.82759	2.053394	1.00e-03	1127
25	13.91147	13.82759	2.049411	1.00e-03	987
26	13.82759	13.82759	2.046616	1.00e-03	813
27	13.91147	13.82759	2.043345	1.00e-03	757
28	13.82759	13.82759	2.042013	1.00e-03	694
29	13.82759	13.82759	2.039557	1.00e-03	623
30	13.82759	13.82759	2.038943	1.00e-03	538

where $\mathcal{L}_{(|\mathcal{X}|)}$ denotes the longest path. This trajectory passes the nodes (in the distance matrix (5.7)) in the order $1 \rightarrow 2 \rightarrow \dots \rightarrow n$ and corresponds to the one which is the right-most to the diagonal elements in (5.7). If not otherwise stated we will use the path with the elements $L_{i,i+1}$ as longest one.

Table 5.4 presents the performance of Algorithm 3.2 for the configuration $n \times m = 500 \times 50$ with the adaptive ρ , $N = 500 \times 50 = 25,000$ trajectories and $L_{ij} \sim \text{Beta}(\alpha = 1, \beta = 10)$. Here

$$\text{Beta}(\alpha, \beta) = \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} \left(\frac{1}{b-a} \right)^{\alpha+\beta-1} (b-x)^{\beta-1} (x-a)^{\alpha-1}, \quad a \leq x \leq b, \quad \alpha > 0, \quad \beta > 0. \quad (5.8)$$

Table 5.3. Performance of Algorithm 3.2 for the shortest path problem with the configuration $n \times m = 1000 \times 100$ and adaptive ρ .

t	$\bar{\mathcal{L}}_{t,(1)}$	$\widehat{\mathcal{L}}_{t,(1)}$	γ_t	ρ_t	$N_{t,raj}$
1	53.93194	53.93194	16.45824	$1.00e-03$	100000
2	42.77205	42.77205	16.46679	$1.00e-03$	100000
3	28.04077	28.04077	13.44240	$1.00e-03$	100000
4	25.29249	25.29249	11.24455	$1.00e-03$	100000
5	21.63693	21.63693	9.273934	$1.00e-03$	100000
6	19.24309	19.24309	7.617015	$1.00e-03$	100000
7	17.15285	17.15285	6.258752	$1.00e-03$	100000
8	15.06331	15.06331	5.171850	$1.00e-03$	100000
9	14.35793	14.35793	4.339736	$1.00e-03$	99996
10	13.82759	13.82759	3.704786	$1.00e-03$	99672
11	13.82759	13.82759	3.224149	$1.00e-03$	96463
12	13.82759	13.82759	2.870155	$1.00e-03$	85109
13	13.82759	13.82759	1.931060	$1.00e-04$	65782
14	13.82759	13.82759	1.808255	$1.00e-04$	38875

Note that $\text{Beta}(\alpha, \beta)$ reduces to $\mathcal{U}(a, b)$ for $\alpha = 1$ and $\beta = 1$. Table 5.5 presents data similar to Table 5.4 for $L_{ij} \sim \text{Beta}(x, \alpha = 1, \beta = 0.1)$, $1 \leq x \leq 10$.

The CPU times of Algorithm 3.2 for the Tables 5.4 and 5.5 are 981.9 and 2148.2 sec., respectively. It follows also from the results of Tables 5.4 and 5.5 that the relative accuracy (compared with the exact result $\mathcal{L}_{(|\mathcal{X}|)} = 10(n-1)$) is $\varepsilon_1 = 0$ and $\varepsilon_2 = 0.002$, respectively. That is, the SE method is quite accurate.

We also run Algorithm 3.2 for the longest path with different exponential family distributions of L_{ij} . In approximately 95% of the cases we found that Algorithm 3.2 is accurate in the sense of (3.61).

5.2.3. Traveling Salesman

Since no exact method is known here to check the accuracy of our method except for the naive, total enumeration routine, which is suitable for small networks, say with $n = 10$ cities containing $9! = 9 \times 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 362,880$ trajectories. For large networks (in analogy to the longest path) one can select in advance a tour, which is known to be shortest one and to check again the accuracy of our method. For example, one can define the shortest tour by taking the elements $L_{i,i+1} = 1$, for all $i = 1, 2, \dots, n-1$ and $L_{n,1} = 1$, while the rest of the elements $L_{ij} \sim U(1, 10)$, $j \neq i+1$ and $L_{ii} \equiv 0$. In this case the shortest tour $\mathcal{K}_{(1)}$ is the one where the cities (nodes) are visited on the order $1 \rightarrow 2 \rightarrow 3 \rightarrow \dots \rightarrow n \rightarrow 1$. We have

Table 5.4. Performance of Algorithm 3.2 for the longest path with the configuration $n \times m = 500 \times 50$, adaptive ρ and $L_{ij} \sim \text{Beta}(x, \alpha = 1, \beta = 10)$, $1 \leq x \leq 10$.

t	$\widehat{\mathcal{L}}_{t,(1)}$	$\widehat{\mathcal{L}}_{t,(1)}$	γ_t	ρ_t	$N_{t,\text{traj}}$
1	127.9258	127.9258	16.65271	1.00e + 03	25000
2	165.9885	165.9885	16.83429	1.00e + 03	25000
3	209.7686	209.7686	21.84240	1.00e + 03	25000
4	260.6800	260.6800	27.60248	1.00e + 03	25000
5	331.6918	331.6918	34.30182	1.00e + 03	25000
6	406.4026	406.4026	43.64693	1.00e + 03	25000
7	470.5328	470.5328	53.47728	1.00e + 03	25000
8	556.8555	556.8555	61.91900	1.00e + 03	25000
9	667.1027	667.1027	73.28139	1.00e + 03	25000
10	742.7772	742.7772	87.78878	1.00e + 03	25000
11	879.7375	879.7375	97.74821	1.00e + 03	25000
12	947.2165	947.2165	115.7683	1.00e + 03	25000
13	1021.532	1021.532	124.6591	1.00e + 03	25000
14	1120.587	1120.587	134.4670	1.00e + 03	25000
15	1272.707	1272.707	147.4838	1.00e + 03	25000
16	1375.542	1375.542	167.5288	1.00e + 03	25000
17	1541.947	1541.947	181.0633	1.00e + 03	25000
18	1581.225	1581.225	202.9619	1.00e + 03	25000
19	1729.261	1729.261	208.1244	1.00e + 03	25000
20	1826.805	1826.805	227.6749	1.00e + 03	25000
...	...	...	...	...	...
60	4869.266	4869.266	170.8865	1.00e + 12	25000
61	4898.025	4898.025	171.9136	1.00e + 12	25000
62	4943.877	4943.877	172.9291	1.00e + 12	25000
63	4957.960	4957.960	174.5506	1.00e + 12	25000
64	4961.482	4961.482	175.0459	1.00e + 12	24998
65	4990.000	4990.000	175.1704	1.00e + 12	25000
66	4975.770	4990.000	176.1800	1.00e + 12	24990
67	4990.000	4990.000	175.6782	1.00e + 12	24970
68	4990.000	4990.000	141.6340	1.00e + 15	24886
69	4990.000	4990.000	141.6339	1.00e + 15	24673
70	4990.000	4990.000	118.4158	1.00e + 18	24244
71	4990.000	4990.000	118.4158	1.00e + 18	23337

$$\mathcal{K}_{(1)} = \sum_{i=1}^{n-1} L_{i,i+1} + L_{n,1} = n.$$

If not otherwise stated we use the tour $1 \rightarrow 2 \rightarrow 3 \rightarrow \dots \rightarrow n \rightarrow 1$ as the shortest one.

Tables 5.6 and 5.7 present the performance of Algorithm 3.2 for $n = 100$, $L_{ij} \sim \text{Beta}(x, \alpha = 1, \beta = 10)$, $1 \leq x \leq 10$ and $\widehat{L}_{ij} \sim \text{Beta}(\alpha = 1, \beta = 0.1)$, respectively. Note that in both cases the true shortest route $\widehat{\mathcal{L}}_{(1)} = 100$.

The CPU times of Algorithm 3.2 for the Tables 5.6 and 5.7 1343.0 and 370.1 sec., respectively. It follows also from the results of Tables 5.6 and 5.7 that the relative accuracy

Table 5.5. Performance of Algorithm 3.2 for the longest path with the configuration $n \times m = 500 \times 50$, adaptive ρ and $L_{ij} \sim \text{Beta}(x, \alpha = 1, \beta = 0.1)$, $1 \leq x \leq 10$.

t	$\widehat{\mathcal{L}}_{t,(1)}$	$\widehat{\mathcal{P}}_{t,(1)}$	γ_t	ρ_t	$N_{t, \text{traj}}$
1	355.7819	355.7819	46.08031	1.00e+03	25000
2	378.0045	378.0045	46.83992	1.00e+03	25000
3	411.2683	411.2683	49.77094	1.00e+03	25000
4	434.2211	434.2211	54.14874	1.00e+03	25000
5	484.6731	484.6731	57.17003	1.00e+03	25000
...	...	...	...	...	...
55	2804.028	2804.028	189.7255	1.00e+06	25000
56	2869.131	2869.131	193.2930	1.00e+06	25000
57	2942.587	2942.587	197.8003	1.00e+06	25000
58	2977.762	2977.762	202.8532	1.00e+06	25000
59	3070.036	3070.036	205.3083	1.00e+06	25000
60	3113.619	3113.619	211.6440	1.00e+06	25000
...	...	...	...	...	...
95	4220.755	4243.893	194.8499	1.00e+09	25000
96	4232.602	4243.893	197.0890	1.00e+09	25000
97	4328.622	4328.622	149.4345	1.00e+12	25000
98	4321.786	4328.622	152.8255	1.00e+12	25000
99	4361.265	4361.265	152.5840	1.00e+12	25000
100	4376.888	4376.888	153.9794	1.00e+12	25000
...	...	...	...	...	...
128	4928.264	4928.264	116.7422	1.00e+18	25000
129	4909.833	4928.264	116.9516	1.00e+18	25000
130	4919.444	4928.264	116.5137	1.00e+18	25000
131	4929.983	4929.983	100.2998	1.00e+21	25000
132	4949.954	4949.954	100.5147	1.00e+21	25000
...	...	...	...	...	...
145	4970.000	4970.000	71.23334	1.00e+30	22100
146	4980.000	4980.000	71.23334	1.00e+30	20915
147	4980.000	4980.000	71.37667	1.00e+30	19927
148	4980.000	4980.000	71.37668	1.00e+30	18520
149	4980.000	4980.000	64.94654	1.00e+33	17068

(compared with the exact result $\mathcal{K}_{(1)} = n = 100$)) is $\varepsilon_1 = 0.011$ and $\varepsilon_2 = 0.063$, respectively. Consider for example the results of Table 5.7. It follows that algorithm stopped after 65 iterations with an estimate $\widehat{\mathcal{L}}_{K,(1)}$ equal to 106.34. The total number of trajectories generated is $M = NK = n^2K = 10,000 \times 65 = 65,000$, $\rho_k = 10^{-15}$ and the relative accuracy $\varepsilon = 0.0634$. We found that the CPU time for this particular run was 370.1 sec. We also found that increasing the sample size from $N = 10,000$ to $N = 25,000$ the algorithm stopped after 56 iterations producing a more accurate estimator, namely $\widehat{\mathcal{L}}_{t,(1)} = 102.674$. However, the CPU time increased approximately two times. Similar results were obtained for different distributions of L_{ij} , provided $n^2 \leq N \leq 10n^2$, $10^{-5} < \rho_0 < 10^{-2}$. Such results indicate robustness of Algorithm 3.2.

Table 5.6. Performance of Algorithm 3.2 for the traveling salesman problem with adaptive ρ , $L_{ij} \sim \text{Beta}(\alpha = 1, \beta = 10)$ and $n = 100$.

t	$\widehat{\mathcal{L}}_{t,(1)}$	$\widehat{\mathcal{L}}_{t,(1)}$	γ_t	ρ_t	$N_{t,\text{traj}}$
1	155.9937	155.9937	26.00050	1.00e-03	10000
2	153.0488	153.0488	25.99948	1.00e-03	10000
3	151.6683	151.6683	25.70805	1.00e-03	10000
4	152.3387	151.6683	25.44209	1.00e-03	10000
5	151.2572	151.2572	25.20902	1.00e-03	10000
...	...	...	...	...	...
21	137.3821	135.3757	13.44724	1.00e-05	10000
22	136.5978	135.3757	13.35078	1.00e-05	10000
23	134.8346	134.8346	9.452055	1.00e-07	10000
24	132.6972	132.6972	9.396881	1.00e-07	10000
25	132.7752	132.6972	9.299612	1.00e-07	10000
...	...	...	...	...	...
101	110.1829	109.7088	1.518696	1.00e-33	10000
102	110.1516	109.7088	1.516220	1.00e-33	10000
103	109.6558	109.6558	1.387853	1.00e-36	10000
104	109.3259	109.3259	1.386475	1.00e-36	10000
105	109.5708	109.3259	1.383032	1.00e-36	10000
...	...	...	...	...	...
151	105.6676	105.3835	0.608352	1.00e-78	10000
152	105.6529	105.3835	0.607750	1.00e-78	10000
153	105.9055	105.3835	0.584074	1.00e-81	10000
154	105.3686	105.3686	0.583735	1.00e-81	10000
155	105.4446	105.3686	0.583585	1.00e-81	10000
...	...	...	...	...	...
232	101.0817	101.0817	0.293978	1.00e-150	6437
233	101.0790	101.0790	0.293783	1.00e-150	7392
234	101.0727	101.0727	0.293827	1.00e-150	8717
235	101.0727	101.0727	0.293987	1.00e-150	9272
236	101.0727	101.0727	0.294121	1.00e-150	9158
237	101.0727	101.0727	0.294071	1.00e-150	8924
238	101.0727	101.0727	0.288174	1.00e-153	8560

We also performed various case studies with Algorithm 3.2 for the TSP. We found that in approximately 97% of the cases Algorithm 3.2 is accurate in the sense of (3.61). Table 5.8 presents the performance of Algorithm 3.2 with $N = n^2$ for the TSP for a number of case studies taken from the URL

<http://www.iwr.uni-heidelberg.de/iwr/comopt/soft/TSPLIB95/TSPLIB>.

Here $\widehat{\mathcal{L}}_{0,(1)}$ and $\widehat{\mathcal{L}}_{K,(1)}$ denote the initial and the final (before stopping) estimates of the optimal solution; $\mathcal{K}_{(1)}$ denote best known solution; and as before, K and ϵ denote the total number of iterations needed before stopping and the relative accuracy, respectively.

Table 5.7. Performance of Algorithm 3.2 for the traveling salesman problem with adaptive ρ , $L_{ij} \sim \text{Beta}(\alpha = 1, \beta = 0.1)$ for $n = 100$.

t	$\bar{\mathcal{L}}_{t,(1)}$	$\widehat{\mathcal{L}}_{t,(1)}$	γ_t	ρ_t	$N_{t, \text{traj}}$
1	828.9754	828.9754	131.5706	$1.00\text{e} - 03$	10000
2	813.1902	813.1902	131.5447	$1.00\text{e} - 03$	10000
3	821.4815	813.1902	131.1203	$1.00\text{e} - 03$	10000
4	799.6582	799.6582	130.7177	$1.00\text{e} - 03$	10000
5	372.2888	372.2888	63.94892	$1.00\text{e} - 03$	2500
...	...	...	...	...	...
11	767.6005	767.6005	95.47282	$1.00\text{e} - 04$	10000
12	762.3724	762.3724	95.03414	$1.00\text{e} - 04$	10000
13	745.1677	745.1677	94.64940	$1.00\text{e} - 04$	10000
14	766.3890	745.1677	94.25157	$1.00\text{e} - 04$	10000
15	772.7226	745.1677	93.85765	$1.00\text{e} - 04$	10000
16	749.8504	745.1677	74.73510	$1.00\text{e} - 05$	10000
17	748.6085	745.1677	74.43015	$1.00\text{e} - 05$	10000
18	743.8209	743.8209	52.68343	$1.00\text{e} - 07$	10000
19	710.3284	710.3284	52.32841	$1.00\text{e} - 07$	10000
...	...	...	...	...	...
50	295.5159	283.2963	13.57285	$1.00\text{e} - 12$	10000
51	265.0926	265.0926	13.18816	$1.00\text{e} - 12$	10000
52	255.3303	255.3303	12.79462	$1.00\text{e} - 12$	10000
53	229.4370	229.4370	12.10607	$1.00\text{e} - 12$	10000
54	227.0917	227.0917	11.61147	$1.00\text{e} - 12$	10000
55	222.4499	222.4499	10.78217	$1.00\text{e} - 12$	10000
56	199.7345	199.7345	10.27557	$1.00\text{e} - 12$	10000
57	187.3298	187.3298	9.604999	$1.00\text{e} - 12$	10000
58	175.4316	175.4316	8.890889	$1.00\text{e} - 12$	10000
59	145.4870	145.4870	8.219988	$1.00\text{e} - 12$	10000
60	121.5790	121.5790	7.382469	$1.00\text{e} - 12$	10000
61	106.3441	106.3441	6.265843	$1.00\text{e} - 12$	9999
62	106.3441	106.3441	5.293587	$1.00\text{e} - 12$	9579
63	106.3441	106.3441	4.455063	$1.00\text{e} - 12$	6173
64	106.3441	106.3441	4.068666	$1.00\text{e} - 12$	2118
65	106.3441	106.3441	3.108152	$1.00\text{e} - 15$	235

Table 5.8. Case studies for TSP.

file	n	K	$\bar{\mathcal{L}}_{0,(1)}$	$\bar{\mathcal{L}}_{K,(1)}$	$\mathcal{L}_{(1)}$	ε	CPUtime, sec.
ft53	53	85	21266	7111	6905	0.030	73.3
ft70	70	147	65985	38894	38673	0.006	283.8
ftv47	48	78	5448	1795	1776	0.011	46.9
ftv55	56	67	5947	1728	1608	0.075	60.9
ry48p	48	75	42580	15509	14422	0.075	52.3
kro124p	100	112	161000	39712	36230	0.096	643.5

Source. <http://www.iwr.uni-heidelberg.de/iwr/comopt/soft/TSPLIB95/TSPLIBA7>

Summary: Our numerical results with L_{ij} being iid, each distributed according to the exponential family of distributions and same case studies suggest that Algorithm 3.2 performs quite accurately in the sense of (3.61). It is robust with respect to the parameters ρ_0 , r and s in (5.5)–(2.30) for the adaptive ρ . Good performance was obtained if the sample size N at each iteration is taken of order of n^2 . We found empirically that for such $N = n^2$, the number of iterations K required before Algorithm 3.2 stops is of order of n . Thus, the total number of generated trajectories by Algorithm 3.2 is $M = NK = O(n^3)$. In the case of the shortest and the longest path with the model (5.7) (configuration $n \times m$, $m \leq 0.2n$) we obtained good performance when the sample size at each iteration is of order of mn .

REMARK 5.1 *We implemented Algorithm 3.2 for some other Ali-Silvey “distance” measures (1.18), different from the cross-entropy. For example, we implemented the importance sampling counterpart of (3.68), which can be written as*

$$\max_{P_t} \mathcal{L}(\gamma, P_{t-1}^*, P_t) = \max_{P_t} \mathbb{E}_{P_{t-1}^*} \left\{ \prod_{s=1}^{\tau} H_{i_s i_{s+1}}^2(\gamma) \sum_{k=1}^{\tau} \frac{P_{t-1, i_k i_{k+1}}^*}{P_{t, i_k i_{k+1}}} \right\}. \quad (5.9)$$

We found that Algorithm 3.2 based on cross-entropy outperform its “non cross-entropy” counterparts, which is quite remarkable.

6. Concluding Remarks and Directions for Further Research

This paper presents a new and efficient method, called the cross-entropy method, for estimating simultaneously the optimal solution of combinatorial problems and the associated partition functions. Our method solves *analytically* a sequence of simple continuous optimization programs involving cross-entropy and importance sampling. Our numerical studies suggest that the complexity (the total sample size generated) is of order of n^3 . The efficiency of our algorithms was supported by extensive simulation experiments. Some further topics for investigation are listed below.

1. Investigation of the complexity and conditions under which the proposed algorithm converges quickly to the neighborhood of the optimal solution. In other words, investigation of the conditions under which the complexity of the proposed algorithm is bounded by a polynomial with respect to the problem size function.
2. Establishment of confidence intervals (regions) for the optimal solution.
3. Investigation of the convergence rate of different Ali-Silvey “distance” functions given in (1.18).
4. Application of the proposed methodology to a wide variety of combinatorial estimation and optimization problems, such as the quadratic assignment problem, minimal cut, vehicle routing, graph coloring and graph partitioning problems.
5. Presentation of case studies.
6. Application of parallel optimization techniques to the proposed methodology.

As a final remark, it is important to note that we selected Kullback-Leibler’s relative entropy “distance” among many alternative “distances” as per (1.18). It is our

understanding that for the set of the considered combinatorial problems, our choice is the most appropriate, informative, easy to implement, and natural.

7. Appendices

7.1. Appendix A: Kullback-Leibler Cross-Entropy Method

Here via example we show how Kullback-Leibler's cross-entropy method allows to estimate the optimal solution in simple program. This simple example provide some insight to our main algorithms for both continuous and combinatorial optimization.

EXAMPLE 7.1 *Let Y be a discrete random variable with pdf*

$$f(y, \mathbf{p}) = P(Y = y_i) = p_i, \quad i = 1, \dots, n. \quad (7.1)$$

Assume that we can take a sample from the pdf in (7.1), but the values $y_i, i = 1, \dots, n$ are not available in advance and

Assume that we want to estimate the largest value

$$y_{(n)} = \max(y_1, y_2, \dots, y_n). \quad (7.2)$$

of the pdf $P(Y = y_i) = p_i$. Typically one would take the maximum value of the sample $Y_1, \dots, Y_N$ from $P(Y = y_i) = p_i$ as an estimate of $y_{(n)}$. We purposely use Kullback-Leibler cross-entropy to provide some insight to the main ideas of this work.

Consider the following recursive Kullback-Leibler cross-entropy procedure

$$\max_{\mathbf{p}_t} D(\mathbf{p}_t) = \max_{\mathbf{p}_t} \mathbb{E}_{\mathbf{p}_{t-1}^*} \{L(Z_{t-1}) \ln f(Z_{t-1}, \mathbf{p}_t)\} = \max_{\mathbf{p}_t} \left\{ \sum_{k=1}^n p_{t-1,k}^* L(y_k) \ln p_{t,k} \right\}. \quad (7.3)$$

Here $Z_{t-1} \sim f(z, \mathbf{p}_{t-1}^)$ and $f(z, \mathbf{p}_1^*) \equiv f(z, \mathbf{p})$ is the pdf in (7.1).*

Solving (7.3) we obtain for the $t + 1$ -th iteration

$$p_{t+1,i}^* = \frac{p_i [L(y_i)]^t}{\sum_{k=1}^n p_k [L(y_k)]^t}. \quad (7.4)$$

Since the values $y_i, i = 1, \dots, n$ are not available, the solution of the problem (7.4) is likewise not available. We can, however, consider the following stochastic counterpart of (7.3)

$$\max_{\mathbf{p}_t} \widehat{D}(\mathbf{p}_t) = \max_{\mathbf{p}_t} \left\{ N^{-1} \sum_{j=1}^N \sum_{k=1}^n L(Z_{t,j,k}) \ln p_{t,k} \right\}. \quad (7.5)$$

Its solution is

$$\hat{p}_{t+1,i}^* = \frac{\sum_{j=1}^N L(Z_{t,j,i})}{\sum_{j=1}^N \sum_{k=1}^n L(Z_{t,j,k})} \quad (7.6)$$

where $Z_{t,j,k} \sim f(y, \hat{\mathbf{p}}_{t-1})$, and $\sum_{k=1}^n N_k = N$.

Note that

$$Z_{t,j,r} = \begin{cases} y_k & \text{if } r = k, \ r = 1, \dots, n \\ 0 & \text{otherwise.} \end{cases}$$

From (7.4) we obtain that

$$(p_{t,1}^*, p_{t,2}^*, \dots, p_{t,n}^*) \rightarrow (0, 0, \dots, 1)$$

(deterministically) as $t \rightarrow \infty$ and similarly from (7.6) we obtain that

$$(\hat{p}_{t,1}^*, \hat{p}_{t,2}^*, \dots, \hat{p}_{t,n}^*) \rightarrow (0, 0, \dots, 1)$$

(almost surely) as $t \rightarrow \infty$ and $N \rightarrow \infty$. This implies that solving the stochastic counterpart (7.5) with probability 1 we select the largest value $y_{(n)} = \max(y_1, y_2, \dots, y_N)$, provided both t and N approach infinity.

As a simple example, let $L(y_i) = y_i = i$ and $p_i = 1/n$, then we have

$$p_{t+1,i}^* = \frac{i^t}{\sum_{k=1}^n k^t} \quad (7.7)$$

and

$$\hat{p}_{t+1,i}^* = \frac{\sum_{j=1}^N Z_{t,j,i}}{\sum_{j=1}^N \sum_{k=1}^n Z_{t,j,k}}, \quad (7.8)$$

respectively. It is not difficult to see that if n is large then in order to obtain

$$\hat{p}_{t+1,n}^* \geq 1 - \varepsilon,$$

both N and t must be typically quite substantial.

To speed up the convergence of (7.6) to $(\hat{p}_{t,1}^*, \hat{p}_{t,2}^*, \dots, \hat{p}_{t,n}^*) \rightarrow (0, 0, \dots, 1)$, we can take $L(y_k)$ in (7.3) as

$$L(y_k) = H^+(y_k, \gamma) = \exp\left(\frac{y_k}{\gamma}\right), \quad \gamma > 0, \quad (7.9)$$

which presents a Boltzmann-type function. In this case (7.3) becomes

$$\max_{\mathbf{p}_t} D(\mathbf{p}_t) = \max_{\mathbf{p}_t} \sum_{k=1}^n p_{t-1,k}^* \exp\left(\frac{y_k}{\gamma}\right) \ln p_{t,k}. \quad (7.10)$$

The optimal solution of the stochastic counterpart of (7.10) can be written in analogy to (7.6) as

$$\hat{p}_{t+1,i}^* = \frac{\sum_{j=1}^N \exp\left(\frac{Z_{t,j,i}}{\gamma}\right)}{\sum_{j=1}^N \sum_{k=1}^n \exp\left(\frac{Z_{t,j,k}}{\gamma}\right)}. \quad (7.11)$$

It is readily seen that the speed of convergence of $(\hat{p}_{t,1}^*, \hat{p}_{t,2}^*, \dots, \hat{p}_{t,n}^*)$ in (7.11) to

$(0, 0, \dots, 1)$ might be quite substantial, in particular, when γ is very small. In fact, it is straightforward to see that for finite n ; all $p_k > 0$, $k = 1, \dots, n$; $y_{(n)} - y_{(n-1)} = \eta > 0$ and $\varepsilon > 0$, one can obtain $\hat{p}_{t+1,n}^* \geq 1 - \varepsilon$ in a single iteration with the stochastic counterpart of (7.10) by taking a very large sample N and keeping γ small enough.

If we are interested in finding

$$y_{(1)} = \min(y_1, y_2, \dots, y_n). \quad (7.12)$$

rather than $y_{(n)}$ as per (7.2), we can use instead of (7.9) the following Boltzmann-type function

$$H(y_k, \gamma) = \exp\left(-\frac{y_k}{\gamma}\right), \quad \gamma > 0. \quad (7.13)$$

In this case, for fixed γ , the number of iterations t needed in order to get $p_{t-1,1}^* \geq 1 - \varepsilon$ can be obtained from the solution of the following system

$$\frac{p_1 \exp\left(-\frac{y_1}{\gamma}\right)}{\sum_{k=1}^n p_k \exp\left(-\frac{y_k}{\gamma}\right)} \geq 1 - \varepsilon. \quad (7.14)$$

Clearly, that $t(\gamma)$ is monotone increasing in γ and it is not difficult to find a small γ such that $p_{t-1,1}^* \geq 1 - \varepsilon$ satisfies in a single iteration. As example, let $y_k = k$ and $p_k = 1/n$, then straightforward calculation show that $p_{t-1,1}^* \geq 1 - \varepsilon$ holds (in a single iteration), if γ is derived from the following equation

$$e^{-\frac{1}{\gamma}} \approx e^{-\frac{n}{\gamma}} + \varepsilon.$$

Note, however, that while sampling from the distribution of Y , the convergence of the estimates $\hat{p}_{t,i}^*$ to the true values $p_{t,i}^*$ in a single iteration might be problematic. There reason is that for very small γ , the estimates $\hat{p}_{t,i}^*$ is typically be unreliable. To avoid such situation, multiple iteration must be used instead. For more details see (3.71) and (3.72).

Since the above results are relevant to our main procedure (3.69) for combinatorial optimization, we summarize it as

LEMMA 7.1 Consider the CE procedure (7.10). Let n be finite, all $p_k \geq p > 0$, $k = 1, \dots, n$, $y_{(n)} - y_{(n-1)} = \eta > 0$ and $\varepsilon > 0$. Then

1. For fixed γ the inequality $p_{t-1,n}^* \geq 1 - \varepsilon$ holds after a finite number of iterations t with (7.10).
2. There exist $\gamma = \gamma(\varepsilon, \eta, p) > 0$ (small enough), such that the inequality $p_{t-1,n}^* \geq 1 - \varepsilon$ holds after a single iteration with (7.10).
3. Both 1 and 2 hold for the stochastic counterpart of (7.10), provided $N(\varepsilon) \rightarrow \infty$ as $\varepsilon \rightarrow 0$.

Consider finally the following cross-entropy procedure

$$\max_{\mathbf{v}_t} D(\mathbf{v}_t, \mathbf{v}_{t-1}^*) = \max_{\mathbf{v}_t} \mathbb{E}_{\mathbf{v}_{t-1}^*} \{L(\mathbf{Z}) \ln f(\mathbf{Z}, \mathbf{v}_t)\}, \quad (7.15)$$

which presents a continuous version of (7.3). Here $\mathbf{Z} \sim f(\mathbf{z}, \mathbf{v}_{t-1}^*)$ and $f(\mathbf{z}, \mathbf{v}_0^*) = f(\mathbf{z}, \mathbf{v}_0)$.

It can be shown in analogy to Proposition 9.4.1, pp. 329–330 of Rubinstein and Melamed (1998), (see also 7.1), that if $L(\mathbf{y})$ is monotone increasing (decreasing) at each component of the vector $\mathbf{y}$ and $\mathbf{Y}$ is distributed according to the exponential family, then as $t \rightarrow \infty$, the random variable $L(\mathbf{Z})$ in (7.15) converges in probability to the largest (smallest) value of $L(\mathbf{y})$. We shall use a program of type (7.15) for continuous global optimization in Section 2.

7.2. Appendix B: Alias Method

The most common procedure for generating discrete variables is the inverse-transform method Rubinstein and Melamed (1998). An alternative approach for generating any discrete random variable with a *finite* range of values is the so-called *alias method*, due to Walker (1977) and is based on the fact that any arbitrary n -point pdf

$$f = P(X = x_i) = p_i, i = 1, \dots, n$$

can be represented as an *equally weighted mixture* of $n - 1$ discrete pdf's, $g^{(k)}$, $k = 1, \dots, n - 1$, each having at most *two* non-zero components. That is, any n -point pdf f can be represented as

$$f = \frac{1}{n-1} \sum_{k=1}^{n-1} g^{(k)} \quad (7.16)$$

for suitably defined 2-point pdf's $g^{(k)}$, $k = 1, \dots, n - 1$ (see Walker (1977)).

The alias method is rather general and efficient, but requires an initial setup and extra storage for the $n - 1$ pdf's, $g^{(k)}$. A procedure for computing these two-point pdf's, $g^{(k)}$, $k = 1, \dots, n - 1$, can be found in Devroye (1986).

Once the representation (7.16) has been established, generation from f is simple and can be written as:

ALGORITHM 7.1

1. Generate a random variate U from the discrete uniform pmf $U(1, n - 1)$. Let u ($u = 1, \dots, n - 1$) be the outcome.
2. Generate a random variate X from the two-point pdf $g^{(k)}$, $k = 1, \dots, n - 1$.

Acknowledgment

I would like to thank Aharon Ben-Tal for helpful discussion, Eitan Rubinstein for providing us with the results of Remark 3.1 and Alexander Podgaetsky for performing the computational part of this work.

References

- E. H. L. Aarts and J. H. M. Korst, *Simulated Annealing and Boltzmann Machines*, John Wiley & Sons: 1989.
- E. H. L. Aarts and J. K. Lenstra, *Local Search in Combinatorial Optimization*, Wiley: Chichester, 1997.
- W. A. T. W. Abdullah, "Seeking Global Minima," *Journal of Computational Physics* vol. 110 p. 320, 1994.
- R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network Flows Theory, Algorithms and Applications*, Prentice Hall: 1993.
- S. M. Ali and S. D. Silvey, "A General Class of Coefficients of Divergence of one Distribution from Another" *Journal of the Royal Statistical Society, Series B* vol. 28, pp. 131–142, 1966.
- S. Andradóttir, "A Method for Discrete Stochastic Optimization," *Management Science* vol. 41 pp. 1946–1961, 1995.
- S. Andradóttir, "A Global Search Method for Discrete Stochastic Optimization," *SIAM J. Optimization* vol. 6 pp. 513–530, 1996.
- S. Asmussen, *Applied Probability and Queues*, John Wiley & Sons: 1987.
- H. Cohn and M. Fielding, "Simulated Annealing Searching for an Optimal Temperature Schedule," 1999 (to appear in SIAM journal of optimization).
- L. Devroye, *Non-Uniform Random Variate Generation*, Springer: New York, 1986.
- M. Dyer, A. Frieze, and R. Kannan, "A Random Polynomial-Time Algorithm for Approximation the Volume of Convex Bodies" *Journal on ACM* vol. 38 pp. 1–17, 1991.
- B. L. Fox and G. W. Heine, "Probabilistic Search with Overrides," *Annals of Applied Probability* vol. 6 pp. 1087–1094, 1996.
- C. J. Geyer and E. A. Thompson, "Annealing Markov Chain Monte-Carlo with Applications to to Ancestral Inference" *Journal of the American Statistical Association* vol. 90 pp. 909–920, 1995.
- W. R. Gilks, S. Richardson, and D. J. Spiegelhalter, *Markov Chain Monte Carlo in Practice*, Chapman & Hall: 1996.
- F. Glover and M. Laguna, "Tabu Search," a chapter in *Modern Heuristic Techniques for Combinatorial Optimization*, 1992.
- D. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*, Addison Wesley, MA., 1989.
- W. B. Gong, Y. C. Ho, and W. Zhai "Stochastic Comparison Algorithm for Discrete Optimization with Estimation," *Proceedings of the 31st IEEE Conference on Decision and Control*, pp. 795–800, 1992.
- W. J. Gutjahr and G. Ch. Pflug, "Simulated Annealing for Noisy Cost Functions," *Journal of Global Optimization* vol. 8 pp. 1–13, 1996.
- W. K. Hastings, "Monte Carlo Sampling Methods using Markov Chains and their Applications," *Biometrika* vol. 57 pp. 92–109, 1970.
- R. Horst, P. M. Pardalos, and N. V. Thoai, *Introduction to Global Optimization*, Kluwer Academic Publishers: 1996.
- M. R. Jerrum and A. J. Sinclair, "Approximating the Permanent," *SIAM Journal on Computing* vol. 18 pp. 1149–1178, 1989.
- M. R. Jerrum and A. J. Sinclair, "Polynomial-Time Approximation Algorithms for the Ising Model," *SIAM Journal on Computing* vol. 22 pp. 1087–1116, 1993.
- N. L. Johnson and S. Kotz, *Urn Models and Their Applications: An Approach to Modern Discrete Probability Theory*, John Wiley and Sons, Inc.: 1977.
- R. M. Karp and M. Luby, "Monte-Carlo Algorithms for Enumeration and Reliability Problems," *Proceedings of the 24th Annual IEEE Symposium on Foundations of Computer Science* pp. 56–64, 1983.
- J. N. Kapur and H. K. Kesavan, *Entropy Optimization Principles with Applications*, Academic Press: 1992.
- S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by Simulated Annealing," *Science* vol. 220 pp. 671–680, 1983.
- D. Lieber, *Rare Events Estimation Via Cross-Entropy and Importance Sampling*, PHD Thesis, William Davidson Faculty of Industrial Engineering and Management, Technion, Haifa, Israel, 1999.
- D. Lieber, R. Y. Rubinstein, and D. Elmakis, "Quick Estimation of Rare events in Stochastic Networks," *IEEE, Transactions on Reliability* vol. 46(2) pp. 254–265, 1997.

- L. Lovasz, "Randomized Algorithms in Combinatorial Optimization," *DIMACS Series in Discrete Mathematics and Theoretical Computer Science* vol. 25, pp. 153–179, 1995.
- E. Marinari and G. Parisi, "Simulated Tempering: A New Monte Carlo Scheme," *Europhysics letters* vol. 19 pp. 451–458, 1992.
- R. Mead and J. A. Nedler, "A Simplex Method for Function Minimization," *Computer Journal* vol. 7 pp. 308–313, 1965.
- M. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller "Equations of State Calculations by Fast Computing Machines," *J. of Chemical Physics* vol. 21 pp. 1087–1092, 1953.
- W. I. Norkin, G. C. Pflug, and A. Ruszczyński, "A Branch and Bound Method for Stochastic Global Optimization". Working paper, International Institute for Applied System Analysis, WP-96-065, Laxenburg, Austria, 1996.
- I. H. Osman and G. Laporte, "Metaheuristics: A bibliography," *Annals of Operations Research* vol. 63 pp. 513–523, 1996.
- J. D. Pinter, *Global Optimization in Action*, Kluwer Academic Publishers: 1996.
- R. G. Parker and R. L. Rardin, *Discrete Optimization*, Academic Press: San Diego, 1988.
- G. Potamianos and J. Goutsias, "Stochastic Approximation Algorithms for Partition Function Estimation of Gibbs Random Fields," *IEEE Transactions on Information Theory* vol. 43(6) pp. 1948–1965, 1997.
- V. J. Rayward-Smith, I. H. Osman, C. R. Reeves, and G. D. Smith, *Modern Heuristic Search Methods*, Wiley: Chichester, 1996.
- C. R. Reeves, Modern heuristic techniques, in: *Modern Heuristic Search Methods* (eds. V. J. Rayward-Smith, I. H. Osman, C. R. Reeves and G. D. Smith), Wiley: Chichester, 1996.
- H. E. Romeijn and R. L. Smith, "Simulated Annealing for Constrained Global Optimization," *Journal of Global Optimization* vol. 5 pp. 101–126, 1994.
- R. Y. Rubinstein, *Simulation and the Monte Carlo Methods*, John Wiley and Sons, Inc., 1981.
- R. Y. Rubinstein, "Optimization of Computer Simulation Models with Rare Events," *European Journal of Operations Research* vol. 99 pp. 89–112, 1997.
- R. Y. Rubinstein and B. Melamed, *Efficient Simulation and Modeling*, John Wiley & Sons: 1998.
- R. Y. Rubinstein and A. Shapiro, *Discrete Event Systems: Sensitivity Analysis and Stochastic Optimization via the Score Function Method*, John Wiley & Sons: 1993.
- L. Shi and S. Olafsson, "Nested Partitioning Method for Global Optimization" Manuscript, Department of Industrial Engineering, University of Wisconsin-Madison.
- A. J. Walker, "An Efficient Method for Generating Discrete Random Variables with General Distributions," *Assoc. Comput. Mach. Trans. Math. Software* vol. 3, pp. 253–256, 1977.